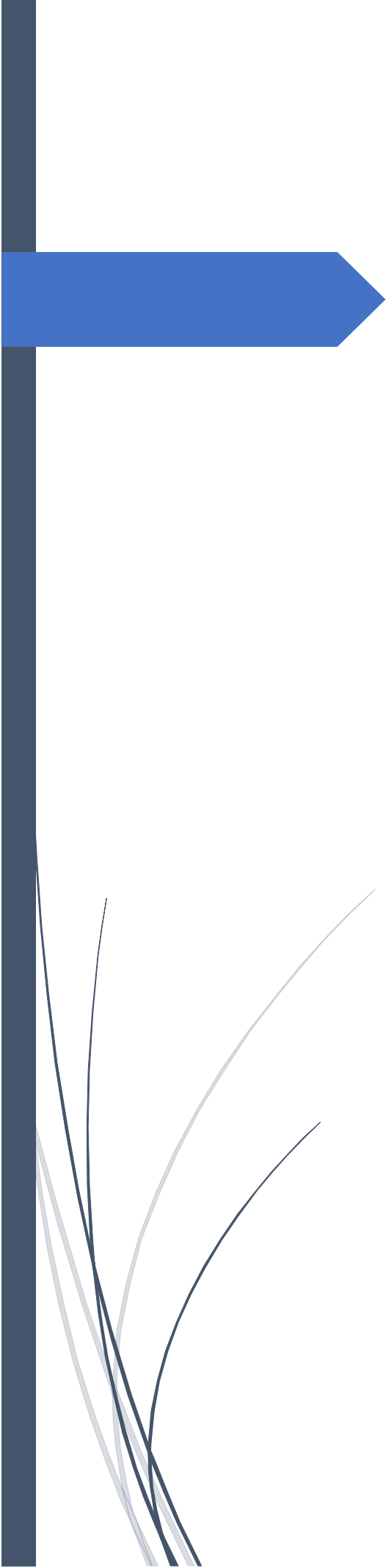




Algoritmi semplici (per davvero)

Rovesti Gabriel

Attenzione



Il file non ha alcuna pretesa di correttezza; di fatto, è una riscrittura attenta di appunti, slide, materiale sparso in rete, approfondimenti personali dettagliati al meglio delle mie capacità. Credo comunque che, per scopo didattico e di piacere di imparare (sì, io studio per quello e non solo per l'esame) questo file possa essere utile. Semplice si pone, per davvero ci prova.
Thank me sometimes, it won't kill you that much.

Gabriel

Sommario

Lezione 1: Algoritmi: definizione e ordinamenti	3
Lezione 2: Merge sort: analisi e complessità	6
Lezione 4: Ricorrenze e master theorem.....	17
Lezione 5: Heap sort: analisi e complessità. Code con priorità e metodi di utilità. Heap e tipi (alberi e procedure)	28
Lezione 6: Quicksort	35
Lezione 7: Counting Sort.....	44
Lezione 8: Radix Sort.....	49
Come definire (buone) funzioni di hash?	54
Open Addressing.....	55
Hashing uniforme	57
Lezione 10: Alberi binari di ricerca: descrizione ed operazioni	60
Lezione 11: Red-Black Trees	65
Lezione 12: Arricchimento di strutture dati e conclusioni su RB-Trees.....	75
Inizio Parte Scquizzato – Lezione 13: Programmazione dinamica.....	79
Lezione 14 – Memoizzazione e problemi di ottimizzazione	81
Lezione 15: Longest Common Subsequence (LCS)	84
Lezione 16: Longest Increasing Subsequence	92
Lezione 17: Shortest Palindrome Completion	96
Lezione 18: Algoritmi greedy	100
Lezione 19: Compressione dei dati e Huffman ed esempi di algoritmi greedy	105
Confronto programmazione dinamica e greedy	113
Chiarimento sul calcolo esatto delle somme nelle serie	116

Lezione 1: Algoritmi: definizione e ordinamenti

Algoritmo → Procedura che descrive tramite passi elementari come risolvere un problema (tramite un modello di computazione).

Uno stesso problema può essere risolto da diversi algoritmi.

Di ogni algoritmo siamo interessati a conoscere:

- la correttezza
- la stabilità
- la complessità (quanto tempo ci mette ad eseguire/quanto spazio occupa)

Iniziamo analizzando gli algoritmi di ordinamento, in particolare analizzeremo:

- insertion sort (algoritmo incrementale, complessità n^2)
- merge sort (divide-et-impera, complessità $n \log n$)

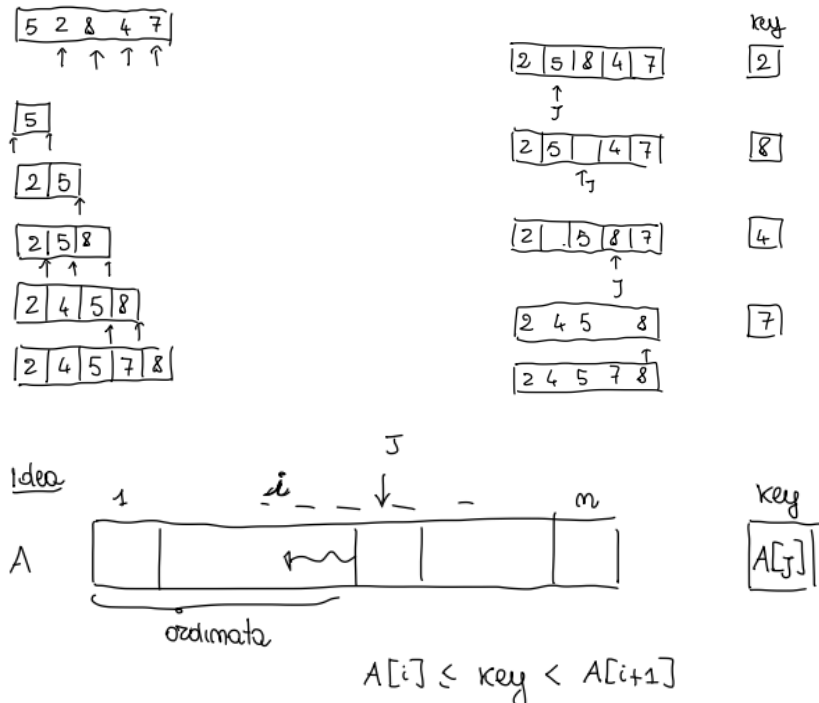
Consideriamo prima di tutto l'insertion sort.

Esso è un semplice algoritmo di ordinamento che funziona in modo simile a quello delle carte da gioco. L'array viene virtualmente diviso in una parte ordinata e una non ordinata. I valori della parte non ordinata vengono prelevati e collocati nella posizione corretta della parte ordinata.

Caratteristiche dell'ordinamento per inserzione:

- Questo algoritmo è uno dei più semplici e di semplice implementazione.
- Fondamentalmente, l'ordinamento per inserzione è efficiente per piccoli valori di dati
- L'ordinamento per inserzione è di natura adattativa, cioè è adatto a insiemi di dati già parzialmente ordinati.

Quello che sostanzialmente fa è esaminare gli elementi a coppie e ordinarli gradualmente per come si presentano.



Per delineare l'algoritmo, useremo una struttura definita *pseudocodice*, che permette di implementare facilmente in maniera visiva il codice per poi essere genericamente implementato in un qualsiasi linguaggio di programmazione.

INSERTION-SORT(*A*)

```

1  n = A.length
2  for j = 2 to n // il primo elemento è già ordinato
3      key = A[j] // A[1 .. j - 1] ordinato
4      i = j - 1
5      while (i > 0) and (A[i] > key)
6          A[i + 1] = A[i]
7          i = i - 1
8      A[i + 1] = key

```

Per esaminare la correttezza di un algoritmo, dobbiamo definire quello che viene chiamato invariante, pertanto una condizione che rimane vera per tutta la durata del ciclo, dall'inizio fino alla fine e anche durante l'esecuzione stessa del codice.

In particolare, possiamo definire i seguenti passi, valenti per *A* da indice 1 ad indice "*j*-1":

- Inizializzazione: *j* = 2, *A*[1,1] ordinato
- Mantenimento: "Inserisce *A*[*j*] in *A*[1 ... *j* - 1] ordinato"
- Cancellazione: *j* = *n* + 1

Animazione del funzionamento:

<https://upload.wikimedia.org/wikipedia/commons/9/9c/Insertion-sort-example.gif>

Astrazione Prendiamo ad esempio il seguente array:

5	2	8	4	7
---	---	---	---	---

Partiamo dal primo elemento: 5. È già ordinato con se stesso, quindi procediamo con il secondo elemento.

Confronto il numero 2 con l'elemento alla sua sinistra:

$2 \geq 5$? No, quindi lo inverto con l'elemento alla sua sinistra, come segue

2	5	8	4	7
---	---	---	---	---

Key:

8

La key analizzata è 8.

$8 \geq 5$? Sì, quindi è ordinato in modo corretto.

2	5	8	4	7
---	---	---	---	---

Key:

4

La key analizzata è 4.

$4 \geq 8$? No, quindi lo sposto a sinistra invertendolo con 8.

$4 \geq 5$? No, lo sposto a sinistra invertendolo con 5.

$4 \geq 2$? Sì, quindi è nella posizione corretta.

2	4	5	8	7
---	---	---	---	---

Key:

7

Key analizzata 7.

$7 \geq 8$? No, lo sposto a sinistra invertendolo con 8.

$7 \geq 5$? Sì, è nella posizione corretta.

Otengo l'array ordinato:

2	4	5	7	8
---	---	---	---	---

Ora ci chiediamo: quanto costa l'esecuzione? Ciò dipende:

- dal tipo di operazioni fatte
- dalla dimensione del problema di analisi

Scritto da Gabriel

Nel caso in oggetto:

- tutte le operazioni elementari (assegnazione/operazioni aritmetiche) hanno costo costante
- i cicli hanno un costo pari ad una serie/sommatoria, dato che si ha un possibile numero massimo di iterazioni

In particolare (solo qui faremo un'analisi completa):

INSERTION-SORT(A)

```

1   $n = A.length$ 
2  for  $j = 2$  to  $n$  // il primo elemento è già ordinato
3       $key = A[j]$  //  $A[1..j-1]$  ordinato
4       $i = j - 1$ 
5      while ( $i > 0$ ) and ( $A[i] > key$ )
6           $A[i+1] = A[i]$ 
7           $i = i - 1$ 
8       $A[i+1] = key$ 
    
```

$$c_0 \rightarrow 1$$

$$c_1 \rightarrow 1$$

$$c_2 \rightarrow n$$

$$c_3 \rightarrow (n-1)$$

$$c_4 \rightarrow (n-1)$$

$$c_5 \rightarrow \sum_{j=2}^n t_j + 1$$

$$c_6, c_7 \rightarrow \sum_{j=2}^n t_j$$

$$c_8 \rightarrow (n-1)$$

Casi di interesse:

$$T^{IS}(n) = c_0 + c_1 + c_2 n + (c_3 + c_4 + c_8)(n-1) + c_5 \sum_{j=2}^n (t_j + 1) + (c_6 + c_7) \sum_{j=2}^n t_j$$

- Caso migliore
- Caso peggiore
- Caso medio

1) Caso migliore $\rightarrow A$ è già ordinato.

Non è di molto interesse analizzare il caso migliore in quanto improbabile.

$$T_{min}^{IS}(n) = c_0 + c_1 + c_2 n + (c_3 + c_4 + c_5 + c_8)(n-1) = an + b \approx n$$

2) Caso peggiore $\rightarrow$ Limite superiore al tempo di esecuzione, es. array ordinato in modo decrescente.

La complessità diventa:

$$T_{max}^{IS}(n) = c_0 + c_1 + c_2 n + (c_3 + c_4 + c_8)(n-1) + c_5 \sum_{j=2}^n j + (c_6 + c_7) \sum_{j=2}^n (j-1)$$

Essendo due somme (j) e ($j-1$), la loro somma è data dalla somma di Gauss come segue (utile perché ci sarà utile nella prog. dinamica)

$$\rightarrow \sum_{j=2}^n j \text{ e di } \sum_{j=2}^n (j-1) \quad \sum_{i=1}^n i = \frac{n(n+1)}{2}$$

3) Caso medio $\rightarrow$ Difficile da calcolare e coincide nella maggior parte dei casi con il caso peggiore. In questo caso, letteralmente si considera il costo medio, ma sostituendo l'indice j .

4)

$$\frac{\sum_{\text{perm. di input}} T^{IS}(p)}{n!} \approx n^2 \quad \text{posso pensare che } t_j \cong \frac{j-1}{2}$$

A questo livello, la complessità è data quindi:

$$T_{max}^{IS}(n) = a'n^2 + b'n + c' \approx n^2$$

Comunque, grazie alla presenza di due soli cicli, si può già capirlo da sé. Ogni ciclo ha complessità n (lineare, per tutte le iterazioni; avendone 2, ecco che abbiamo complessità n^2).

Lezione 2: Merge sort: analisi e complessità

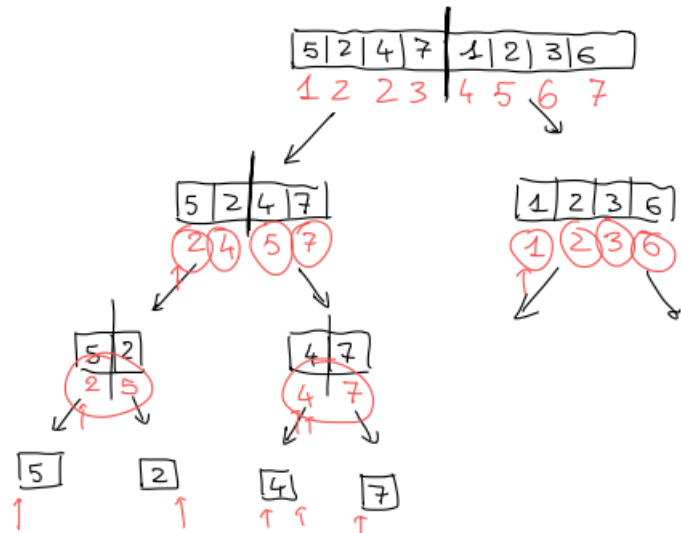
Ora, analizziamo l'algoritmo merge sort.

Esso adotta l'approccio *divide et impera*, quindi:

- divide (prende il problema originale e lo divide in problemi più piccoli)
- impera (ricorsivamente, risolve i sottoproblemi; se abbastanza piccolo, si risolve subito)
- combina (le soluzioni di $P_1, \dots, P_k$ si combinano in una soluzione di P).

Quindi, mergesort adotta i seguenti passi:

- dividere l'array in due parti
- ordina i sottoarray
- fonde i sottoarray ordinati



Sostanzialmente, divide del tutto e poi riordina a mano a mano, come si vede estesamente sotto.

Link animazioni:

<https://commons.wikimedia.org/wiki/File:Merge-sort-example-300px.gif>

Astrazione Consideriamo il seguente array A.

5	2	4	7	1	2	3	6
---	---	---	---	---	---	---	---

Lo divido a metà, ottenendo due parti separate.

5	2	4	7
---	---	---	---

1	2	3	6
---	---	---	---

Consideriamo il primo, ossia A[1..4] (A originale). Divido anche questo a metà.

5	2
---	---

4	7
---	---

Divido nuovamente a metà, ottenendo:

5

2

5 e 2 sono due blocchi già ordinati. Scelgo il minore tra i due e lo metto in prima posizione, mentre l'altro in seconda posizione, ottenendo un blocco composto da 2 e 5.

Riprendo con il blocco composto da 4 e 7. Lo divido in due blocchi da un elemento. Faccio lo stesso procedimento fatto per 2 e 5: metto in prima posizione 4 e in seconda posizione 7. La situazione è la seguente:

2	5
---	---

4	7
---	---

So che i blocchi ottenuti contengono elementi ordinati. Con questa assunzione, posso ragionare nel seguente modo: considero il primo elemento dei due blocchi (2 e 4 in questo caso) e metto in prima posizione il minore tra i due. Ora considero il successivo elemento del blocco che è stato scelto e lo stesso elemento dell'altro blocco, e inserisco nell'array l'elemento minore. Continuo fino ad ottenere un blocco ordinato.

2	4	5	7
---	---	---	---

Faccio lo stesso procedimento con la parte di array originale $A[5..8]$, ottenendo

2	4	5	7
---	---	---	---

1	2	3	6
---	---	---	---

Lo pseudocodice è articolato in due parti:

- un codice che chiama la procedura *MERGE – SORT* in grado di eseguire quanto descritto sopra.

In pratica:

- o prende la metà dell'array
 - o passa a *MERGE – SORT* la prima metà
 - o passa a *MERGE – SORT* la seconda metà
 - o esegue il merge dei due sotto-array
 - un codice *MERGE* che prende i singoli pezzi dell'array e li ordina
- In pratica:
- o inizializza due indici n_1 ed n_2 per tenere conto dei due array
 - o crea copie duplicate dei sottoarray che devono essere ordinate, cioè L ed R
 - o mantiene gli indici i, j, k dei sottoarray e all'array principale
 - o Finché non si raggiunge la fine di L o di R, si sceglie il più grande tra gli elementi L e M e lo si colloca nella posizione corretta in $A[p..r]$.
 - o Quando si esauriscono gli elementi in L o in M, si raccolgono gli elementi rimanenti e li si colloca in $A[p..r]$.

MERGE-SORT(A, p, r)

```

1  if  $p < r$ 
2       $q = \lfloor \frac{p+r}{2} \rfloor$  // arrotondato per difetto
3      MERGE-SORT( $A, p, q$ ) // ordina  $A[p..q]$ 
4      MERGE-SORT( $A, q+1, r$ ) // ordina  $A[q+1..r]$ 
5      MERGE( $A, p, q, r$ ) // "Merge" dei due sotto-array

```

MERGE(A, p, q, r)

```

1   $n_1 = q - p + 1$  // gli indici partono da 1
2   $n_2 = r - q$ 
   // L sotto-array sx, R sotto-array dx
3  for  $i = 1$  to  $n_1$ 
4       $L[i] = A[p + i - 1]$ 
5  for  $j = 1$  to  $n_2$ 
6       $R[j] = A[q + j]$ 
7   $L[n_1 + 1] = R[n_2 + 1] = \infty$ 
8   $i = j = 1$ 
9  for  $k = p$  to  $r$ 
10     if  $L[i] \leq R[j]$ 
11          $A[k] = L[i]$ 
12          $i = i + 1$ 
13     else //  $L[i] > R[j]$ 
14          $A[k] = R[j]$ 
15          $j = j + 1$ 

```

In merito alla correttezza:

- $A[p \dots k-1]$ contiene $L[1, \dots, i-1]$ e $R[i \dots j-1]$
- $A[p \dots k-1] \leq L[1, \dots, n_1-1]$ e $R[i \dots n_2-1]$ è ordinato

Inizializzazione:

- $k = p$ $A[p \dots k-1] = A[p, p-1]$

Mantenimento

- Divisione in due metà progressive dell'array

Conclusione:

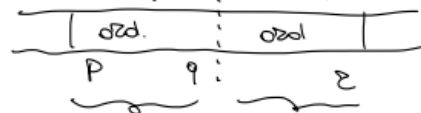
- $K = r + 1$ $A[p \dots r]$ è ordinato e contiene i più piccoli elementi $L[1 \dots n_1 + 1]$ e $R[1 \dots n_2 + 1]$

La dimostrazione del perché sia corretto viene fatta per induzione: se vale per il primo elemento, vale per tutti i casi successivi.

Induzione su $z - p = l$

$(l = 0)$ MergeSort non fa niente ok $A[p..z]$ ha al più un elemento e quindi è già ordinato

$(l > 0)$ si divide l'array in due parti



MergeSort MergeSort

$$q - p < l$$

$\Rightarrow$ per ipotesi
induttiva viene
ordinato

$A[p..q]$ e $A[q+1..z]$ sono ordinati

$\Rightarrow$ la chiamata Merge(A, p, q, z) ordina $A[p..z]$.

Induzione : per dim. che $P(n)$ per ogni n
 $P(0)$

per ogni n $P(n) \implies P(n+1)$

Lezione 3: Principio di induzione e notazioni asintotiche

Normalmente, l'induzione si struttura con:

- Un caso base (vale all'inizio, $P(0)$)
- Un'ipotesi induttiva (se vale per $P(n)$ allora vale anche per $P(n+1)$)

Matematicamente parlando significa che:

- Normalmente ho una formula, ci butto ad esempio $n=1$;
- Se vale per $n=1$, provo con $n=2$
- Se funziona anche con $n=2$, vuol dire che per $P(n)$ varrà anche $P(n+1)$ e tutti i successivi

Si ha anche un'ulteriore variante, l'*induzione forte*.

- U contiene 1 oppure 0
- Se U contiene tutti i numeri minori di n allora contiene anche n

La parola "forte" è legata al fatto che questa formulazione richiede delle ipotesi apparentemente più forti e stringenti per inferire che l'insieme U coincida con $\mathbf{N}$ per ammettere un numero nell'insieme è richiesto infatti che tutti i precedenti ne facciano già parte (e non solo il numero precedente).

Contorno teorico che lascia il tempo che trova.

Per analizzare la correttezza di merge-sort, occorre partire dagli *alberi binari*, struttura che approfondiremo in seguito. A noi interessa sapere ora che:

- è composto da una radice
- l'*altezza* rappresenta il cammino più lungo da un *nodo* (elemento) cosiddetto *radice* (il primo elemento, quello che sta in cima) fino alle *foglie* (che sono i nodi che stanno *più in basso* e non hanno figli, dunque altri nodi sotto di loro)

In particolare, questi hanno come proprietà:

$$P(m) \equiv \text{" per ogni } T \text{ albero di altezza } m \\ \text{ ha } \# \text{ foglie}(T) \leq 2^m \text{ "}$$

Analizziamo tutti i casi per la correttezza, che sintetizzo qui di seguito:

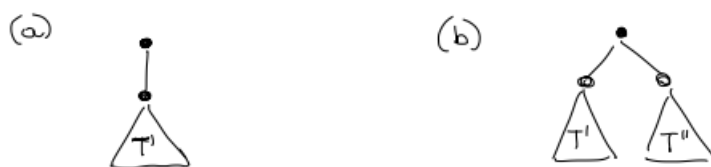
- per il caso base, albero con un solo nodo ($2^0 = 1$)
- per il caso induttivo, albero di altezza $n+1$ ho due casi:
 - o l'albero è composto da un numero bilanciato di foglie (quindi di sicuro va bene, riusciamo a iterare su tutte)
 - o l'albero è composto da almeno un sottoalbero sbilanciato (sappiamo che uno sarà minore dell'altro, ma potenzialmente non sappiamo se la somma del numero di foglie di entrambi rispetti il "limite" N che le contiene tutte)

per induzione

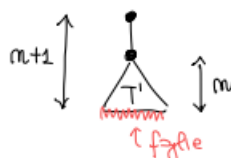
$$(P(0)) \quad m=0 \quad \text{l'albero } T \text{ ha la forma } \bullet \\ \# \text{ foglie}(T) = 1 \leq 2^0 = 1$$

$(m \rightarrow m+1)$ ip. induttiva gli alberi di altezza m hanno un num. foglie $\leq 2^m$

sia T di altezza $m+1$: 2 possibilità:

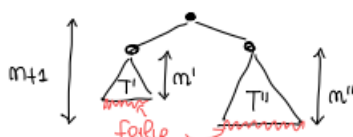


(caso a) $T \in$



$$\# \text{ foglie}(T) = \# \text{ foglie}(T') \leq 2^m \leq 2^{m+1}$$

(caso b) $T \in$



$$m', m'' < m+1$$

$$\# \text{ foglie}(T) = \# \text{ foglie}(T') + \# \text{ foglie}(T'')$$

non posso concludere dato che T' o T'' possono avere altezza $< n$

con induzione completa:

$$\begin{aligned} \# \text{ foglie}(T') &\leq 2^{m'} \\ \# \text{ foglie}(T'') &\leq 2^{m''} \end{aligned} \quad \text{ipotesi induttiva}$$

$$\begin{aligned} \# \text{ foglie}(T) &= \# \text{ foglie}(T') + \# \text{ foglie}(T'') \\ &\leq 2^{m'} + 2^{m''} \leq 2^n + 2^n = 2 \cdot 2^n = 2^{n+1} \end{aligned}$$

$m', m'' \leq n$

Ora, perché parlare di alberi? Dato che abbiamo a che fare con un problema divide et impera, dobbiamo proprio trattare una struttura ad albero, dato che come si può aver già notato, agisce proprio ad albero come procedura.

In particolare, un albero ha di per sé, non essendo normalmente bilanciato, una complessità logaritmica (primo passaggio dell'algoritmo); linearmente, poi, si ricompone tutto l'array (secondo passaggio dell'algoritmo).

La correttezza e la complessità di Mergesort sono così delineate:

* Correttezza MergeSort

MergeSort (A, p, r)

if $p < r$

$$q = \lfloor \frac{p+r}{2} \rfloor$$

MergeSort (A, p, q) *1*

MergeSort ($A, q+1, r$) *2*

Merge (A, p, q, r) *3*

dim. che se chiamo MergeSort (A, p, r) dopo 3 chiamate

il sottoarray $A[p..r]$ è ordinato

dim. per induzione su $r-p$

$(r-p = 0)$ l'array $A[p..r]$ con $p=r$ ha 1 solo elemento

MergeSort non fa niente OK

$(r-p > 0)$ divide $A[p..r]$ $q = \lfloor \frac{p+r}{2} \rfloor$

in due parti $A[p..q]$ $q-p < r-p$
 $A[q+1..r]$ $r-q-1$

quindi per ipotesi induttiva

la chiamare $*1*$ e $*2*$ ordiniamo $A[p..q]$
 $A[q+1..z]$

quindi Merge(A, p, q, z) ordina $A[p..z]$

Merge(A, p, q, r)

inizializzazione: $a'n + b'$;

ciclo: $a'n + b'$;

Sommandoli, ottengo una complessità all'incirca di:

$$T^{\text{merge}}(n) = an + b$$

Nel dettaglio:

$$T^{MS}(n) = \begin{cases} c_0 & \text{se } n \leq 1 \\ T^{MS}(n_1) + T^{MS}(n_2) + T^{\text{merge}}(n) & \text{altrimenti} \end{cases}$$

↓

$$T^{MS}(n) = \begin{cases} c_0 & \text{se } n \leq 1 \\ T^{MS}(n_1) + T^{MS}(n_2) + an + b & \text{altrimenti} \end{cases}$$

* Analisi

* Merge

m = dimensione di $A[p..z]$

Merge(A, p, q, z)

$m_1 = q - p + 1$

$m_2 = z - q$

crea $L[1..m_1+1]$

$R[1..m_2+1]$

for $i = 1$ to m_1
 $L[i] = A[p+i-1]$

for $j = 1$ to m_2
 $R[j] = A[q+j]$

$L[m_1+1] = R[m_2+1] = \infty$

copia $A[p..q] \rightarrow L$
 $A[q+1..z] \rightarrow R$

$a'm + b'$ a', b' costanti opportune

$i = j = 1$

for $k = p$ to z
 if $L[i] \leq R[j]$
 $A[k] = L[i]$
 $i = i + 1$

else $A[k] = R[j]$
 $j = j + 1$

$a''m + b''$ a'', b'' costanti

complessivamente $am + b$

Merge(A, p, q, r)

inizializzazione: $a'n + b'$;ciclo: $a'n + b'$;

Sommandoli, ottengo una complessità all'incirca di:

$$T^{merge}(n) = an + b$$

Nel dettaglio:

$$T^{MS}(n) = \begin{cases} c_0 & \text{se } n \leq 1 \\ T^{MS}(n_1) + T^{MS}(n_2) + T^{merge}(n) & \text{altrimenti} \end{cases}$$

$$\Downarrow$$

$$T^{MS}(n) = \begin{cases} c_0 & \text{se } n \leq 1 \\ T^{MS}(n_1) + T^{MS}(n_2) + an + b & \text{altrimenti} \end{cases}$$

Quello che si sta facendo è quanto descritto;

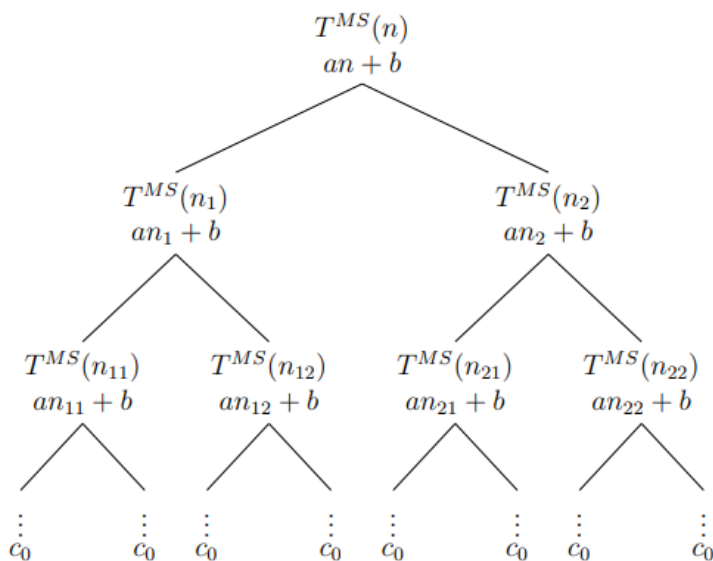
si consideri inoltre che, dato che abbiamo un albero diviso in due parti, le quadre "sopra" e "sotto" ad $n/2$ indicano, per chiarezza matematica, limite inferiore e limite superiore.

$$n_1 = \lfloor \frac{n}{2} \rfloor$$

$$n_2 = \lceil \frac{n}{2} \rceil$$

$$T^{MS}(n) = \begin{cases} c_0 & \text{se } n \leq 1 \\ T^{MS}(\lfloor \frac{n}{2} \rfloor) + T^{MS}(\lceil \frac{n}{2} \rceil) + an + b & \text{altrimenti} \end{cases}$$

Si struttura quindi l'albero di ricorsione, in quanto le due metà portano ad una scansione logaritmica, dato che a livelli eseguiamo una scansione per ogni pezzo dell'array:

Per $n = 2^{(h)}$

$$i = 0 \quad an + b$$

$$i = 1 \quad a(n_1 + n_2) + 2b \approx an + 2b$$

$$i = 2 \quad a(n_{11} + n_{12} + n_{21} + n_{22}) + 4b \approx an + 4b$$

...

$$i = h \quad c_0 n$$

$$\begin{aligned}
T^{MS}(n) &= \sum_{i=0}^{h-1} (an + 2^i b) + c_0 n \\
&= an h + b \sum_{i=0}^{h-1} 2^i \quad (h = \log_2 n) \\
&= an \log_2 n + b 2^h - b + c_0 n \quad (2^h = n) \\
&= an \log_2 n + (b + c_0)n - b \\
T^{MS}(n) &= an \log_2 n + b''n + c'' \approx n \log_2 n
\end{aligned}$$

Andando a confrontare Insertion Sort e Merge Sort, vedremmo che si avrebbe una struttura di limite di $1/\infty$, quindi significa che IS è molto peggio di MS.

$$\lim_{n \rightarrow +\infty} \frac{T^{MS}(n)}{T^{IS}(n)} = \lim_{n \rightarrow +\infty} \frac{a''n \log_2 n + b''n + c''}{a'n^2 + b'n + c'} = 0$$

Il tempo di esecuzione non va calcolato a mano ogni volta; sarebbe difficile, pesante e in tanti casi inutile. Conviene approssimarlo a qualche ordine di grandezza, notando le caratteristiche principali dell'algoritmo eseguito.

Prendiamo in esame due funzioni: $f(n)$, $g(n)$:

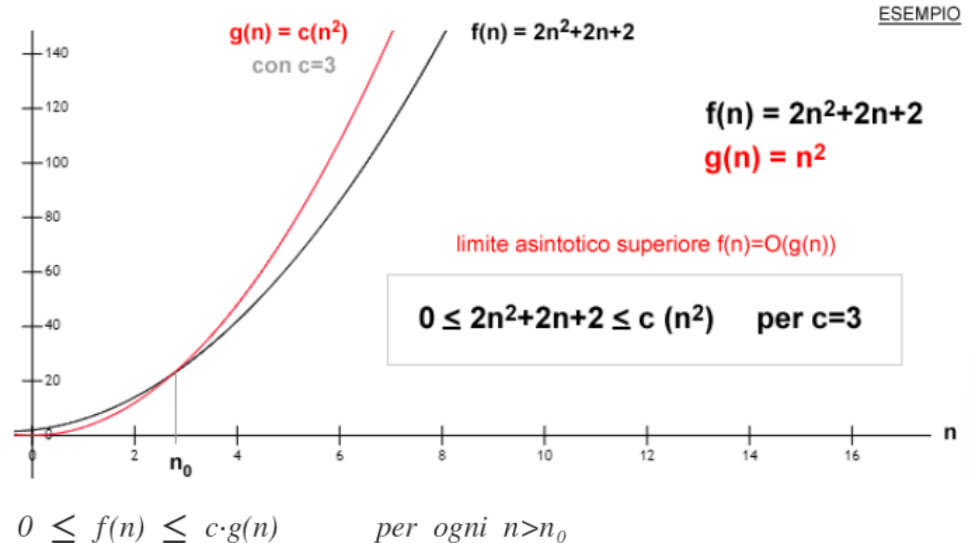
$$f, g : \mathbb{R}^+ \rightarrow \mathbb{R}^+$$

- $f(n)$ è la funzione in esame della complessità del nostro problema P;
- $g(n)$ è la funzione che, moltiplicata per un'opportuna costante c_i , dopo un certo n , fa da limite superiore o inferiore per ogni punto di $f(n)$.

Abbiamo due tipi di limiti:

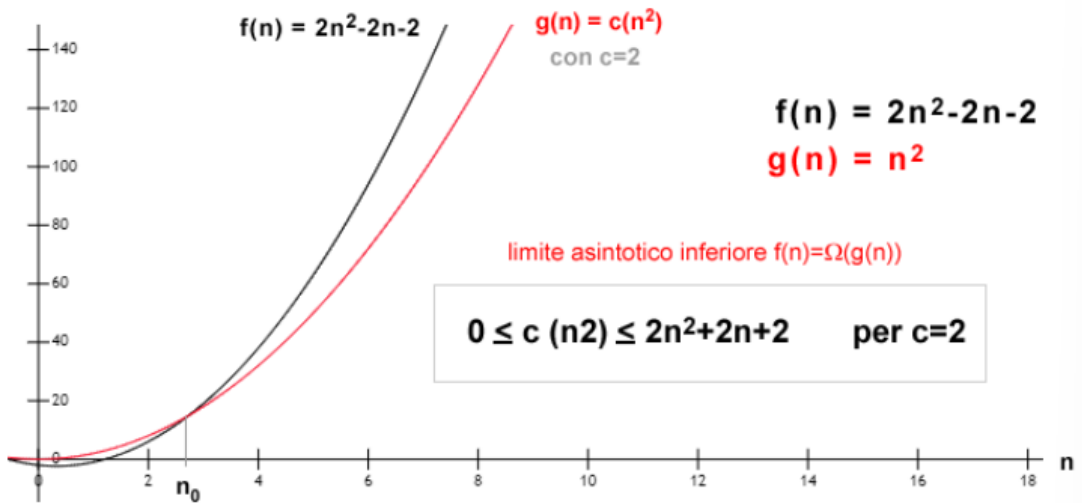
- 1) il limite asintotico superiore (O), per cui

Date due funzioni $f(n)$ e $g(n)$, si dice che $f(n) = O(g(n))$ se esiste un valore $c > 0$ tale che $0 \leq f(n) \leq c \cdot g(n)$ per ogni $n \geq n_0$



Per questa ragione la notazione asintotica O è detta limite asintotico superiore. In tali circostanze la $f(n)$ è una delle funzioni dell'insieme $O(g(n))$. La notazione $f(n)=O(g(n))$ si legge $f(n)$ è "O grande" di $g(n)$.

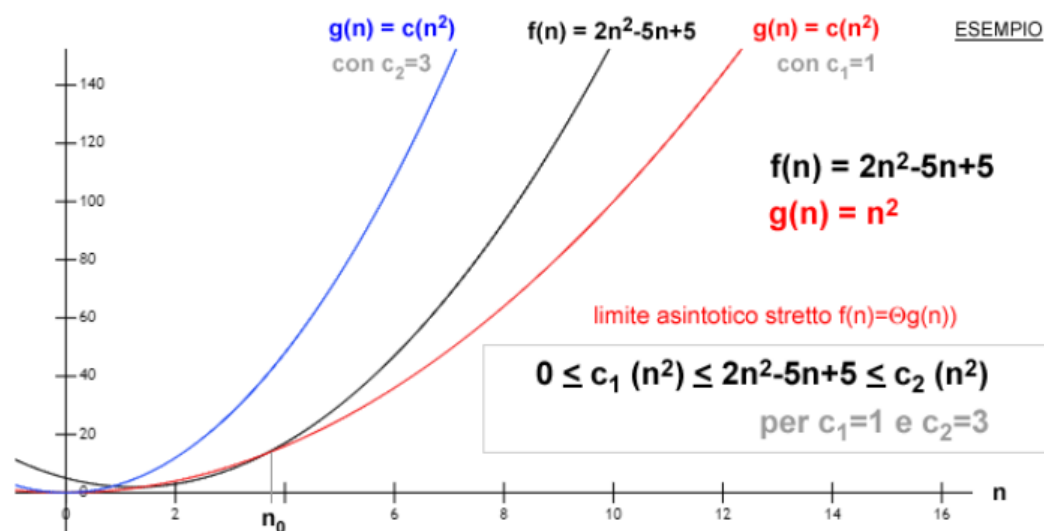
- 2) Il limite asintotico inferiore, detto anche Omega (Ω), per cui
Date due funzioni $f(n)$ e $g(n)$, si dice che $f(n) = \Omega(g(n))$ se esiste un valore $c > 0$ tale che
 $0 \leq c \cdot g(n) \leq f(n)$ per ogni $n \geq n_0$



$$0 \leq c \cdot g(n) \leq f(n) \quad \text{per ogni } n > n_0$$

Per questo motivo la notazione asintotica omega (Ω) è detta limite asintotico inferiore. Quindi la $f(n)$ è una delle funzioni dell'insieme $\Omega(g(n))$. La notazione $f(n) = \Omega(g(n))$ si legge $f(n)$ è "omega" di $g(n)$.

- 3) Il limite asintotico stretto, detto anche Theta (θ)
Date due funzioni $f(n)$ e $g(n)$, si dice che $f(n) = \theta(g(n))$ se esistono due valori $c_1 > 0$ e $c_2 > 0$ tali che $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ per ogni $n \geq n_0$



$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{per ogni } n > n_0$$

Esempi per i 3 casi:

1) Limite asintotico superiore

◦ $f_1(n) = 2n^2 + 5n + 3 = O(g(n^2))$? Sì.

Deve valere $f_1(n) < cn^2 \quad \exists c > 0, n \geq n_0$

Ipotizziamo $c = 3$

$$2n^2 + 5n + 3 \leq 3n^2$$

$$n^2 - 5n - 3 \geq 0$$

$$\frac{5 \pm \sqrt{2 \cdot 5 + 12}}{2} = \frac{5 \pm \sqrt{37}}{2} \cong 5.54$$

(Non considero la soluzione negativa, poiché siamo in $\mathbb{R}^+$)

Prendo $c = 3$ e $n_0 = 6$. Vale dunque:

$$f_1(n) \leq cn^2 \quad \forall n \geq n_0$$

◦ $f_1(n) = O(g(n^3))$? Sì.

$$c = 3$$

$$n_0 = 6 \quad \forall n \geq n_0$$

$$f_1(n) \leq cn^2 \leq cn^3$$

◦ $f_2(n) = 2 + \sin(n) = O(1)$? Sì.

$$-1 \leq \sin(n) \leq 1$$

$$1 \leq f_2(n) \leq 3$$

Vale la seguente

$$\exists c > 0 \quad \exists n_0 : \forall n \geq n_0 \quad f_2(n) \leq c \cdot 1$$

ok per $c = 3, n_0 = 0$

2) Limite asintotico inferiore

◦ $f_1(n) = 2n^2 + 5n + 3 = \Omega(g(n^2))$? Sì.

Deve valere:

$$\exists c > 0 \quad \exists n_0 : \forall n \geq n_0 \quad cn^2 \leq 2n^2 + 5n + 3$$

Basta porre $c = 1, n_0 = 0$.

◦ $f_2(n) = 2 + \sin(n) = \Omega(1)$? Sì.

$$1 \leq f_2(n) \leq 3 \quad c = 1, n_0 = 0$$

$$f_1(n) = 2n^2 + 5n + 3 = \Theta(n^2)$$

$$c_1 = 1 \quad c_2 = 3 \quad n_0 = 6$$

$$f_2(n) = 2 + \sin(n) = \Theta(1)$$

$$c_1 = 1 \quad c_2 = 3 \quad n_0 = 0$$

$$f_1(n) \neq \Theta(n^3)$$

$$f_1(n) = O(n^3)$$

$$f_1(n) \neq \Omega(n^3)$$

↓

$$\frac{f_1(n)}{n^3} \rightarrow 0$$

3) Limite asintotico stretto

Prima, intuitivamente, abbiamo descritto cosa succede a livello di limite; si basa semplicemente sull'osservazione le funzioni ed applicare un semplice principio dell'analisi: il metodo del limite, basato su confronto asintotico.

Siano $f(n), g(n) > 0 \quad \forall n$

Se $\exists \lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)}$, allora:

1. Se $\lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} = k > 0$ allora $f(n) = \Theta(g(n))$.
2. Se $\lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} = 0$ allora $f(n) = O(g(n))$ e $f(n) \neq \Omega(g(n))$.
3. Se $\lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} = \infty$ allora $f(n) = \Omega(g(n))$ e $f(n) \neq O(g(n))$.

Dimostrazioni (messe perché richieste in qualche esercizio, ma letteralmente applico le definizioni):

1. Sia $\lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} = k > 0$

$$\begin{aligned} \Rightarrow \forall \varepsilon > 0 \quad \exists n_0 : \forall n \geq n_0 \quad \left| \frac{f(n)}{g(n)} - k \right| &\leq \varepsilon \\ \Rightarrow -\varepsilon &\leq \frac{f(n)}{g(n)} - k \leq \varepsilon \\ \Rightarrow k - \varepsilon &\leq \frac{f(n)}{g(n)} \leq k + \varepsilon \\ \Rightarrow (k - \varepsilon)g(n) &\leq f(n) \leq (k + \varepsilon)g(n) \quad \text{per } 0 < \varepsilon < k \\ \Rightarrow f(n) &= \Theta(g(n)) \end{aligned}$$

2. Sia $\lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} = 0$

$$\begin{aligned} \Rightarrow \forall \varepsilon > 0 \quad \exists n_0 : \forall n \geq n_0 \quad \frac{f(n)}{g(n)} &\leq \varepsilon \\ \Rightarrow f(n) &\leq \varepsilon g(n) \\ \Rightarrow f(n) &= O(g(n)) \end{aligned}$$

Sia $f(n) = \Omega(g(n))$

$$\begin{aligned} \Rightarrow \exists c > 0 \quad \exists n_0 \in \mathbb{N} : \forall n \geq n_0 \quad cg(n) &\leq f(n) \\ \Rightarrow \text{Impossibile, infatti sia } \varepsilon = \frac{c}{2} > 0 \\ \Rightarrow \exists n_1 \in \mathbb{N} : \forall n \geq n_1 \quad f(n) &\leq \frac{c}{2}g(n) < cg(n) \\ \Rightarrow \text{Contraddizione!} \end{aligned}$$

3. Si dimostra in modo analogo al punto (2)

Per gli ordini di grandezza normalmente:

- non vale il confronto tra potenze ($O(n^2)$ e $O(n^3)$ non sono uguali)
- non vale il confronto tra basi (basi diverse/complessità diversa)

Lezione 4: Ricorrenze e master theorem

Abbiamo già avuto a che fare prima con mergesort con una ricorrenza, ma l'abbiamo risolta in un modo specifico e non generalizzato.

Prima di tutto, chiariamo cosa sarebbe una ricorrenza. Una relazione di ricorrenza è una formula ricorsiva che esprime il termine n -esimo di una successione (in generale a valori complessi) in relazione ai precedenti. La relazione si dice di ordine r se il termine n -esimo è espresso in funzione al più dei termini $(n-1)$ -esimo, ..., $(n-r)$ -esimo.

Parliamo prima del master theorem.

Normalmente, dato un problema con size n lo dividiamo in a sottoproblemi con *dimensione* n/b ed $f(n)$ costo della "combina" di entrambe le cose.

Otteniamo come ricorrenza (con $a \geq 1$, $b > 1$):

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

Esso stabilisce che, avendo a che fare con le ricorrenze di questo tipo, avremmo tre casi per confronto tra

- $f(n)$
- $n^{\log_b(a)}$

1. Se $f(n) = O(n^{\log_b a - \varepsilon})$ per qualche $\varepsilon > 0$, allora

$$T(n) = \Theta(n^{\log_b a})$$

2. Se $f(n) = \Theta(n^{\log_b a})$ allora

$$T(n) = \Theta(n^{\log_b a} \cdot \log n)$$

3. Se $f(n) = \Omega(n^{\log_b a + \varepsilon})$ per qualche $\varepsilon > 0$, e vale la **regolarità**

$$\exists 0 < k < 1 : a \cdot f\left(\frac{n}{b}\right) \leq k \cdot f(n)$$

allora

$$T(n) = \Theta(f(n))$$

Ciascuna delle condizioni di cui sopra può essere interpretata come:

1. Se il costo della risoluzione dei sotto-problemi ad ogni livello aumenta di un certo fattore, il valore di $f(n)$ diventerà polinomialmente più piccolo di $n^{\log_b a}$. Pertanto, la complessità temporale è oppressa dal costo dell'ultimo livello, vale a dire. $n^{\log_b a}$
2. Se il costo per risolvere il sotto-problema ad ogni livello è quasi uguale, allora il valore di $f(n)$ sarà $n^{\log_b a}$. Pertanto, la complessità temporale sarà $f(n)$ volte il numero totale di livelli, ad esempio. $n^{\log_b a} \cdot \log(n)$

3. Se il costo della risoluzione dei sottoproblemi ad ogni livello diminuisce di un certo fattore, il valore di $f(n)$ diventerà polinomialmente più grande di $n^{\log_b a}$. Pertanto, la complessità temporale è oppressa dal costo di $f(n)$.

L'idea è che la funzione si ripeta come rapporto ricorsivamente; questo corrisponde alla divisione in due parti e ancora in due parti, ecc.

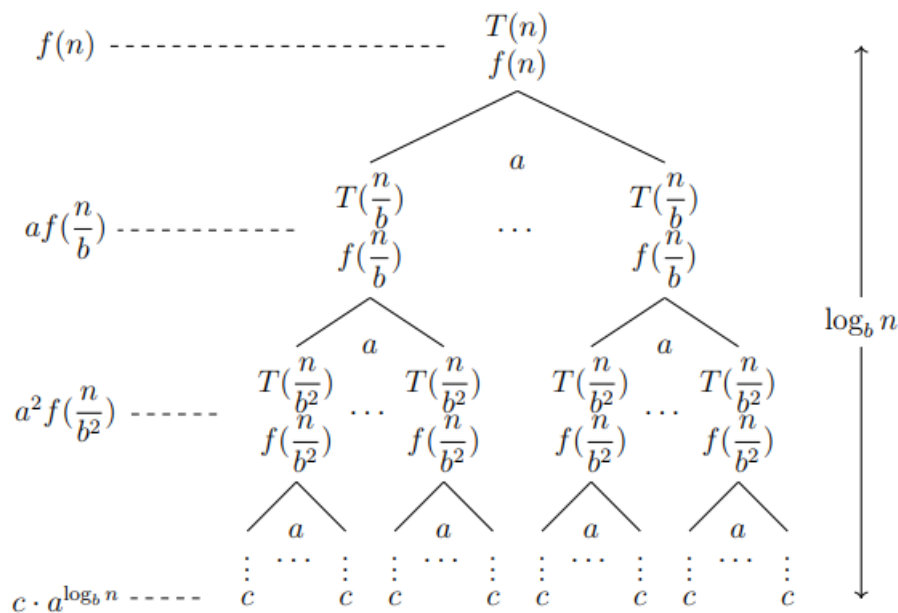
L'idea alla base riporta ad un albero di ricorsione (mostrato solo per capire come si visualizza questo ragionamento). A mano a mano che si ricorre, si ha che i rapporti e vari termini hanno varie divisioni e la potenza aumenta sempre più:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

$$T(n) = f(n) + af\left(\frac{n}{b}\right) + a^2 f\left(\frac{n}{b^2}\right) + \dots + a^{\log_b n} f\left(\frac{n}{b^{\log_b n}}\right) + c \cdot a^{\log_b n}$$

$$a^{\log_b n} = (b^{\log_b a})^{\log_b n} = (b^{\log_b n})^{\log_b a} = n^{\log_b a}$$

Nota bene: $af\left(\frac{n}{b}\right) \leq k \cdot f(n)$ con $k < 1$



Quello che si va a fare è quindi andare ogni volta a confrontare tra di loro (confronto tra limiti, banalmente):

- $f(n)$
- $n^{\log_b(a)}$

- | | | |
|-------------------------|---|---------------------------------|
| ① "vima" $n^{\log_b a}$ | ~ | ④ $(n^{\log_b a})$ |
| ② "potremmo" | ~ | ⑤ $(n^{\log_b a} \cdot \log n)$ |
| ③ vima $f(n)$ | ~ | ⑥ $(f(n))$ |

A)

$$\lim_{n \rightarrow \infty} \frac{f(n)}{n^{\log_b a}} = l(> 0) \neq \infty$$

$$\text{Caso 2} \Rightarrow T(n) = \Theta(n^{\log_b a} \cdot \log n)$$

B)

$$\lim_{n \rightarrow \infty} \frac{f(n)}{n^{\log_b a}} = 0$$

Potrei essere nel **Caso 1** $\Rightarrow$ se $\lim_{n \rightarrow \infty} \frac{f(n)}{n^{\log_b a - \varepsilon}} = l(\geq 0) \neq \infty$ ($\varepsilon > 0$)
 $\Rightarrow T(n) = \Theta(n^{\log_b a})$

C)

$$\lim_{n \rightarrow \infty} \frac{f(n)}{n^{\log_b a}} = \infty \quad \& \quad \exists \varepsilon > 0 : \lim_{n \rightarrow \infty} \frac{f(n)}{n^{\log_b a + \varepsilon}} = \infty$$

$\&$ **Regolarità** $\Rightarrow$ **Caso 3:** $T(n) = \Theta(f(n))$

Esercizi

- $T^{MS} = 2T\left(\frac{n}{2}\right) + a'n + b'$

Abbiamo (rispetto alla forma $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$)

$$a = 2, \quad b = 2$$

$$f(n) = a'n + b' \quad n^{\log_2 2} = n$$

È chiaro che le due funzioni hanno lo stesso andamento (di ordine $\Theta(n)$):

$$a'n + b' = \Theta(n)$$

$$\text{Caso 2} \Rightarrow T(n) = \Theta(n^{\log_2 2} \log n) = \Theta(n \log n)$$

- $T(n) = 5T\left(\frac{n}{2}\right) + 2n^2 + n \log n$

Abbiamo (rispetto alla forma $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$)

$$a = 5, \quad b = 2$$

$$f(n) = n^2 + n \log n \quad n^{\log_2 5} \quad (\log_2 5 > 2)$$

$$0 < \varepsilon < \log_2 5 - 2 \Rightarrow \lim_{n \rightarrow \infty} \frac{2n^2 + n \log n}{n^{\log_2 5 - \varepsilon}} = 0 \Rightarrow f(n) = O(n^{\log_2 5})$$

$$\text{Caso 1} \Rightarrow T(n) = \Theta(n^{\log_2 5})$$

- $T(n) = 5T\left(\frac{n}{2}\right) + n^3 \log n$

Abbiamo

$$\begin{aligned} a &= 5, \quad b = 2 \\ f(n) &= n^3 \log n \quad n^{\log_2 5} \quad (\log_2 5 < 3) \\ 0 < \varepsilon < 3 - \log_2 5 &\Rightarrow \lim_{n \rightarrow \infty} \frac{n^3 \log n}{n^{\log_2 5 + \varepsilon}} = \infty \end{aligned}$$

Possibile **caso 3. Regolarità?**

$$\begin{aligned} af\left(\frac{n}{b}\right) &\leq kf(n) \quad \text{per } 0 < k < 1 \text{ opportuno} \\ 5\left(\frac{n}{2}\right)^3 \log \frac{n}{2} &= \frac{5}{8}n^3 \log \frac{n}{2} \leq \frac{5}{8}n^3 \log n \leq kn^3 \log n \quad \text{per } 0 < k \leq \frac{5}{8} < 1 \\ &\Downarrow \\ \text{Caso 3: } T(n) &= \Theta(f(n)) = \Theta(n^3 \log n) \end{aligned}$$

Qui si usano solo una serie di trucchi algebrici per dimostrare la validità di tutta l'affermazione; bene o male, negli esercizi basta solo fare i calcoli.

Il numeratore è di ordine 3 grazie ad n^3 mentre il denominatore è di ordine inferiore a 3 sicuramente dato che $\log_2 5 < 3$. Vince il numeratore e quindi il limite va ad infinito; sono nel terzo caso:

$$T(n) = \Theta(n^3 \log(n))$$

Devo ora trovare, se esiste, un epsilon che soddisfi la condizione di esistenza (quella del se nella tabella). Il $\log_2 5$ è sicuramente più piccolo di 3 dato che $\log_2 8 = 3$ e $\log_2 5 < \log_2 8$. Possiamo quindi concludere che

$$0 < \varepsilon < 3 - \log_2 5$$

soddisfa le condizioni ed ε è sufficientemente piccolo e maggiore di zero. Come ultimo step devo verificare la regolarità risolvendo la disequazione e trovando un k compreso fra 0 ed 1.

$$5\left(\frac{n}{2}\right)^3 * \log\left(\frac{n}{2}\right) \leq kn^3 * \log(n)$$

Trucco: siccome è una disequazione posso maggiorare le equazioni; sappiamo che $\log\left(\frac{n}{2}\right) \leq \log(n)$ quindi posso scrivere:

$$\begin{aligned} 5\left(\frac{n}{2}\right)^3 * \log\left(\frac{n}{2}\right) &\leq 5\left(\frac{n}{2}\right)^3 * \log(n) \leq kn^3 * \log(n) \\ 5\left(\frac{n}{2}\right)^3 * \log(n) &\leq kn^3 * \log(n) \end{aligned}$$

Risolvendola ottengo che $0 < k < 1$ perché il risultato sarebbe $k > 5/8$ quindi anche questa condizione è verificata. La soluzione è davvero $\Theta(n^3 \log(n))$.

Ora descriviamo il metodo di sostituzione. Questo può venire in aiuto quando non riusciamo bene a risolvere con il Master Theorem, ipotizzando di conoscere la soluzione (quindi, dico che la ricorrenza avrà soluzione $T(n)$ uguale a qualcosa e provo a risolvere). Alternativamente a questa idea, si può usare l'albero delle ricorrenze. Alcuni esempi (con entrambe le modalità):

Esempio 1

Dimostrare che $T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + n$ ha soluzione $T(n) = \theta(n)$.

Soluzione

Siccome ho il θ devo fare le due disequazioni e dimostrare che esistono due costanti che verificano la veridicità dell'ipotesi $\theta(n)$.

Caso $T(n) \leq d \cdot f(n)$

$$1. \ d\left(\frac{n}{2}\right) + d\left(\frac{n}{4}\right) + n$$

$$2. \ d\left(\frac{n}{2}\right) + d\left(\frac{n}{4}\right) + n \leq dn$$

3. Faccio i conti e risolvo

$$\begin{aligned} n\left(\frac{d}{2} + \frac{d}{4} + 1\right) &\leq dn \\ \left(\frac{d}{2} + \frac{d}{4} + 1\right) &\leq d \\ \frac{d}{2} + \frac{d}{4} - d &\leq -1 \\ -\frac{d}{4} &\leq -1 \rightarrow d \geq 4 \end{aligned}$$

Caso $T(n) \geq c \cdot f(n)$

$$1. \ c\left(\frac{n}{2}\right) + c\left(\frac{n}{4}\right) + n$$

$$2. \ c\left(\frac{n}{2}\right) + c\left(\frac{n}{4}\right) + n \geq cn$$

3. Faccio i conti e risolvo

$$\begin{aligned} n\left(\frac{c}{2} + \frac{c}{4} + 1\right) &\geq cn \\ \left(\frac{c}{2} + \frac{c}{4} + 1\right) &\geq c \\ \frac{c}{2} + \frac{c}{4} - c &\geq -1 \\ -\frac{c}{4} &\geq -1 \rightarrow c \leq 4 \end{aligned}$$

Ho trovato un valore per c e d costanti quindi l'assunzione che quell'equazione abbia soluzione $\theta(n)$ è vera. Dato che $f(n) = n$ mettendo la n dentro alla T il risultato non cambia, ovvero $T\left(\frac{n}{2}\right)$ resta $T\left(\frac{n}{2}\right)$. Se avessi avuto $f(n) = n^2$ allora $T\left(\frac{n}{2}\right)$ sarebbe diventato un $T\left(\frac{n^2}{4}\right)$. In altre parole, devo mettere dentro alla funzione $f(n)$ quello che sta dentro alla T .

Esempio 2

Dimostrare che $T(n) = 2T\left(\frac{n}{2}\right) + 6n$ ha soluzione $T(n) = O(n \cdot \log n)$.

Soluzione

Siccome ho 0 devo fare una sola disequazione e verificare se esiste una costante d che mostra la veridicità dell'ipotesi.

$$\text{Caso } T(n) \leq d \cdot f(n)$$

$$1. 2d \left(\frac{n}{2} * \log \left(\frac{n}{2} \right) \right) + 6n$$

$$2. 2d \left(\frac{n}{2} * \log \left(\frac{n}{2} \right) \right) + 6n \leq d * n \log(n)$$

3. Faccio i conti e risolvo

$$2d \left(\frac{n}{2} * \log \left(\frac{n}{2} \right) \right) + 6n \leq d * n \log(n)$$

$$d \left(n * \log \left(\frac{n}{2} \right) \right) + 6n \leq d * n \log(n)$$

$$d(n * \log(n) - n * \log(2)) + 6n \leq d * n \log(n)$$

$$d * n * \log(n) - n(d * \log(2) - 6) \leq d * n \log(n)$$

$$-n(d * \log(2) - 6) \leq 0$$

$$d * \log(2) - 6 \geq 0$$

$$d \geq \frac{6}{\log(2)}$$

Se invece usassi l'albero delle ricorrenze, si cerca di capire "come ricorsivamente si sviluppa la ricorrenza e si cerca di quantificarne l'ordine di complessità" (es. la risolvo e vedo che "assomiglia" a MergeSort, quindi avrà complessità $n \log n$, oppure vedo che assomiglia ad InsertionSort e avrà complessità n^2 , ecc.)

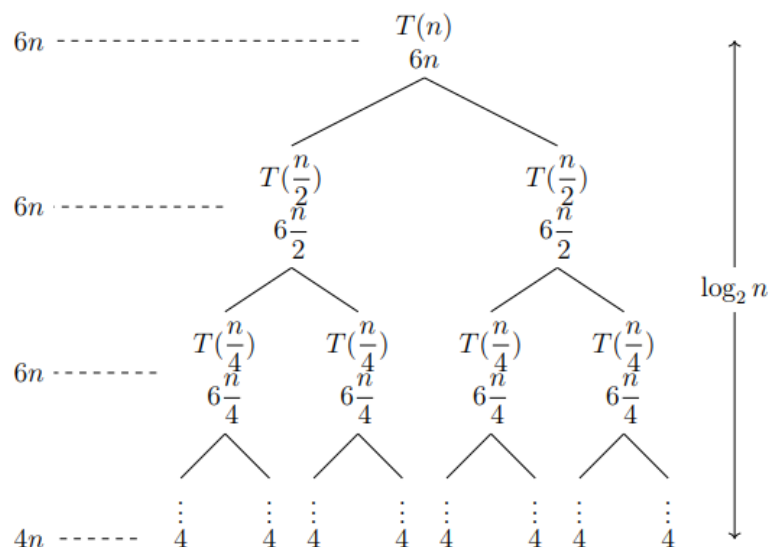
Esempio

$$T(n) = \begin{cases} 4 & \text{se } n = 1 \\ 2T\left(\frac{n}{2}\right) + 6n & \text{se } n > 1 \end{cases}$$

In generale, si può benissimo trascurare il caso base per poter ottenere espressioni meno verbose, in questo caso otterremmo:

$$T(n) = 2T\left(\frac{n}{2}\right) + 6n$$

Costruendo l'albero delle ricorrenze si intuisce già la soluzione:



Per essere sicuri della soluzione, facciamo il procedimento per intero. Proviamo a “indovinare” la soluzione. Assomiglia a MergeSort, quindi ipotizziamo abbia una complessità con un andamento simile

$$T(n) = an \log n + bn + c$$

Facciamo la prova induttiva.

$$\begin{aligned} (n = 1) \quad T(1) &= 4 \\ &= a \cdot 1 \cdot \log 1 + b \cdot 1 + c & (\log 1 = 0) \\ &= b + c & \text{ok se } b + c = 4 \\ (n > 1) \quad T(n) &= 2T\left(\frac{n}{2}\right) + 6n \end{aligned}$$

Per ipotesi induttiva

$$T\left(\frac{n}{2}\right) = a \frac{n}{2} \cdot \log \frac{n}{2} + b \frac{n}{2} + c$$

Calcolo ora $T(n)$

$$\begin{aligned} T(n) &= an \log_2 \frac{n}{2} + bn + 2c + 6n = \\ &= an \log_2 n - an \log_2 2 + bn + 6n + 2c = \quad (\log_2 2 = 1) \\ &= an \log_2 n + n(b + 6 - a) + 2c = \\ &= an \log_2 n + bn + c \\ &\Downarrow \end{aligned}$$

$$b + 6 - a = b \Rightarrow a = 6$$

$$2c = c \Rightarrow c = 0$$

$$b + c = 4 \Rightarrow b = 4$$

$$\begin{aligned} T(n) &= an \log n + bn + c \\ &= 6n \log n + 4n \end{aligned}$$

Esercizio (importante)

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + 6n \\ &= 2T\left(\frac{n}{2}\right) + \Theta(n) = \Theta(n \log n) \\ \text{vale } \exists c > 0 \exists n_0 : \forall n \geq n_0 \quad \Theta(n) &\leq cn \end{aligned}$$

Voglio dimostrare che

1. $T(n) = O(n \log n)$
2. $T(n) = \Omega(n \log n)$

$$1. \quad T(n) = O(n \log n)$$

significa che $\exists d > 0 \exists n_1 \in \mathbb{N} : T(n) \leq dn \log n \quad \forall n \geq n_1$

Dimostro per induzione $T(n) \leq dn \log n \quad \forall n \geq n_1$.

Ometto il caso base, poiché non è molto interessante (mi basterebbe aumentare ulteriormente d per avere un valore accettabile).

$$\begin{aligned} T(n) &\leq 2T\left(\frac{n}{2}\right) + cn & \text{ip. induttiva } T\left(\frac{n}{2}\right) &= d\frac{n}{2} \log \frac{n}{2} \\ &\leq 2 \cdot \frac{n}{2} d \log \frac{n}{2} + cn & \left(\log \frac{n}{2} &= \log n - \log 2\right) \\ &= dn \log n - dn \log 2 + cn \\ &= dn \log n - n(d \log 2 - c) \leq dn \log n \\ &\Rightarrow -n(d \log 2 - c) \leq 0 \\ &n(d \log 2 - c) \geq 0 \\ &d \log 2 - c \geq 0 \\ &d \geq \frac{c}{\log 2} \end{aligned}$$

$$2. \quad T(n) = \Omega(n \log n) \text{ è analoga.}$$

$$\exists \delta > 0 : \forall n > n_0 \Rightarrow T(n) \geq \delta n \log n$$

Ho l'ipotesi induttiva $T\left(\frac{n}{2}\right) \geq \delta \frac{n}{2} \log \frac{n}{2}$

$$\begin{aligned} T(n) &\geq 2\delta \frac{n}{2} \log \frac{n}{2} + cn = \\ &= \delta n \log n - \delta n \log 2 + cn = \\ &= \delta n \log n + n(c - \delta \log 2) \geq \delta n \log n \\ &\text{Deve valere } c - \delta \log 2 \geq 0 \\ &\Rightarrow 0 < \delta \leq \frac{c}{\log 2} \end{aligned}$$

$$\textbf{Esercizio} \quad T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + \Theta(n) \quad (\Theta(n) \leq c \cdot n)$$

Ipotizzo un andamento simile a MergeSort: $\Theta(n \log n)$. Dimostro:

$$1. \quad T(n) = O(n \log n)$$

$$2. \quad T(n) = \Omega(n \log n)$$

$$1. \quad T(n) = O(n \log n)$$

$$\exists d > 0 : \forall n > n_0 \Rightarrow T(n) \leq dn \log n$$

Ometto il caso base. L'ipotesi induttiva è la seguente:

$$T(n) \leq d \frac{n}{3} \log \frac{n}{3} + d \frac{2n}{3} \log \frac{2n}{3} + cn$$

Procedo con i calcoli ...

$$\begin{aligned} T(n) &\leq T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + cn \\ &\leq d \frac{n}{3} \log \frac{n}{3} + d \frac{2n}{3} \log \frac{2n}{3} + cn = \\ &= d \frac{n}{3} \left(\log n - \log 3 \right) + d \frac{2n}{3} \left(\log n - \log \frac{2}{3} \right) + cn = \\ &= dn \log n - \frac{dn}{3} \left(\log 3 - 2 \log \frac{2}{3} \right) + cn = \\ &= dn \log n - \frac{dn}{3} \left(\log 3 - \log \frac{4}{9} \right) + cn = \\ &= dn \log n - n \left(\frac{d}{3} \log \frac{27}{4} - c \right) \leq dn \log n \\ &\quad \frac{d}{3} \log \frac{27}{4} - c \geq 0 \\ &\Rightarrow d \geq \frac{3c}{\log \frac{27}{4}} \quad \left(\log \frac{27}{4} > 1 \text{ poiché } \arg > 1 \right) \end{aligned}$$

$$2. \quad T(n) = \Omega(n \log n) \text{ è analoga}$$

$$\exists \delta > 0 : \forall n > n_0 \Rightarrow T(n) \geq \delta n \log n$$

L'ipotesi induttiva è la seguente:

$$T(n) \geq \delta \frac{n}{3} \log \frac{n}{3} + \delta \frac{2n}{3} \log \frac{2n}{3} + cn$$

Calcoli ...

$$\begin{aligned} T(n) &\geq T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + cn \\ &\geq \delta \frac{n}{3} \log \frac{n}{3} + \delta \frac{2n}{3} \log \frac{2n}{3} + cn = \\ &= \delta \frac{n}{3} \left(\log n - \log 3 \right) + \delta \frac{2n}{3} \left(\log n - \log \frac{2}{3} \right) + cn = \\ &= \delta n \log n + \frac{\delta n}{3} \left(-\log 3 + 2 \log \frac{2}{3} \right) + cn = \\ &= \delta n \log n + \frac{\delta n}{3} \left(-\log 3 + \log \frac{4}{9} \right) + cn = \\ &= \delta n \log n + n \left(-\frac{\delta}{3} \log \frac{27}{4} + c \right) \geq \delta n \log n \\ &\quad -\frac{\delta}{3} \log \frac{27}{4} + c \geq 0 \\ &\Rightarrow 0 < \delta \leq \frac{3c}{\log \frac{27}{4}} \end{aligned}$$

Ora, abbiamo introdotto anche questa perché ora abbiamo un esempio dove il MT (Master Theorem) è inapplicabile e possiamo provare per sostituzione.

- $T(n) = 27T\left(\frac{n}{3}\right) + n^3 \log n$

$$f(n) = n^3 \log n \quad n^{\log_3 27} \quad (\log_3 27 = 3)$$

$$\lim_{n \rightarrow \infty} \frac{n^3 \log n}{n^{3+\varepsilon}} = +\infty \quad \forall \varepsilon > 0, \text{ non possiamo dimostrare 3}$$

$\Rightarrow$ Non siamo in **nessun** caso del Master Theorem.

Anche valutando la **regolarità**, ricadiamo in un assurdo. Dobbiamo dimostrare che $af\left(\frac{n}{b}\right) < kf(n)$ per qualche $k > 0$

$$27\left(\frac{n}{3}\right)^3 \log \frac{n}{3} = n^3(\log n - \log 3) \not< kn^3 \log n \quad \text{per nessun } k > 0$$

$$\text{Infatti } \frac{(\log n - \log 3)n^3}{n^3 \log n} \rightarrow 1$$

(Posso usare il Metodo di Sostituzione)

$$T(n) = 27T\left(\frac{n}{3}\right) + n^3 \log n$$

Costruiamo l'albero delle ricorrenze:

- **radice**: costo $n^3 \log n$;
- ogni nodo ha 27 figli.
 - ◊ i 27 figli del primo livello hanno costo $\left(\frac{n}{3}\right)^3 \log \frac{n}{3}$;
 - ◊ i 27^2 figli del secondo livello hanno costo $\left(\frac{n}{9}\right)^3 \log \frac{n}{9}$;
 - ◊ ...
 - ◊ le 27^n foglie terminali hanno costo $O(1)$.

$$\begin{aligned} T(n) &= \sum_{j=0}^{\log_3 n} n^3 \log \frac{n}{3^j} = n^3 \sum_{j=0}^{\log_3 n} (\log n - j \log 3) + cn = \\ &= n^3 (\log n)^2 - n^3 \log 3 \sum_{j=0}^{\log_3 n} j + cn \quad \left(\sum_{j=0}^{\log_3 n} j \cong (\log_3 n)^2 \right) \end{aligned}$$

$$T(n) = 27T\left(\frac{n}{3}\right) + n^3 \log n$$

$$T(n) = \Theta(n^3 (\log n)^2) \quad \text{ipotesi ricavata}$$

Devo dimostrare che valgano le seguenti condizioni:

$$1. T(n) = O(n^3(\log n)^2)$$

$$2. T(n) = \Omega(n^3(\log n)^2)$$

$$1. T(n) = O(n^3(\log n)^2)$$

$$T(n) \leq c \cdot n^3(\log n)^2 \quad c > 0$$

$$T(n) = 27T\left(\frac{n}{3}\right) + n^3 \log n$$

$$\left(\text{ipotesi induttiva } T\left(\frac{n}{3}\right) \leq c \cdot \left(\frac{n}{3}\right)^3 \left(\log \frac{n}{3}\right)^2\right)$$

$$\leq 27c \left(\frac{n}{3}\right)^3 \left(\log \frac{n}{3}\right)^2 + n^3 \log n =$$

$$= \frac{27cn^3}{27} (\log n - \log 3)^2 + n^3 \log n =$$

$$= cn^3 \left((\log n)^2 - 2 \log 3 \log n + (\log 3)^2 \right) + n^3 \log n =$$

$$= cn^3 (\log n)^2 - n^3 \left(\log n (2c \log 3 - 1) - c(\log 3)^2 \right)$$

$$\leq cn^3 (\log n)^2$$

Per un n abbastanza grande, vale la disuguaglianza con un opportuno valore di c :

$$c > \frac{1}{2 \log 3}$$

$$2. T(n) = \Omega(n^3(\log n)^2)$$

$$\exists d > 0 : T(n) \geq dn^3(\log n)^2$$

$$\geq 27 \left(\frac{n}{3}\right)^3 \left(\log \frac{n}{3}\right)^2 + n^3 \log n$$

$$= \dots = dn^3 (\log n)^2 - n^3 \left(\log n (2d \log 3 - 1) - d(\log 3)^2 \right)$$

$$\geq dn^3 (\log n)^2$$

Per un n abbastanza grande, vale la disuguaglianza con un opportuno valore di d :

$$2d \log 3 - 1 < 0 \quad \text{ok per } 0 < d < \frac{1}{2 \log 3}$$

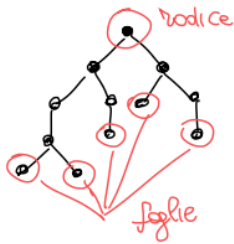
Lezione 5: Heap sort: analisi e complessità. Code con priorità e metodi di utilità. Heap e tipi (alberi e procedure)

L'algoritmo *heapsort* è un algoritmo di ordinamento basato su una struttura dati chiamata *heap*.

Prima di poter introdurre effettivamente cos'è uno heap, dobbiamo dare alcune definizioni.

Partiamo proprio dal concetto di *albero* accennato in precedenza, identificando per esso una serie di caratteristiche:

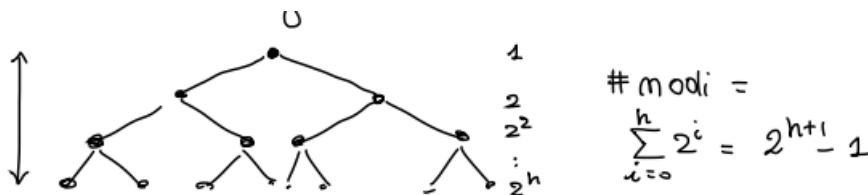
* Alberi



- binario (ogni nodo ha max 2 figli)
- altezza (distanza dalla radice alla foglia più distante)
- ordinato (ogni nodo ha figlio dx e figlio sx, se binario)

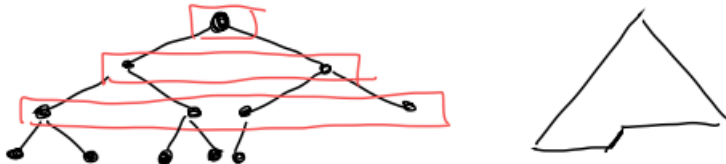
- albero binario completo (ogni nodo non foglia ha due figli e ogni cammino radice-foglia ha la stessa lunghezza). Esso, infatti:

è un albero di altezza h con $\sum_{i=0}^h 2^i - 1$ nodi;



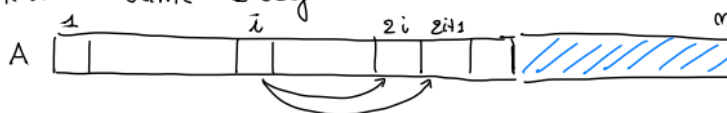
$$\# \text{ nodi} = \sum_{i=0}^h 2^i = 2^{h+1} - 1$$

- albero quasi completo, ogni livello è completo, tranne eventualmente l'ultimo con foglie tutte a sx



Noi partiamo da questa nozione; infatti, lo heap è un albero binario ordinato quasi completo, ma implementato come se fosse un array.

implementato come array



Caratteristiche:

- $A[1]$ è radice
- Un generico nodo sta in posizione $A[i]$.
- Un nodo $A[2i]$ è il figlio sinistro del nodo $A[i]$
- Un nodo definito come nodo *parent/genitore* sta in posizione $A[i/2]$
- Lo spazio occupato effettivamente è $A.size$, la lunghezza dell'array (intesa come capacità massima/potenziale spazio) è $A.length$

LEFT(i)

// restituisce il figlio sx del nodo i

1 return $2 * i$

RIGHT(i)

// restituisce il figlio dx del nodo i

1 return $2 * i + 1$

PARENT(i)

// restituisce il genitore del nodo i

1 return $[i/2]$

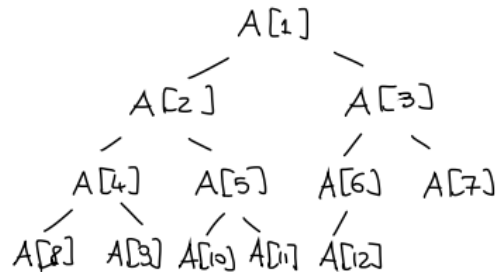
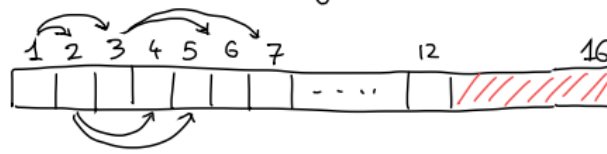
Scritto da Gabriel

Esempio

$A[1..16]$

$A.length = 16$

$A.size = 12$



$Parent(i)$
return $i/2$

$Left(i)$
return $i*2$

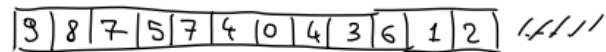
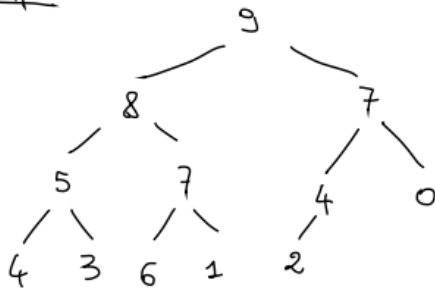
$Right(i)$
return $i*2+1$



Ora veniamo al concetto di max heap, cioè un heap dove

- per ogni nodo $A[i]$, $A[i] \geq$ nodi discendenti (quindi, $A[Left(i)], A[Right(i)]$)
- per ogni nodo $A[i]$, $A[i] \leq$ nodi antenati (quindi, $A[i] \leq A[Parent(i)]$)

Esempio :



Osservazioni

- Uno heap con un solo elemento è un Max Heap.
- Dati due Max Heap T1 e T2 e un nodo N, possiamo “combinarli” in uno heap con N come radice, T1 come left e T2 come right.

Un Max Heap è ottenuto da una procedura *Max – Heapify* che, dato un nodo i , trasforma in un Max Heap il sottoalbero in esso (con radice i).

MAX-HEAPIFY(A, i)

```

1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if ( $l \leq A.\text{heapsize}$ ) and ( $A[l] > A[i]$ )
4       $\text{max} = l$ 
5  else
6       $\text{max} = i$ 
7  if ( $r \leq A.\text{heapsize}$ ) and ( $A[r] > A[\text{max}]$ )
8       $\text{max} = r$ 
9  if ( $\text{max} \neq i$ )
10      $A[i] \leftrightarrow A[\text{max}]$ 
11     MAX-HEAPIFY( $A, \text{max}$ )
    
```

Prendiamo come input un array A e un indice di un nodo i . Le variabili l e r indicano il nodo figlio sinistro e destro del nodo iniziale s .

Organizziamo quindi il nodo i e i suoi sottoalberi dall'array A di input in modo tale da soddisfare la proprietà max-heap.

Utilizzando *Max – Heapify* costruiamo un max-heap. Iniziamo il nostro algoritmo con un nodo che si trova al livello più basso dell'albero e che ha dei nodi figli. Quindi disponiamo il nodo corrente e i suoi nodi figli in base alla proprietà max-heap.

IN: A heap
 i modo di riferimento

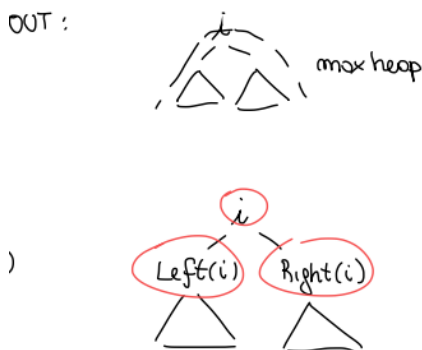
Correttezza:

o **Casi base:** $\text{max} = i$, distinguo due casi:

- i è foglia ($l, r > A.\text{heapsize}$);
- $A[i] \geq A[l], A[r]$;

o **Induzione:** $\text{max} \neq i$, distinguo due casi:

- $\text{max} = l$, $A[l] \geq A[i], A[r]$;
- $\text{max} = r$, $A[r] \geq A[i], A[l]$.



In merito alla complessità, tutto dipende dall'albero, il quale avrà n elementi possibili.

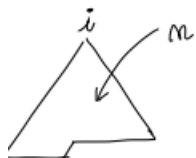
Infatti, la complessità $O(h)$, con h altezza del sottoalbero, alternativamente $O(\log(n))$.

$$n \geq (2^{(h-1)+1}) + 1 = 2^h$$

$$h \leq \log_2 n$$

$$\Rightarrow O(h) \cong O(\log n)$$

Se invece fosse visto come ricorrenza, si può dimostrare con il Master Theorem:



$$T(m) = T\left(\frac{2}{3}m\right) + c$$

$\uparrow$ $\uparrow$ $\uparrow$
 $a=1$ $b=\frac{3}{2}$ $f(m)=c$

master theorem

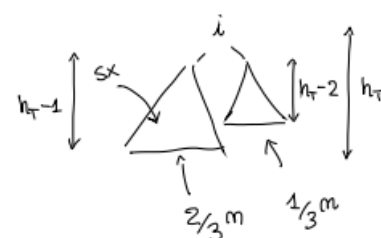
$$f(m) = c$$

$$m^{\log_a b} = m^{\log_{\frac{3}{2}} 1} = m^0 = 1$$

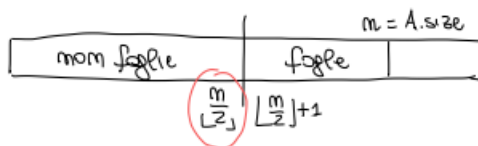
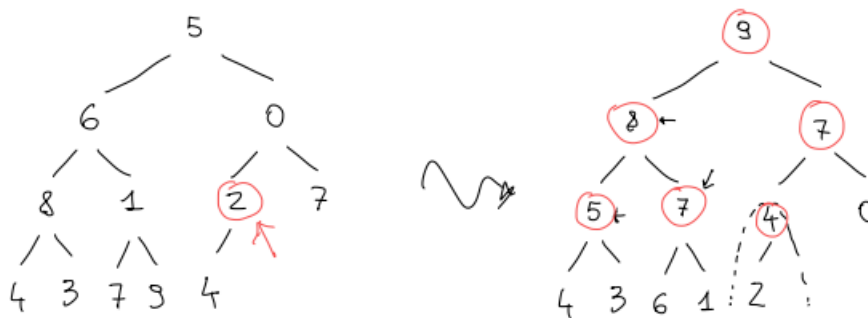
$$f(m) = c = \Theta(1) (m^{\log_a b}) = \Theta(1) (m^0)$$

CASO 2

$$T(m) = \Theta\left(\frac{m^{\log_a b}}{m^0} \cdot \log m\right) = \Theta(\log m)$$



Per costruire un Max Heap, ci vuole una procedura apposita, chiamata *Build – Max – Heap*, dato A array le foglie sono radici di Max Heap e, ricorsivamente, si controlla ogni singolo elemento, come si vede di seguito:



Dato quindi A array visto come heap ($A.size = n$), i nodi foglia sono $A[i]$, con $i \geq \lfloor n/2 \rfloor + 1$. Infatti:

- se $i \geq \lfloor \frac{n}{2} \rfloor + 1$ $Left(i) = 2 \cdot i \geq 2 \left(\lfloor \frac{n}{2} \rfloor + 1 \right) = 2 \lfloor \frac{n}{2} \rfloor + 2 \geq n+1$
 $\Rightarrow i$ non ha figlio sinistro $\Rightarrow$ è foglia
- se $i \leq \lfloor \frac{n}{2} \rfloor$ allora $Left(i) = 2 \cdot i \leq 2 \lfloor \frac{n}{2} \rfloor \leq n$
 $\Rightarrow i$ ha figlio sinistro $\Rightarrow$ non è foglia

Andiamo quindi alla procedura di costruzione *Build – Max – Heap*.

Animazione di costruzione di heap:

https://www.tutorialspoint.com/data_structures_algorithms/images/max_heap_deletion_animation.gif

BUILD-MAX-HEAP(A)

- 1 $A.heapsize = A.length$
- 2 **for** $i = \lfloor A.length/2 \rfloor$ **down to** 1
- 3 **MAX-HEAPIFY**(A, i)

L'algoritmo esegue $\frac{n}{2}$ volte **MaxHeapify** (che ha complessità $O(\log n)$), ottenendo una complessità finale $O(n \log n)$, tuttavia questa stima è molto pessimistica.

La proprietà è data dal fatto che, $\forall j > i, A[j]$ è radice di max heap.

- Inizio $\rightarrow I = \lfloor A.size/2 \rfloor, \forall j > i, A[j]$ foglia è radice di max heap.
- Mantenimento $\rightarrow Left$ e $Right$ sono $> i$ e quindi sono radice di max heap. La procedura **MaxHeapify**(A, i)
- Conclusione $\rightarrow i = 0, \forall j > 0, A[j]$ radice di max heap

Ora, vedendo un limite superiore semplificato, avremmo che, per $n/2$ iterazioni, con:

- h_T altezza del cammino più lungo dello heap;
- $h_T - 1$ di conseguenza è l'altezza dell'albero meno l'ultimo livello, che è generalmente incompleto.

$$\begin{aligned} n &= \left(2^{(h_T-1)+1} - 1\right) + 1 \\ &= 2^{h_T} \\ h_T &\leq \log n \\ n &\geq 2^{h_T} \end{aligned}$$

Ottenendo quindi che il costo totale è $O(n \log n)$.

Volendo fare un calcolo più preciso (inserito solo per completezza):

$$\begin{aligned} T(n) &= \sum_{h=1}^{\lfloor \log n \rfloor} 2^{h_T-h} \cdot O(h) \\ &\quad (2^{h_T-h} = \# \text{ chiamate a MaxHeapify al livello } h) \\ &= \sum_{h=1}^{\lfloor \log n \rfloor} \frac{2^{h_T}}{2^h} O(h) \quad (2^{h_T} = n) \\ &= O\left(\left(\sum_{h=1}^{\lfloor \log n \rfloor} \frac{h}{2^h}\right)n\right) = O(n) \quad \left(\sum_{h=1}^{\lfloor \log n \rfloor} \frac{h}{2^h} \leq \sum_{h=1}^{\infty} \frac{h}{2^h} = \frac{\frac{1}{2}}{(1 - \frac{1}{2})^2} = 2\right) \end{aligned}$$

Ora, passiamo finalmente ad *Heapsort*, per cui la radice di un Max Heap contiene il valore massimo. Quindi, la prima operazione, e quella su cui si basa Heapsort, consiste nel mettere la radice in ultima posizione.

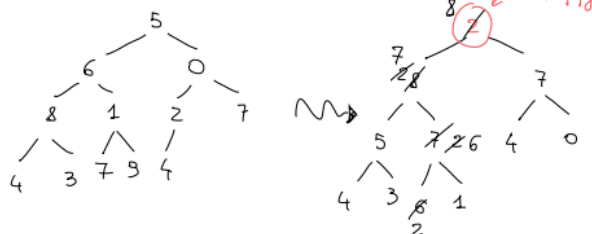
Poi chiama MaxHeapify sul resto dell'array per renderlo un Max Heap, e itera il procedimento sul nuovo array.

Es. A: 9 8 7 5 7 4 0 4 3 6 1 2
 $\Rightarrow$ 8 7 5 7 4 0 4 3 6 1 2 9

è un max heap.

ignoro l'ultimo elemento, chiamo MaxHeapify sulla radice e itero.

[5 | 6 | 0 | 8 | 1 | 2 | 7 | 4 | 3 | 7 | 3 | 4]



[9 | 8 | 7 | 5 | 7 | 4 | 0 | 4 | 3 | 6 | 1 | 2]

[2 | 8 | 7 | 5 | 7 | 4 | 0 | 4 | 3 | 6 | 1 | 9]

[8 | 7 | 7 | 5 | 6 | 4 | 0 | 4 | 3 | 2 | 1 | 9]

[1 | 7 | 7 | 5 | 6 | 4 | 0 | 4 | 2 | 8 | 9]

HEAPSORT(A)

```
1 BUILD-MAX-HEAP(A) // O(n)
2 for i = A.length down to 2
3     A[1] ↔ A[i]
4     A.heapsize = A.heapsize - 1
5     MAX-HEAPIFY(A, 1) // O(log n)
```

Complessità $O(n \log n)$.

Correttezza:

- Invariante $\rightarrow A[1, \dots i]$ max heap e $A[i + 1, \dots n]$ ordinato e $A[1, \dots i] \leq A[i + 1, \dots n]$
- Conclusione $\rightarrow i = 1, A[1]$ max heap, $A[2, \dots n]$ ($A[1] \leq A[2, \dots n] \rightarrow A[1, \dots n]$ ordinato)

Uso di Max Heap $\rightarrow$ Le code con priorità.

Esse sono strutture dinamiche con un insieme S di elementi. Su di esse definiamo varie operazioni, quali *Insert*, *Max*, *ExtractMax*, ecc.

L'idea è usare un Max Heap A per una serie di procedure.

Ad esempio, voglio trovare il massimo di una coda con la procedura *Max*; esso, avendo un Max Heap, corrisponderà alla radice.

MAX(A)

```

1  if A.heapsize = 0
2      error
3  else return A[1]
```

La procedura **Max(A)** ha complessità costante $\Theta(1)$.

Ulteriore procedura l'estrazione del massimo con la procedura *Extract - Max*, la quale salva il primo elemento (più grande, radice), scambia primo ed ultimo elemento e ripristina le proprietà di Max Heap attraverso *MaxHeapify*.

EXTRACT-MAX(A)

```

1  max = A[1]
2  A[1] = A[A.heapsize]
3  A.heapsize = A.heapsize - 1
4  MAX-HEAPIFY(A, 1) // ripristina le proprietà di MaxHeap
5  return max
```

La procedura **ExtractMax(A)** ha la stessa complessità di **MaxHeapify**: $O(\log n)$.

In merito all'inserimento, le cose diventano più delicate. L'idea è quella di inserire in coda ad A : in questo modo, l'unico elemento che potrebbe compromettere la proprietà di Max Heap è la cella di indice i (nel nostro caso, l'ultima).

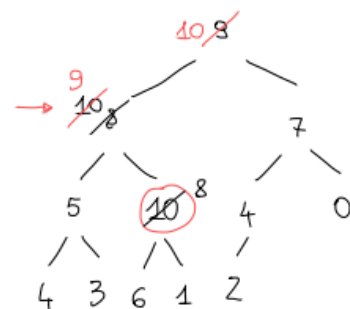
Deve quindi valere che $\forall j \neq i \quad A[j] \leq \text{antenati}$

Ogni volta, bisogna sempre ristabilire le proprietà di Max Heap e quindi usiamo la procedura *MaxHeapifyUp*, per la quale si trasforma A in un max heap.

MAX-HEAPIFY-UP(A, i)

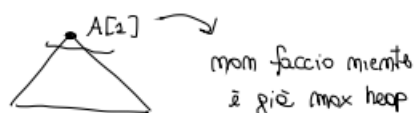
```

1  if (i > 1) and (A[i] > A[PARENT(i)])
2      A[i]  $\leftrightarrow$  A[PARENT(i)]
3      MAX-HEAPIFY-UP(A, PARENT(i))
```

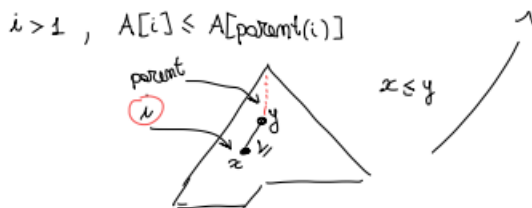


Correttezza:

($i = 1$) ok, non faccio nulla;

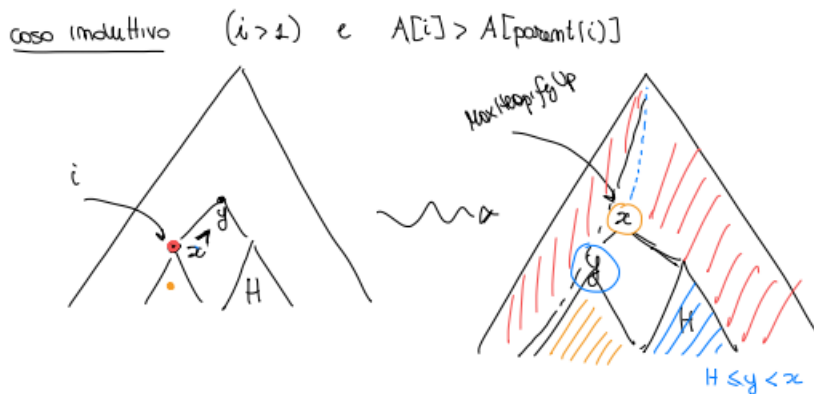


$(A[i] \leq A[\text{Parent}(i)])$ ok, la proprietà di **Max Heap** è mantenuta.



Induzione

$(A[i] > A[\text{Parent}(i)])$ scambio le due celle. I discendenti (sottoalberi) della nuova cella $A[i]$ mantengono la proprietà di **Max Heap**.



Complessità $\rightarrow O(\log n)$

La funzione *Insert*, dunque, ha il seguente pseudocodice:

```

INSERT(A, x)
1  A.heapsize = A.heapsize + 1
2  A[A.heapsize] = x
3  MAX-HEAPIFY-UP(A, A.heapsize)
    
```

Insert ha complessità $O(\log n)$, la stessa di *MaxHeapifyUp*.



L'aumento della chiave con *IncreaseKey* permette quindi di introdurre un valore e ristabilire di volta in volta le proprietà di max heap.

```

INCREASE-KEY(A, i, δ)
  // Precondizione: δ ≥ 0
1  A[i] = A[i] + δ
2  MAX-HEAPIFY-UP(A, i)
    
```

IncreaseKey ha complessità $O(\log n)$.

Similmente, anche *ChangeKey* permette di trovare il nodo della chiave, cambiare la chiave del nodo con la nuova chiave, ristabilire la proprietà di heap per il nodo e aggiornare correttamente.

CHANGE-KEY(A, i, δ)

```

1   $A[i] = A[i] + \delta$ 
2  if  $\delta > 0$ 
3      MAX-HEAPIFY-UP( $A, i$ )
4  else //  $\delta \leq 0$ 
5      MAX-HEAPIFY( $A, i$ )

```

Può usare valori qualsiasi, ma è corretto per la seguente proprietà
 $\forall j \neq i \quad A[j] \geq \text{discendenti} \Rightarrow \text{dopo MaxHeapify ho un Max Heap.}$

Altre operazioni (citare per completezza) $\rightarrow$ Delete e Decrease – Key

DELETE(A, i)

```

1  old =  $A[i]$ 
2   $A[i] = A[A.heapsize]$ 
3   $A.heapsize = A.heapsize - 1$ 
4  if  $old \leq A[i]$ 
5      MAX-HEAPIFY-UP( $A, i$ )
6  else
7      MAX-HEAPIFY( $A, i$ )

```

Delete ha complessità $O(\log n)$.

Algorithm 4: Decrease-Key

Data: A : The heap array
 key: The key to be updated
 sub: The value to subtract from the given key
 map: The index of each element

Result: Subtracts the value from the given key, and updates the map

```

 $i \leftarrow \text{map}[\text{key}];$ 
 $\text{map.remove}(\text{key});$ 
 $A[i] \leftarrow A[i] - \text{sub};$ 
 $\text{map}[A[i]] \leftarrow i;$ 
while  $i > 1$  do
    if  $A[i] < A[i/2]$  then
        swap( $A, i, i/2, \text{map}$ );
         $i \leftarrow i/2;$ 
    end
    else
        break;
    end
end

```

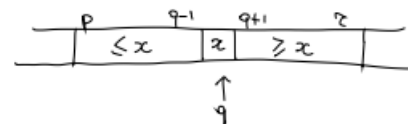
Lezione 6: Quicksort

L'algoritmo di ordinamento più utilizzato e generalmente più efficiente con $O(n^2)$ come caso pessimo ed $O(n \log n)$ come caso medio e migliore è proprio quicksort. Viene definito "algoritmo in-place/in loco", cioè un algoritmo che non ha bisogno di spazio extra e produce un output nella stessa memoria che contiene i dati trasformando l'input "in-place". Tuttavia, è consentito un piccolo spazio aggiuntivo costante per le variabili. Fu inventato nel 1961. Prevede delle costanti additive e moltiplicative basse.

Anche questo algoritmo è basato sul paradigma divide-et-impera, infatti, per ordinare $A[p, \dots r]$:

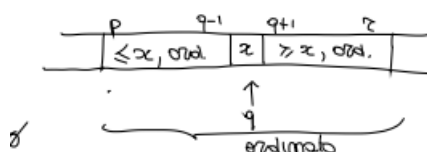
◦ Divide

- $\rightarrow$ Sceglie un *pivot* x in $A[p, r]$;
- $\rightarrow$ partiziona in $A[p, q-1] \leq x$ e $A[q+1, r] \geq x$;



◦ Impera

Ricorre su $A[p, q-1]$ e $A[q+1, r]$;



◦ Combina

(Non fa nulla).

Scritto da Gabriel

Per ordinare un array, seguite i passaggi seguenti:

- Si sceglie come perno/pivot un qualsiasi valore di indice dell'array.
- Quindi si partiziona l'array in base al pivot.
- Quindi si esegue un quicksort ricorsivo della partizione di sinistra.
- Successivamente, si esegue un quicksort ricorsivo della partizione corretta.

Diamo un'occhiata più da vicino alla partizione di questo algoritmo:

- Si sceglie un pivot qualsiasi, ad esempio il valore più alto dell'indice.
- Si prenderanno due variabili per puntare a sinistra e a destra dell'elenco, escludendo il pivot.
- La sinistra punterà all'indice più basso e la destra all'indice più alto.
- Ora si spostano a destra tutti gli elementi maggiori del pivot.
- Quindi si sposteranno tutti gli elementi più piccoli del perno nella partizione di sinistra.

QUICKSORT(A, p, r)

```

1  if  $p < r$ 
2     $q = \text{PARTITION}(A, p, r)$ 
3    QUICKSORT( $A, p, q$ )
4    QUICKSORT( $A, q + 1, r$ )

```

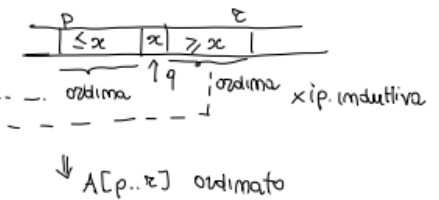
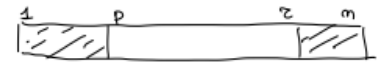
Quicksort (A, p, r)

if $p < r$

$q = \text{Partition}(A, p, r)$

Quicksort ($A, p, q-1$)

Quicksort ($A, q+1, r$)



Per l'algoritmo di partizione *Partition* un approfondimento:

Innanzitutto, concettualmente la funzione di partizione è in realtà una funzione che produce tre valori: gli elenchi di tutto ciò che è minore e maggiore del perno e il perno stesso. Questo è un po' oscurato dal fatto che il valore che restituisce è un indice dell'array; questo perché i due elenchi vengono "passati" al chiamante nell'array stesso e l'indice comunica tutto ciò che serve:

1. L'indice punta alla cella che contiene il pivot.
2. Tutti gli indici minori puntano a celle con valori minori del pivot.
3. Tutti gli indici maggiori puntano a celle con valori maggiori.

Ora, con queste conoscenze, lo scopo della partizione dovrebbe essere chiaro: data una porzione non ordinata dell'array (designata da *low* e *high*), essa sceglie un pivot e riordina quella porzione dell'array in modo che le affermazioni (1), (2) e (3) di cui sopra siano valide.

L'ultima domanda riguarda il modo in cui la partizione svolge il suo lavoro. Il ciclo for lo fa mantenendo la regola che, alla fine di ogni iterazione, ogni cella compresa tra *lo* (incluso) e *i* (esclusivo) contiene un elemento non superiore al pivot. (L'elemento *i* e tutti quelli che seguono possono essere o meno maggiori del perno). Chiameremo questi elementi minori - da $A[low]$ ad $A[i - 1]$ - "il fronte".

Nel blocco if del ciclo, scambiando $A[i]$ con $A[j]$ e incrementando *i* si ha l'effetto di spostare un numero inferiore al perno nella parte anteriore.

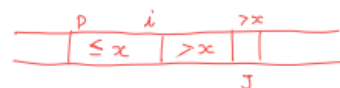
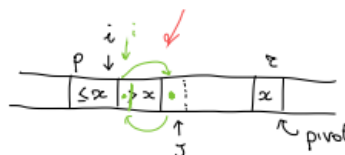
Al termine del ciclo, tutti i numeri inferiori o uguali al perno sono stati spostati in avanti. Scambiando $A[i]$ con $A[high]$ - ricordandosi che $A[high]$ era il pivot - si sposta il perno nel primo slot dopo il fronte. A quel punto, i tre criteri (1, 2 e 3) sono validi.

PARTITION(A, p, r)

```

1   $x = A[r]$  // pivot  $A[r]$ 
2   $i = p - 1$ 
   //  $A[p, i] \leq x$ 
   //  $A[i+1, j-1] > x$ 
3  for  $j = p$  to  $r - 1$ 
4    if  $A[j] \leq x$ 
5       $i = i + 1$ 
6       $A[i] \leftrightarrow A[j]$ 
7   $A[i + 1] \leftrightarrow A[r]$ 
8  return  $i + 1$ 

```



Correttezza

Caso base array già ordinato, 0 o 1 elemento.

Induzione Abbiamo, dopo Partition

$$\boxed{\leq A[q] \mid A[q] \mid \geq A[q]}$$

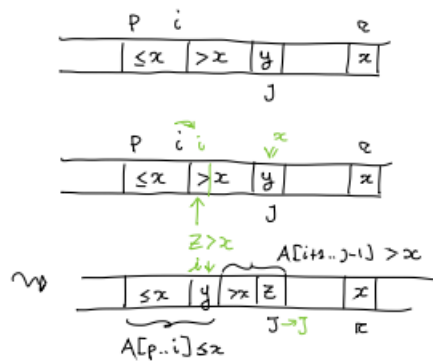
$$\text{Quicksort}(A, p, q) \boxed{\leq A[q], \text{ord} \mid A[q] \mid > A[q]}$$

$$\text{Quicksort}(A, q+1, r) \boxed{\leq A[q], \text{ord} \mid A[q] \mid > A[q], \text{ord}}$$

inizializzazione $i = p-1$ $A[p..i] = A[p..p-1] = \emptyset$
 $j = p$ $A[i+1..j-1] = A[p..p-1] = \emptyset$

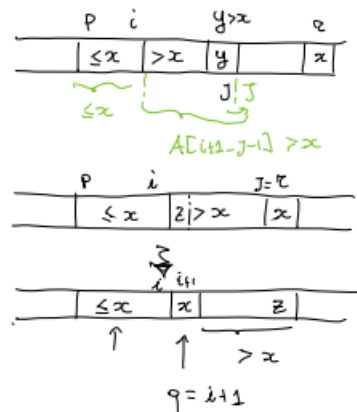
conservazione 1

$y \leq x$?
 SI



NO

conclusione



Esempio Dato l'array A , scelgo come **pivot** x l'ultimo elemento.

9	6	0	8	4	2
---	---	---	---	---	---

pivot:

2

i punta alla cella 0 (ossia nessuna cella)

j punta alla cella 1:

9

$9 > 2$? Sì $\Rightarrow j++$

$6 > 2$? Sì $\Rightarrow j++$

$0 > 2$? No $\Rightarrow i++, A[i] \leftrightarrow A[j], j++$

0	6	9	8	4	2
---	---	---	---	---	---

pivot:

2

i punta alla cella 1:

0

j punta alla cella 4:

8

$8 > 2$? Sì $\Rightarrow j++$

$4 > 2$? Sì $\Rightarrow j++$

Scambio $A[i+1]$ con x , ottenendo

0	2	9	8	4	6
---	---	---	---	---	---

I primi due $(i + 1)$ elementi sono ordinati:

0	2
---	---

Chiamo ricorsivamente **Quicksort** con $q = i + 1$.

9	8	4	6
---	---	---	---

pivot:

6

i punta alla cella 0 (ossia nessuna cella)

j punta alla cella 1:

9

$9 > 6$? Sì $\Rightarrow j++$

$8 > 6$? Sì $\Rightarrow j++$

$4 > 6$? No $\Rightarrow i++, A[i] \leftrightarrow A[j], j++$

4	8	9	6
---	---	---	---

pivot:

2

i punta alla cella 1:

4

j punta alla cella 4:

6

, quindi ho finito.

Scambio $A[i+1]$ con x , ottenendo

4	6	9	8
---	---	---	---

I primi due $(i + 1)$ elementi sono ordinati:

4	6
---	---

Chiamo ricorsivamente **Quicksort** con $q = i + 1$.

9	8
---	---

pivot:

8

i punta alla cella 0 (ossia nessuna cella)

j punta alla cella 1:

9

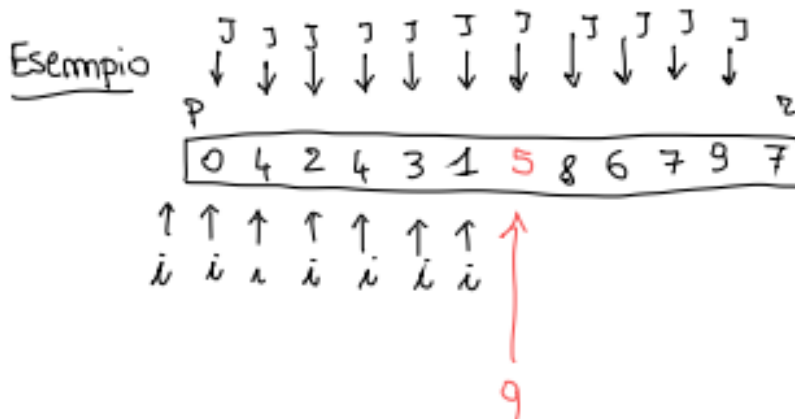
$9 > 8$? Sì $\Rightarrow j++$

Ho finito, scambio $A[i+1]$ con x , ottenendo

8	9
---	---

Guardando l'array completo ottengo il risultato atteso:

0	2	4	6	8	9
---	---	---	---	---	---



Per quanto riguarda la complessità di Quicksort:

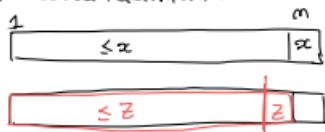
Partition costa $\Theta(n)$

$$T^{QS} = \Theta(n) + T^{QS}(q-p) + T^{QS}(n-(q-p)-1)$$

$q-p < n$

Caso peggiore $\rightarrow$ Massimo sbilanciamento e occorre controllare tutto l'array, controllando partizione per partizione.

massimo sbilanciamento



$$T^{QS} = \Theta(n) + T^{QS}(n-1) = \Theta(n^2) \quad (\Theta(n) = cn)$$

$$T(n)$$

$$cn$$

$$cn - 1$$

$$cn - 2$$

$$\dots$$

$$d$$

Infatti, si considera da un punto di vista

matematico l'esame di tutta la partizione,

considerando un esame in due parti (lineare per due volte), di entrambe le partizioni e sintetizzando il tutto con una sommatoria di Gauss:

$$\sum_{j=1}^{n-1} c(n-j) + d = \sum_{k=1}^n ck + d =$$

$$= c \sum_{k=1}^n k + d \quad \left(\frac{c(n+1)n}{2} + d = \Theta(n^2) \right)$$

$$T(n) = \Theta(n^2) \Rightarrow \begin{cases} = O(n^2) \\ = \Omega(n^2) \end{cases}$$

In modo concreto, si può invece vedere che esso è visibile come una ricorrenza con termine $(an + b)$ lineare per esaminare tutte le partizioni ed $(n-1)$ ipotizzando l'ordinamento successivo di ogni elemento, risolvibile (volendo) per sostituzione.

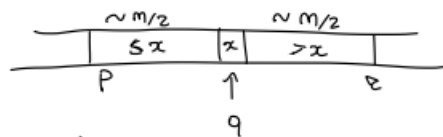
$$T(m) = \begin{cases} c & m \leq 1 \\ T(m-1) + am + b & m > 1 \end{cases}$$

Da un punto di vista di complessità:

- $T(n) = O(n^2)$

$$\begin{aligned}
 T(n) &= O(n^2) \Rightarrow T(n) \leq cn^2 \quad \forall n \geq n_0, c > 0 \\
 &= T(n-1) + \Theta(n) \leq dn \\
 &\leq c(n-1) + dn \\
 &= cn^2 - 2cn + c + dn \leq cn^2 \\
 &2cn - dn - c \geq 0 \\
 &n(2c - d) - c \geq 0 \quad \text{ok, } c > \frac{d}{2} \\
 T(n) &= O(n^2)
 \end{aligned}$$

- $T(n) = \Omega(n^2)$ analogo.



$$\begin{aligned}
 T(n) &= 2T\left(\frac{n}{2}\right) + \Theta(n) \\
 \uparrow \quad \uparrow \quad \uparrow \\
 a=2 \quad b=2 \quad f(n)
 \end{aligned}$$

Master Theorem

$$n \log_2^2 = n^1 = n$$

$$f(n) = \Theta(n)$$

↳ CASO 2

$$T(n) = \Theta\left(n^{\frac{4}{2}} \log n\right) = \Theta(n \log n)$$

$$T_{min}^{QS}(n) = \Theta(n \log n)$$

Caso migliore → Partizione perfettamente bilanciata

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) = \Theta(n \log n)$$

Analizzare il caso medio è un po' più complicato. Assumiamo che l'array sia in ordine casuale, in modo che ogni elemento abbia la stessa probabilità di essere selezionato come perno. In alternativa, possiamo selezionare il perno in modo casuale, in modo che l'ordine originale dell'array non abbia importanza; in entrambi i casi si giunge alla stessa conclusione.

Se i numeri dell'array sono [1, 2, 3, 4, 5], ad esempio, ogni numero ha una probabilità di 1/5 di essere scelto come perno.

- Se viene scelto 1 come perno, allora le chiamate ricorsive sono su array di dimensioni 0 e 4. Se 2 è il perno, allora le chiamate ricorsive sono su array di dimensioni 0 e 4.
- Se il perno è il 2, le chiamate ricorsive sono su array di dimensione 1 e 3.
- Se il perno è 3, le chiamate ricorsive saranno effettuate su array di dimensione 2 e 2. Se il perno è 4, le chiamate ricorsive saranno effettuate su array di dimensione 0 e 4.
- Se il perno è 4, le chiamate ricorsive verranno effettuate su array di dimensione 3 e 1.
- Se il perno è 5, le chiamate ricorsive saranno effettuate su array di dimensione 4 e 0. La ricorsività è quindi che T è il risultato di una serie di chiamate ricorsive.

Quindi la ricorrenza è che $T(5)$ è $1/5$ di $T(4) + T(0)$, $T(3) + T(1)$, $T(2) + T(2)$, $T(1) + T(3)$ e $T(0) + T(4)$, più un termine $O(n)$ per il costo del partizionamento. La forma generale di questa relazione di ricorrenza è una somma su ogni possibile pivot, divisa per il numero di pivot possibili:

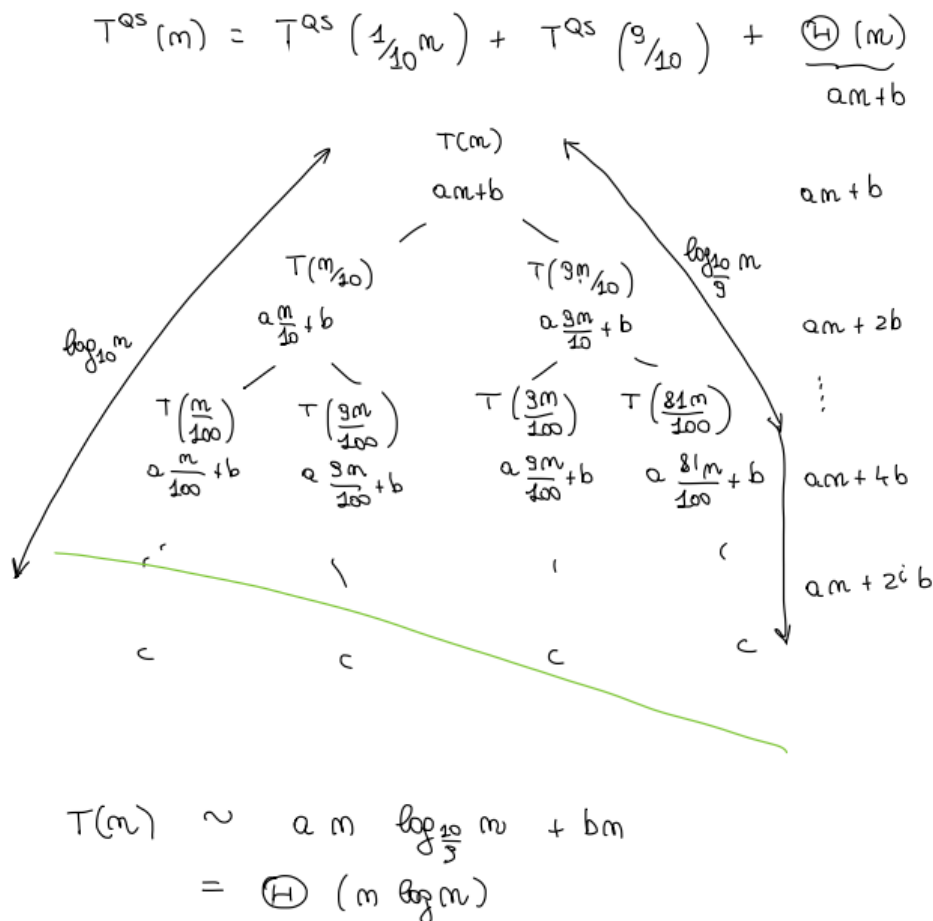
$$T(n) = \left(\frac{1}{n} \sum_{i=0}^{n-1} T(i) + T(n-i-1) \right) + O(n)$$

La soluzione a questa relazione di ricorrenza è che $T(n)$ è in $O(n \log n)$.

Quanto appena presentata è la situazione più tipica: alternanza di partizionamenti "ottimi" e "pessimi".

Come mostrato dall'esempio, l'idea è che si abbia un partizionamento proporzionale p nel caso in cui si ipotizzi, partendo da un partizionamento casuale, una situazione del tipo:

$$T(n) = T\left(\frac{n}{10}\right) + T\left(\frac{9n}{10}\right) + \Theta(n) = \Theta(n \log n)$$



Normalmente, al di là di questi due casi, l'input non ha una distribuzione uniforme, ma è quasi ordinato. Si usa una versione di *Partition* in grado di rendere impossibile il partizionamento costante.

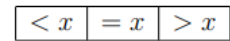
RANDOMIZED-PARTITION(A, p, r)

- 1 $q = \text{RANDOM}(p, r)$
- 2 $A[q] \leftrightarrow A[r]$
- 3 **return** **PARTITION**(A, p, r)

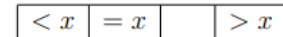
Esiste una variante di QuickSort a Tre Partizioni.

Quicksort con *RandomizedPartition* funziona bene ed evita, quasi in ogni circostanza, di imbattersi nel caso pessimo, ad eccezione di un caso particolare: se in input viene dato un array con tutti gli elementi uguali, si ottiene il temuto caso pessimo $O(n^2)$.

Per ovviare al problema, è sufficiente partizionare Quicksort in tre partizioni invece di due.



Durante l'algoritmo, la disposizione sarà questa:



Dato un pivot x , si partiziona A come a lato:

(La cella vuota è la regione ancora da esplorare).

In *3 Way QuickSort/Tripartition*, un array $A[l..r]$ viene diviso in 3 parti:

- a) $A[l..i]$ elementi inferiori al pivot.
- b) $A[i+1..j-1]$ elementi uguali al perno.
- c) $A[j..r]$ elementi maggiori del perno.

TRIPARTITION(A, p, r)

```

1   $x = A[r]$ 
2   $i = p - 1$ 
3   $k = p$ 
4   $j = r$ 
5  while  $k < j$ 
6      if  $A[k] < x$ 
7           $i = i + 1$ 
8           $A[i] \leftrightarrow A[k]$ 
9           $k = k + 1$ 
10     else if  $A[k] > x$ 
11          $j = j - 1$ 
12          $A[j] \leftrightarrow A[k]$ 
13     else
14          $k = k + 1$ 
15     //  $k = j$ 
16      $A[j] \leftrightarrow A[r]$ 
17 return  $(i + 1, j)$  // restituisce una coppia di valori
```

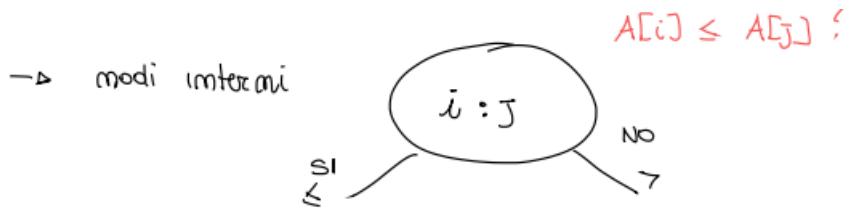
QUICKSORT(A, p, r)

```

1  if  $p < r$ 
2       $q_1, q_2 = \text{TRIPARTITION}(A, p, r)$ 
3      QUICKSORT( $A, p, q_1 - 1$ )
4      QUICKSORT( $A, q_2 + 1, r$ )
```

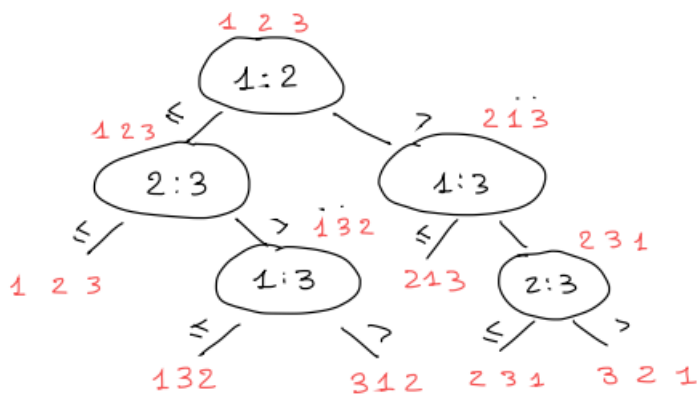
In merito all'ordinamento, consideriamo una *permutazione* (insieme di elementi casuali) ordinata di $a_1, \dots, a_n$. L'ordinamento è basato comunque su confronti e, immaginando ciascun elemento sia distinto, anche contando solo *alcune* operazioni, viene garantito un bilanciamento qualora gli elementi siano disposti casualmente.

Similmente, un algoritmo può essere rappresentato attraverso un albero di decisione. Esso è una rappresentazione astratta dell'esecuzione di un algoritmo su input di dimensione prefissata. Semplicemente, un nodo rappresenta un valore e ogni ramo a sx rappresenta un valore $\leq$ (generalmente anche un valore di scelta Sì) e ogni a ramo a dx rappresenta un valore $>$ (generalmente anche un valore di scelta No). L'altezza in un albero di decisione dà *il numero di confronti necessari*.



→ foglie : output permutazione dell'input

Per esempio, su *InsertionSort*, con un array $A[1,2,3]$ con $n = 3$:



```

InsertSort ( A, m )
  for j = 2 to m
    i = j - 1
    while ( i > 0 and ( A[i] > A[i+1] ) )
      A[i] ↔ A[i+1]
      i = i - 1
  
```

Dato che ogni foglia corrisponde ad una decisione, allora:

- ogni permutazione deve comparire in almeno una foglia (tutte hanno valori diversi)
- ogni foglia contiene una sola permutazione

Normalmente, la complessità è di ordine $O(n \log n)$.

$$\begin{aligned}
 h &\geq \log_2 n! \\
 &\geq \log_2 \left(n(n-1)(n-2) \dots \frac{n}{2} \right) \\
 &\geq \log_2 \left(\frac{n}{2} \left(\frac{n}{2} - 1 \right) \left(\frac{n}{2} - 2 \right) \dots \frac{n}{2} \right) \\
 &\geq \log_2 \left(\frac{n}{2} \right)^{\left(\frac{n}{2} \right)} = \frac{n}{2} (\log_2 n - \log_2 2) = \frac{n}{2} (\log_2 n - 1) = \Theta(n \log n)
 \end{aligned}$$

Lezione 7: Counting Sort

Il counting sort è una tecnica di ordinamento stabile, utilizzata per ordinare gli oggetti in base alle chiavi che sono piccoli numeri. Conta il numero di chiavi i cui valori sono uguali. Questa tecnica di ordinamento è efficiente quando la differenza tra le diverse chiavi non è così grande, altrimenti può aumentare la complessità dello spazio.

Come funziona:

1. Scopri l'elemento massimo (sia `max`) dall'array specificato.

max								
8	4	2	2	8	3	3	1	

2. Inizializzare una matrice di lunghezza `max+1` con tutti gli elementi 0. Questa matrice viene utilizzata per memorizzare il conteggio degli elementi nella matrice.

0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8

3. Memorizza il conteggio di ciascun elemento nel rispettivo indice nell'array `count`.
Ad esempio: se il conteggio dell'elemento 3 è 2, 2 viene memorizzato nella terza posizione dell'array `count`. Se l'elemento "5" non è presente nell'array, 0 viene memorizzato nella quinta posizione.

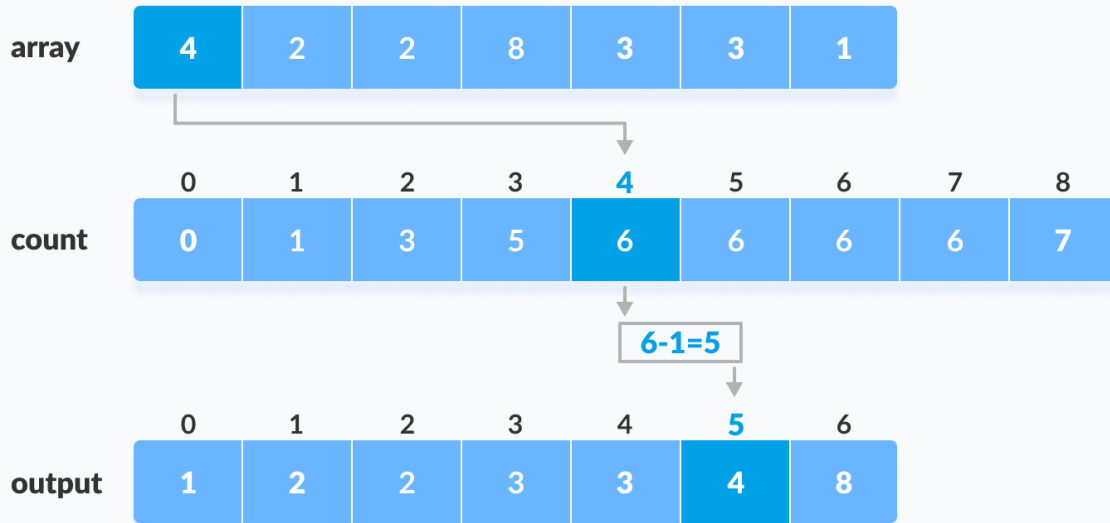
0	1	2	2	1	0	0	0	1
0	1	2	3	4	5	6	7	8

4. Memorizzare la somma cumulativa degli elementi dell'array `count`. Aiuta a posizionare gli elementi nell'indice corretto dell'array ordinato.

0	1	3	5	6	6	6	6	7
0	1	2	3	4	5	6	7	8

5. Individuare l'indice di ogni elemento della matrice originale nell'array `count`.
Questo dà il conteggio cumulativo. Posizionate l'elemento in corrispondenza

dell'indice calcolato come mostrato nella figura seguente.



6. Dopo aver posizionato ogni elemento nella sua posizione corretta, diminuisci il suo conteggio di uno.

COUNTING-SORT(A, B, k)

```

1   $C[0 \dots k] \leftarrow 0$ 
2  for  $j = 1$  to  $A.length$ 
    //  $C[x] = \#$  elementi in  $A$  con valore  $x$ 
3     $C[A[j]] = C[A[j]] + 1$ 
4  for  $i = 1$  to  $k$ 
    //  $C[x] = \#$  elementi in  $A$  con valore  $\leq x$ 
5     $C[i] = C[i - 1] + C[i]$ 
6  for  $j = A.length$  down to 1
7     $B[C[A[j]]] = A[j]$ 
8     $C[A[j]] = C[A[j]] - 1$ 

```

Cerchiamo di capirlo con l'aiuto di un esempio.

1. Per semplicità, considerare i dati nell'intervallo da 0 a 9.
2. Dati di input: {1, 4, 1, 2, 7, 5, 2}
3. Prendere un array di conteggio per memorizzare il conteggio di ogni oggetto univoco.

For simplicity, consider data in range of 0 to 9

1	4	1	2	7	5	2
---	---	---	---	---	---	---

Index :	0	1	2	3	4	5	6	7	8	9
	0	0	0	0	0	0	0	0	0	0

Count each element in the given array and place the count at the appropriate index.

1. Ora memorizza il conteggio di ogni elemento univoco nell'array count
2. Se un elemento si ripete, è sufficiente aumentarne il numero.

For simplicity, consider data in range of 0 to 9

1	4	1	2	7	5	2
---	---	---	---	---	---	---

Index :	0	1	2	3	4	5	6	7	8	9
	0	2	0	0	1	0	0	0	0	0

Qui, il conteggio di ogni elemento univoco nella matrice di conteggio è come mostrato di seguito:

Index: 0 1 2 3 4 5 6 7 8 9

Count: 0 2 2 0 1 1 0 1 0 0

For simplicity, consider data in range of 0 to 9

1	4	1	2	7	5	2
---	---	---	---	---	---	---

Index :	0	1	2	3	4	5	6	7	8	9
	0	2	2	0	1	1	0	1	0	0

Modify the count array by adding the previous counts.

Modificare l'array count in modo che ogni elemento in ogni indice memorizzi la somma dei conteggi precedenti.

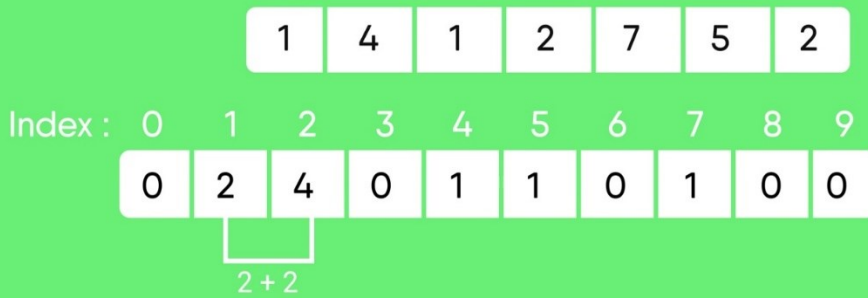
Index: 0 1 2 3 4 5 6 7 8 9

Count: 0 2 4 4 5 6 6 7 7 7

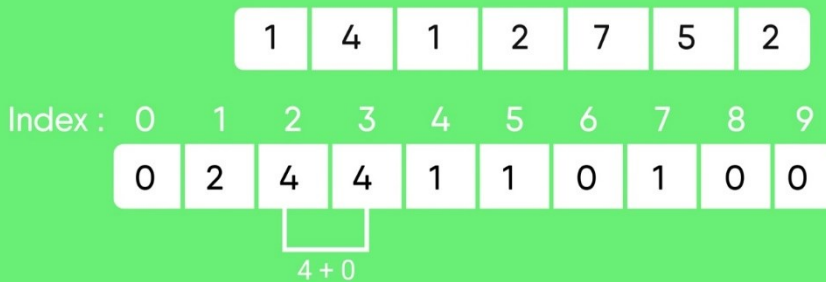
L'array di count modificato indica la posizione di ciascun oggetto nella sequenza di output.

Individuare l'indice di ogni elemento dell'array originale nell'array count. Questo dà

For simplicity, consider data in range of 0 to 9



For simplicity, consider data in range of 0 to 9



Ruotare l'array in senso orario per una volta.

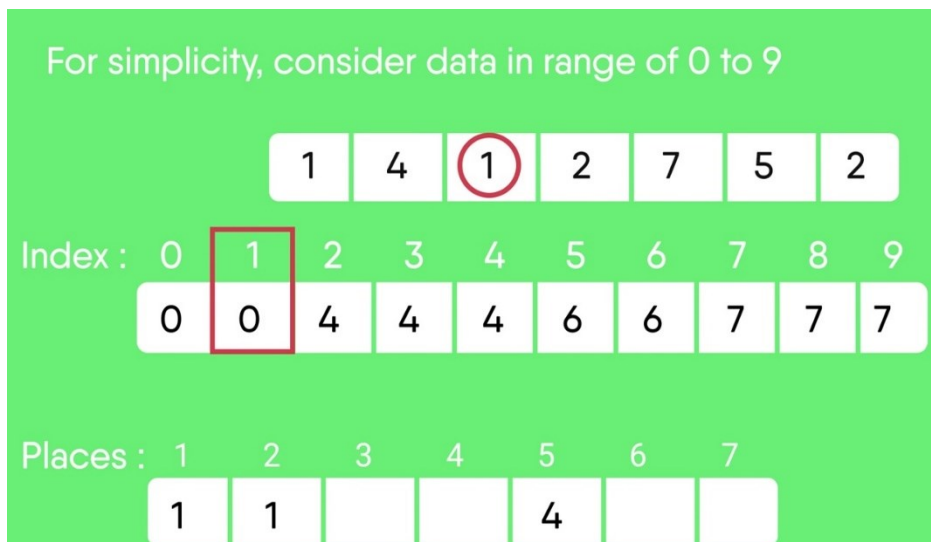
Index: 0 1 2 3 4 5 6 7 8 9

Count: 0 0 2 4 4 5 6 6 7 7

For simplicity, consider data in range of 0 to 9



1. Output di ogni oggetto dalla sequenza di input seguito dall'aumento del conteggio di 1.
2. Elaborare i dati di input: 1, 4, 1, 2, 7, 5, 2. La posizione di 1 è 0.
3. Metti i dati 1 all'indice 0 in output. Aumentare il conteggio di 1 per posizionare il dato successivo 1 in un indice 1 maggiore di questo indice.



Dopo aver posizionato ogni elemento nella sua posizione corretta, diminuisci il suo conteggio di uno.

La complessità è lineare, tecnicamente $O(n + k)$, dove n è il numero di elemento e k è il range (tra minimo e massimo). Il problema di CountingSort è proprio la memoria. Infatti, al crescere di k , la memoria richiesta per allocare C cresce esponenzialmente.

Complessità

$C[0, k] \leftarrow 0$	$\Theta(k)$
for $j=1 \dots$	$\Theta(n)$
for $i=1 \dots$	$\Theta(k)$
for $j=A.length \dots$	$\Theta(n)$
Somma $\Theta(n + k)$ con $k = \Theta(1) \Rightarrow \Theta(n)$	

Per capire perché l'ordinamento per conteggio è stabile, è necessario comprendere che l'ordinamento per conteggio non può essere utilizzato solo per ordinare un elenco di numeri interi, ma anche per ordinare un elenco di elementi la cui chiave è un numero intero, e questi elementi saranno ordinati in base alle loro chiavi con informazioni aggiuntive associate a ciascuno di essi.

Poiché ogni elemento non contiene solo un intero come chiave, ma anche alcune informazioni aggiuntive, anche se la loro chiave è la stessa, si può dire che ogni elemento è diverso utilizzando le informazioni aggiuntive, in modo da poter dire se si tratta di un algoritmo di ordinamento stabile (sì, è un algoritmo di ordinamento stabile se implementato in modo appropriato).

Per realizzare la stabilità, si scorre A dal fondo nell'ultimo ciclo. Questo *preserva gli elementi nel loro ordine originale*. Altri algoritmi stabili sono Insertion Sort e MergeSort mentre QuickSort ed Heapsort non sono stabili. Tecnicamente:

Dato $A[1 \dots n]$ in input, se $A[i] \leq A[j]$ con $i \leq j$, allora nell'output $A[i]$ e $A[j]$ sono nello stesso ordine relativo.

Esiste una possibile semplificazione dell'algoritmo: essa conta sempre gli elementi e li riorganizza riordinandoli come visto.

La differenza sta nel fatto che se ci sono elementi che compaiono poche volte o anche una sola, l'algoritmo le posiziona direttamente sapendo che è ordinato. Esso ragiona sulla chiave e non sul valore; invece, la versione generalizzata usa gli oggetti e, anche se gli oggetti compaiono più volte, non effettua alcuna ottimizzazione.

Questo è possibile grazie al salvataggio direttamente in C con A interno; permette quindi di risparmiare un ciclo di analisi, lavorando direttamente sul conteggio degli elementi e accorgendosi quando li deve ordinare. Funziona solo se non ci sono dati satellite (quindi, in sola presenza di un array lineare).

Counting Sort (A, B, k)

```

C[0..k] ← 0
for j = 1 to A.Length
    C[A[j]]++

i = 1
for j = 0 to k
    while C[j] > 0
        B[i] = j
        C[j]--
        i++
    
```

Lezione 8: Radix Sort

Il Radix sort è un algoritmo di ordinamento che ordina gli elementi raggruppando prima le singole cifre tutte nella stessa attuale posizione. Quindi, ordina gli elementi in base al loro ordine crescente/decescente. L'idea è quella di ordinare cifra per cifra con un algoritmo stabile e risolvere i problemi di memoria di Counting Sort.

Ci sono due idee per farlo:

- 1) partire dalla cifra più significativa
- 2) ordinare dalla cifra meno significativa a quella più significativa

ESEMPIO

3 2 9 ←	7 2 0 -	→ 7 2 0	3 2 3
4 5 7 ←	3 5 5 -	→ 3 2 3	3 5 5
6 5 7 ←	4 5 7 -	→ 8 3 9	4 3 9
8 3 9 ←	6 5 7 -	→ 4 3 9	4 5 7
4 3 9 ←	3 2 9 -	→ 3 5 5	6 5 7
7 2 0 ←	8 3 9 -	→ 4 5 7	7 2 0
3 5 5 ←	4 3 9 -	→ 6 5 7	8 3 9

Input: $A[1..n]$ con $A[i]$ di d cifre e base b , $A[i] = a_d a_{d-1} \dots a_1$.

RADIX-SORT(A, d)

- 1 for $j = 1$ to d
- 2 ordina A rispetto alla cifra j // $A^j[i] = a_j a_{j-1} \dots a_1$
con COUNTING-SORT // A^{j-1} ordinato

Correttezza → Invariante A^{j-1} ordinato

- Inizio: $j = 1$, $A^{j-1} = A^0$
- Mantenimento: A^{j-1} ordinato e ordino rispetto alla j -esima cifra con un algoritmo stabile

Scritto da Gabriel

$$A^j \text{ ordinato} \quad \forall i, i' \quad i \leq i' \quad A^{(j-1)}[i] \leq A^{(j-1)}[i']$$

2 Possibilità:

- 1) $a_j \neq a'_j \Rightarrow a_j < a'_j$
- 2) $a_j = a'_j \Rightarrow$ dato che abbiamo usato un algoritmo stabile $\Rightarrow A^{(j-1)}[i] \leq A^{(j-1)}[i']$

Complessità $\rightarrow$ d volte CountingSort $\Theta(n + b) \Rightarrow \Theta(d(n + b)) = \Theta(n)$
 (d cifre
 b base
 n elementi
 dell'array)

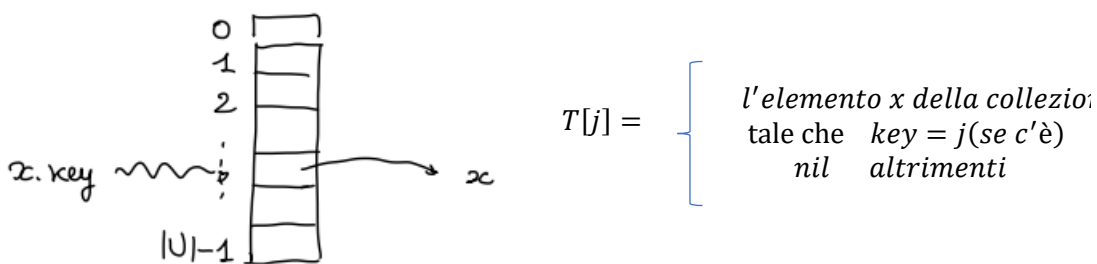
$$\begin{aligned} m \text{ bit, } r \text{ bit per cifra, } \frac{m}{r} \text{ cifre, base } 2^r \\ \Theta\left(\frac{m}{r}(m + 2^r)\right) &= r = \log_2 n \\ &= \Theta\left(\frac{m}{\log n}(n + 2^{\log n})\right) \\ &= \Theta\left(\frac{m}{\log n}n\right) \quad m = O(\log n) \\ &= \Theta(n) \end{aligned}$$

Lezione 9: Tabelle Hash

La tabella hash è una struttura di dati che memorizza i dati in modo associativo. In una tabella hash, i dati sono memorizzati in un formato array, dove ogni valore di dati ha un proprio valore di indice univoco. L'accesso ai dati diventa molto veloce se si conosce l'indice del dato desiderato.

In questo modo, diventa una struttura di dati in cui le operazioni di inserimento e ricerca sono molto veloci, indipendentemente dalla dimensione dei dati. La tabella Hash utilizza un array come supporto di memorizzazione e usa la tecnica dell'hash per generare un indice in cui un elemento deve essere inserito o da cui deve essere individuato.

In particolare, si considera U come insieme (universo) della chiavi, visto come $U = 0, 1, \dots, |U| - 1$
 La tabella hash T viene vista come array $T[0, \dots, |U| - 1]$, in cui un elemento x è inserito in $T[x. \text{key}]$



Problemi:

- 1) Non è possibile avere oggetti con la stessa chiave
- 2) OK se il numero $|U|$ è piccolo

Consideriamo 3 operazioni:

INSERT(T, x)

1 $T[x.key] = x \ // \ \Theta(1)$

DELETE(T, x)

1 $T[x.key] = \text{NIL} \ // \ \Theta(1)$

SEARCH(k)

1 **return** $T[k] \ // \ \Theta(1)$

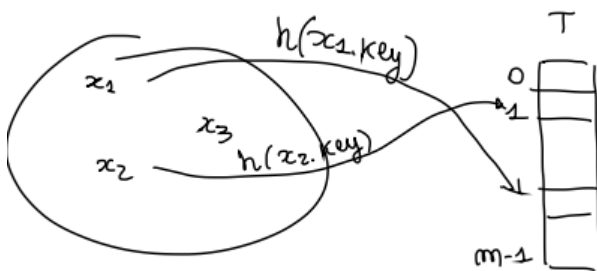
Problema e.g. consideriamo che la key sia di 8 caratteri (e 8 bit per rappresentare un carattere). Risulta molto costosa in termini di memoria la tabella hash.

1 carattere $\rightarrow$ 8 bit $\rightarrow 2^8$ caratteri

$2^8 \times \dots \times 2^8$ stringhe $= 2^{64}$ stringhe $\approx 16 * 10^8$

Obiettivo: Usare quantità di memoria proporzionale al numero di elementi da memorizzare

Idea: Creazione di una tabella T di dimensione $m \ll |U|$



$h: U \rightarrow [0..m-1]$
funzione di hashing

$h: U \rightarrow \{0, 1, \dots, m-1\}$ funzione di **hashing**

$n = \#$ elementi memorizzati nella tabella T

$m = \#$ celle

Se $n > m$, esisteranno $x_1, x_2: h(x_1.key) = h(x_2.key) \Rightarrow$ conflitto

La collisione/conflitto si verifica con n = numero di elementi memorizzati se $m \ll |U|$ se $n > m$.

Il principio è il pigeonhole principle, quindi nella tabella hash, ogni valore trova una corrispondenza.

Il pigeonhole principle è una delle idee più semplici ma più utili della matematica e può aiutarci in questo caso.

Una versione di base dice che se $(N+1)$ piccioni occupano N buche, allora in qualche buca devono esserci almeno 2 piccioni. Quindi, se 5 piccioni occupano 4 buche, deve esserci una buca con almeno 2 piccioni.

Ci sono due soluzioni possibili:

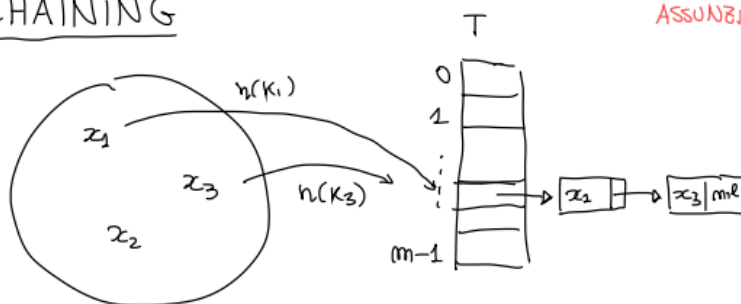
- 1) Chaining
- 2) Open Addressing

Il Chaining propone come soluzione quella di mettere sulla tabella liste dinamiche di elementi, invece che singoli elementi, in modo che in caso si incorra in una cella già occupata dopo un hashing, l'elemento venga inserito in coda (o in testa) alla lista.

Nell'approccio di concatenamento, la tabella hash è una matrice di elenchi collegati, cioè ogni indice ha un proprio elenco collegato. Tutte le coppie chiave-valore mappate allo stesso indice saranno memorizzate nell'elenco collegato di quell'indice.

L'assunzione è che h (funzione di hash) si calcola in $O(1)$.

① CHAINING



ASSUNZIONE: h (funzione di hash)
si calcola in $O(1)$

$T[0, \dots, m-1]$ contiene liste

$T[j] =$ lista degli elementi x t.c. $h(x.key) = j$

Idea $T[i] =$ lista elementi x tali che $h(x.key) = i$

Ci sono come operazioni:

INSERT(T, x)

- 1 Inserisci x nella lista $T[h(x.key)]$ // $O(1)$

DELETE(T, x)

- 1 Elimina x da $T[h(x.key)]$ // $O(1)$

SEARCH(T, k)

- 1 Cerca in $T[h(k)]$ un elemento x tale che $x.key = k$ // $O(n)$

L'operazione peggiore è quella di ricerca, dove alla peggio la complessità è lineare.

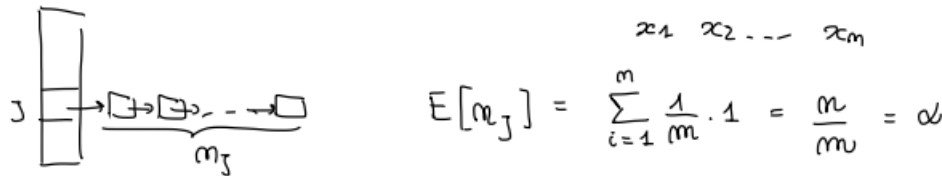
Il caso medio, invece, partendo da:

- $m = \#$ celle tabella
- $n = \#$ elementi inseriti
- $\alpha = n/m$ che può essere minore, maggiore oppure uguale ad 1.

La qualità di una funzione hash si valuta in base alla distribuzione dell'input.

Ipotesi: *Hashing uniforme semplice*

Ogni elemento x in input ha la stessa probabilità di $1/m$ di essere "indirizzato" in una qualunque delle m celle. Esso è indicato come $n_j =$ lunghezza lista in $T[j]$



$$E[n_j] = \sum_{i=1}^m \frac{1}{m} \cdot 1 = \frac{m}{m} = \alpha$$

Il *costo medio di Search* è dato dalla ricerca di una chiave non presente k :

- calcolo $h(k) = j$
- accedo alla lista $T[j]$
- scorro la lista $T[j]$ fino in fondo (n_j elementi) $\rightarrow$ costo n_j

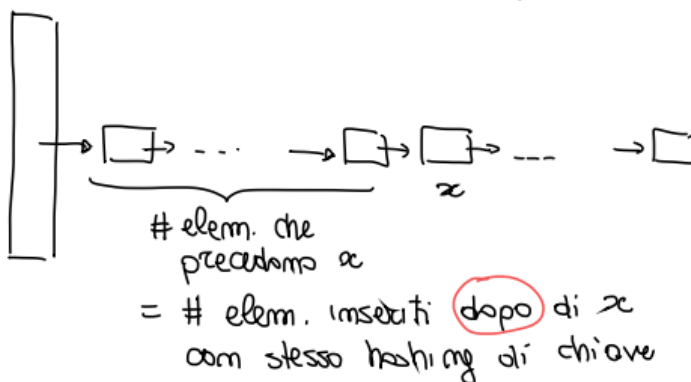
Costo: $\theta(1 + \alpha)$

Ricerca di una chiave presente

- calcolo $h(k) = j$
- accedo alla lista $T[j]$
- scorro la lista $T[j]$ fino a trovare l'elemento con chiave k

Costo $\rightarrow 1 + n_j/2 \Rightarrow 1 + \alpha/2$

Più precisamente: cerco x con chiave $x.key = k$.



Se $x_1, x_2, \dots, x_n$ sono gli elementi inseriti

Costo della ricerca di x_i :

$$\begin{aligned}
 & 1 + \# \text{ elementi } x_j : j > i, h(x_i.key) = h(x_j.key) \\
 & = 1 + \sum_{j=i+1}^n (\text{prob } h(x_i.key) = h(x_j.key)) \\
 & = 1 + \sum_{j=i+1}^n \frac{1}{m} = 1 + \frac{n-i}{m} \\
 & \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{n-i}{m} \right) \\
 & = \frac{1}{n} \left(n + \sum_{i=1}^n \frac{n-i}{m} \right) = \frac{1}{n} \left(n + \frac{1}{m} \sum_{z=0}^{n-1} z \right) \\
 & = 1 + \frac{1}{m \cdot n} \cdot \frac{n(n-1)}{2} = 1 + \frac{n}{2m} - \frac{1}{2m} \cdot \left(\frac{n}{n} \right) \quad \left(\alpha = \frac{n}{m} \right) \\
 & = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} = \Theta(1 + \frac{\alpha}{2}) = \Theta(1 + \alpha)
 \end{aligned}$$

In conclusione: costo della ricerca (in media)

- elemento assente $\Theta(1 + \alpha)$
- elemento presente $\Theta(1 + \alpha/2)$

Nota: se assumiamo $n \leq c m$ $\alpha = n/m \leq c$ (c costante)

Come definire (buone) funzioni di hash?

Nota: se le chiavi fossero numero reali $k \in [0,1)$

$$h(k) = \lfloor mx \rfloor$$

soddisfa la proprietà di hashing uniforme.

L'ipotesi di hash uniforme semplice dipende dalle probabilità con cui vengono estratti gli elementi da inserire; probabilità che in genere non sono note. Le funzioni hash che descriveremo assumono che le chiavi siano degli interi non negativi.

Metodo della divisione

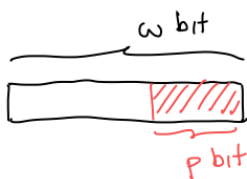
$$U = \{0, 1, \dots, |U| - 1\}$$

$$h(k) \in \{0, 1, \dots, m - 1\}$$

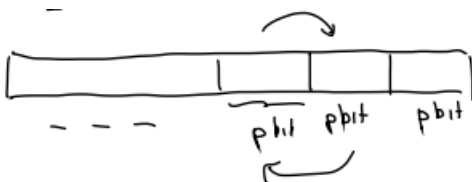
$$h(k) = k \bmod m$$

La scelta di m è critica

- $m = 2^p$



- $m = 2^p - 1$



$h(k) = h(k')$ se k si ottiene "permutando" k' . Es. $\rightarrow$ numero divisibile per 9 $\Leftrightarrow \sum$ cifre divisibile per 9

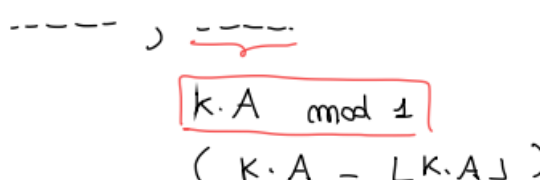
Metodo della moltiplicazione

Se $x \in [0, 1)$ $h(x) = \lfloor mx \rfloor$

ma la nostra chiave è $k \in 0, 1, \dots, |U| - 1$

Dato k

$\rightarrow$ fisso costante $A \in (0, 1)$ ($0 < A < 1$)

$\rightarrow k * A =$ 

$$h(k) = \lfloor m(k * A \bmod 1) \rfloor$$

Pro:

- m non critico
- A non critico ($A = (\sqrt{5} - 1)/2$)

Contro:

$$m = 2^p \quad w = \# \text{ bit parola}$$

$$A = \frac{q}{2^w} \quad 0 < q < 2$$

$$m(kA \bmod 1)$$

$$= m \left(k \frac{q}{2^w} \bmod 1 \right) \quad (\text{shift di } w \text{ bit, prendo la parte decimale } ka \bmod 1 \text{ e la moltiplico per } m = 2^p)$$

= p bit più significativi della parola meno significativa di $k * q$

Open Addressing

Idea: memorizzo gli elementi dell'insieme dinamico solo nello spazio della tabella

item ₂
item ₁
item ₃

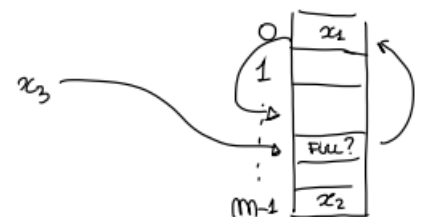
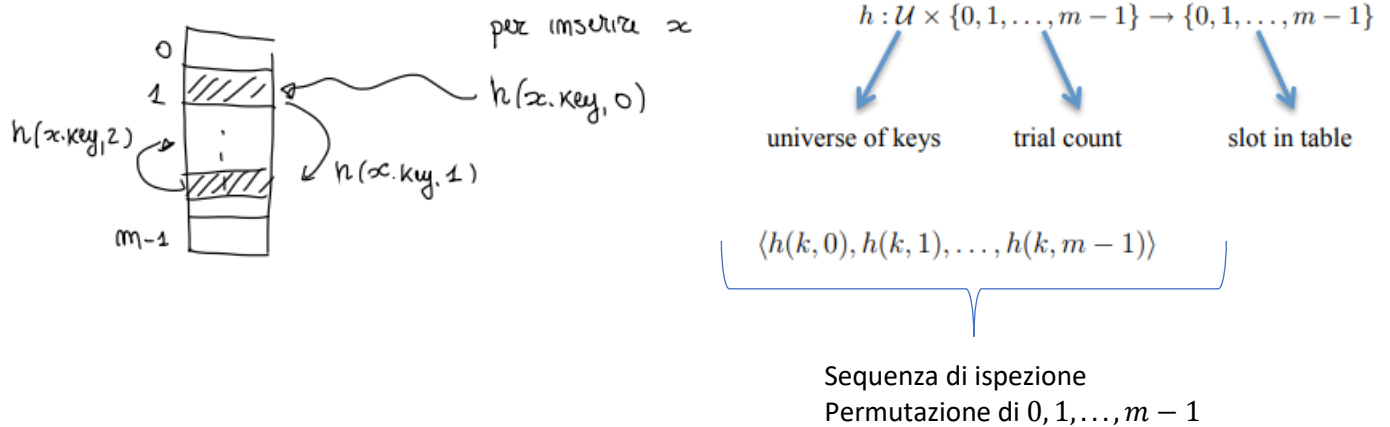


Figure 1: Open Addressing Table

Funzione di hash $h(k, i)$ con k chiave ed i # tentativo.

La funzione hash specifica l'ordine degli slot da sondare (provare) per una chiave (per inserire/ricercare/cancellare).



$h(k, i)$: k è la chiave, i è il tentativo.

Provo con $h(k, 0)$: se capito in una cella occupata, provo con $h(k, 1)$, poi $h(k, 2)$ e così via, fino a che non trovo una cella libera.

Per esplorare tutta la tabella:

$$h(k, 0), h(k, 1), \dots, h(k, m-1) \quad \forall k \in U$$

che è una permutazione di

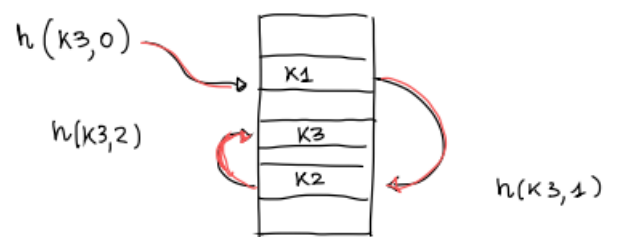
$$\{0, 1, \dots, m-1\}$$

Operazioni possibili:

INSERT(T, x)

```

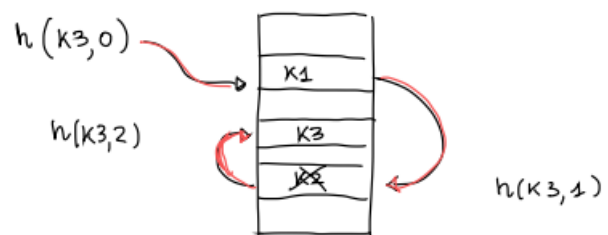
1   $i = 0$ 
2  repeat
3     $j = h(x.key, i)$ 
4    if ( $T[j] = \text{NIL}$ ) or ( $T[j] = \text{DELETED}$ ) // posizione libera
5       $T[j] = x$ 
6      return  $j$ 
7     $i = i + 1$ 
8  until  $i = m$ 
9  error
    
```



SEARCH(T, k)

```

1   $i = 0$ 
2  repeat
3     $j = h(k, i)$ 
4    if  $T[j].key = k$ 
5      return  $j$ 
6     $i = i + 1$ 
7  until ( $i = m$ ) or ( $T[j] = \text{NIL}$ )
8  return NOT FOUND
    
```

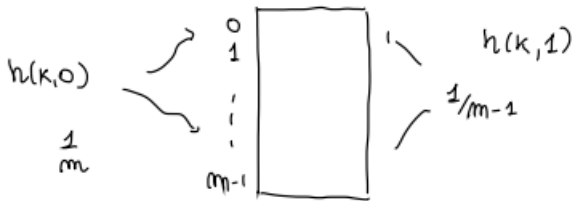


```
DELETE( $T, j$ )
1   $T[j] = \text{DELETED}$ 
```

L'Open Addressing risulta una soluzione inefficiente in caso avvengano molte cancellazioni. Similmente, giusto per citarla, è possibile anche attuare una *randomizzazione* dell'input, per avere una distribuzione più uniforme delle chiavi nelle liste e non dipendente dall'input. La complessità sarà sempre $\theta(1 + \alpha)$

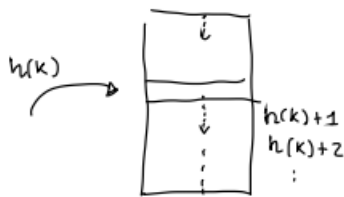
Hashing uniforme

Ogni elemento determina con la stessa probabilità una qualunque delle $m!$ sequenze di ispezione.

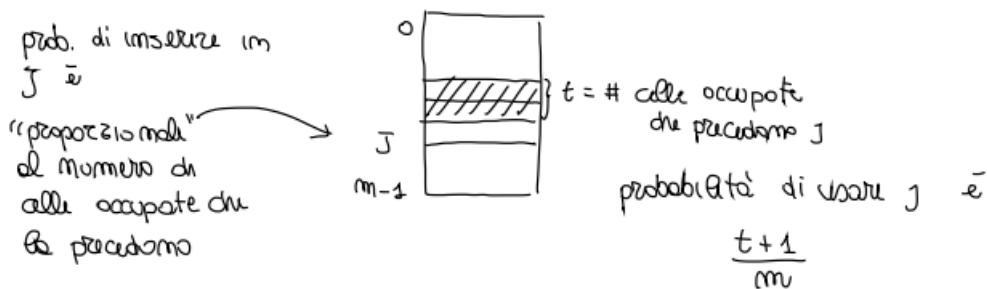


In questo contesto le *funzioni di hash* sono:

- 1) Ispezione lineare. Fisso $h(k)$ funzione di hash e $h'(k + 1) = (h(k) + i) \bmod m$



Vantaggi: Semplice / Poche permutazioni (m dipende solo da $h'(k)$)
Svantaggio: Addensamento primario (addensamenti di celle occupate)



di sequenze di ispezione = m (dipende solo da $h(k) \bmod m$)

- 2) Ispezione quadratica

Dato $h(k)$ funzione di hash $h'(k, i) = (h(k) + c_1 i + c_2 i^2) \bmod m$ con c_1, c_2 opportune e $c_2 > 0$

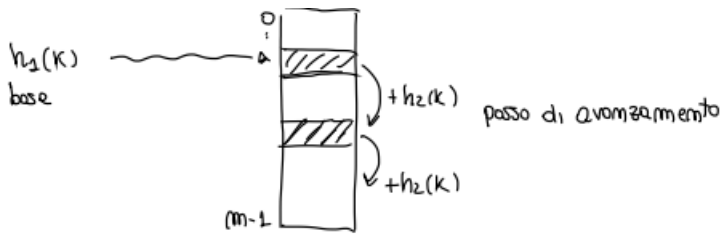
Addensamento secondario (addensamento quadratico di celle occupate)

sequenze di ispezione dipende solo da $h(k) \bmod m \Rightarrow m$ possibilità

- 3) Doppio Hashing

Siano $h_1(k), h_2(k)$ funzioni di hash di un solo argomento.

$h(k, i) = (h_1(k) + i h_2(k)) \bmod m$



Come mi assicuro che vengano ispezionate tutte le celle?

Condizione: $h_2(k)$ e m sono relativamente primi ($MCD(h_2(k), m) = 1$), quindi tutti i valori primi tra di loro

Dati $i, i' < m$

$$i \neq i' \Rightarrow h(k, i) \neq h(k, i')$$

Con m^2 sequenze di ispezione:

$$h(k, i): \{0, \dots, m-1\} \rightarrow \{0, \dots, m-1\}$$

iniettiva $\Rightarrow$ biiettiva

$$h(k, i) = h(k, i')$$

$$(h_1(k) + ih_2(k)) \bmod m = (h_1(k) + i'h_2(k)) \bmod m$$

$$((i - i')h_2(k)) \bmod m = (ih_2(k) - i'h_2(k)) \bmod m = 0$$

$$(i - i') \bmod m = 0$$

$$i \geq i' \quad i - i' < m - 1$$

$$\Rightarrow i - i' = 0$$

$$\Rightarrow i = i'$$

Esempio

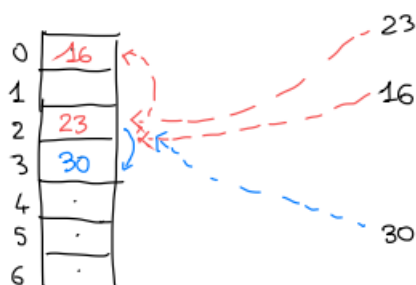
$$\begin{cases} m = 2^p \\ h_2(k) = 2h'(k) + 1 \end{cases} \quad \text{con } h' \text{ funzione di hash qualunque}$$

$$\begin{cases} m \text{ primo} \\ h_2(k) < m \end{cases} \quad h_2(k) = 1 + k \bmod m' \quad \text{con } m' < m \quad (\text{es. } m' = m - 1)$$

esempio

$$\begin{aligned} m &= 7 \\ m' &= 6 \end{aligned}$$

$$\begin{aligned} h_1(k) &= k \bmod 7 \\ h_2(k) &= 1 + k \bmod 6 \end{aligned}$$



$$h_1(k) = 2$$

$$h_1(k) = 2$$

$$h_2(k) = 1 + 16 \bmod 6 = 1 + 4 = 5$$

$$h_1(30) = 2$$

$$h_2(30) = 1 + 0 = 1$$

Costo

Il costo della funzione *Search* con *hashing uniforme* è riassunta come:

$$0 \leq \alpha = \frac{n}{m} \leq 1$$

- Ricerca di una chiave non presente

(a) $\frac{1}{1-\alpha}$ se $\alpha < 1$

(b) m se $\alpha = 1$

Probabilità di ispezionare la i -esima cella

cella	probabilità
$i = 0$	1
$i = 1$	prob. cella 0 occupata: $\frac{n}{m}$
$i = 2$	prob. cella 1 occupata: $\frac{n}{m} \cdot \frac{n-1}{m-1}$
$\dots$	
i	$\frac{n}{m} \cdot \frac{n-1}{m-1} \dots \frac{n-i+1}{m-i+1} \leq \alpha \cdot \alpha \dots \alpha = \alpha^i$

Valore atteso per # tentativi

$$1 + \alpha + \alpha^2 + \dots + \alpha^{i-1} + \dots + \alpha^{m-1}$$

(a) $\alpha < 1 \Rightarrow \frac{1-\alpha^m}{1-\alpha} \leq \frac{1}{1-\alpha}$

(b) m

- Ricerca di una chiave presente

(a) $\frac{1}{\alpha} \log \left(\frac{1}{1-\alpha} \right)$ $\alpha < 1$

(b) $1 + \log m$ $\alpha = 1$

Costo atteso per k_i :

$$\begin{aligned} \frac{1}{1 - \alpha_{i-1}} &= \frac{1}{1 - \frac{i-1}{m}} & \left(\alpha_{i-1} = \frac{i-1}{m} < 1 \right) \\ &= \frac{m}{m - i + 1} \end{aligned}$$

Il costo atteso è la media per $i = 1, \dots, n$

$$\begin{aligned} &= \frac{1}{n} \sum_{i=1}^n \frac{m}{m - i + 1} = \frac{m}{n} \sum_{i=1}^n \frac{1}{m - i + 1} & \left(\frac{m}{n} = \frac{1}{\alpha} \right) \\ &= \frac{1}{\alpha} \sum_{j=m-n+1}^m \frac{1}{j} \end{aligned}$$

- Se $\alpha < 1$

$$\begin{aligned} &\leq \frac{1}{\alpha} \int_{n-m}^m \frac{1}{x} dx \\ &= \frac{1}{\alpha} (\log m - \log(m-n)) = \frac{1}{\alpha} \left(\log \frac{m}{m-n} \right) \\ &= \frac{1}{\alpha} \log \frac{1}{\frac{m-n}{m}} \\ &= \frac{1}{\alpha} \log \left(\frac{1}{1 - \left(\frac{n}{m}\right)} \right) = \frac{1}{\alpha} \log \left(\frac{1}{1 - \alpha} \right) \end{aligned}$$

- Se $\alpha = 1$

$$\begin{aligned} \sum_{l=1}^m \frac{1}{l} &= 1 + \sum_{l=2}^m \frac{1}{l} \leq \int_1^m \frac{1}{x} dx \\ &= 1 + (\log m - \log 1) = 1 + \log m \end{aligned}$$

α	$\frac{1}{1-\alpha}$	$\frac{1}{\alpha} \log \left(\frac{1}{1-\alpha} \right)$
$\alpha = 0.3$	1.43	1.19
$\alpha = 0.5$	2.00	1.39
$\alpha = 0.7$	3.33	1.72
$\alpha = 0.9$	10	2.56

Confrontiamo le complessità dei due casi.

Lezione 10: Alberi binari di ricerca: descrizione ed operazioni

Un albero binario è una struttura di dati ad albero composta da *nodi*, ognuno dei quali ha al massimo due figli, chiamati nodo destro e nodo sinistro. L'albero inizia con un singolo nodo, noto come *radice*.

Matematicamente:

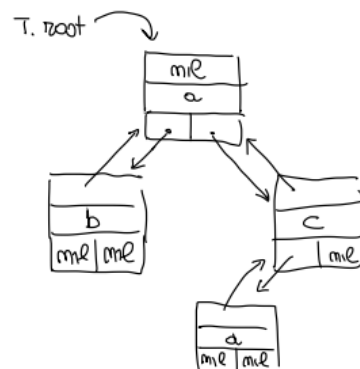
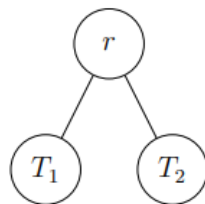
- nodi x ed elemento/valore/chiave x . *key*
- operazioni hanno costo $O(h)$ quando non bilanciato, $O(\log(n))$ se bilanciato

Noi esamineremo:

- i binary search trees/alberi binari di ricerca/BSTs
- alberi Red-Black/RB-Trees

In merito agli alberi binari (definizione induttiva):

- $\emptyset$ è un albero;
- Se r è un nodo, T_1 e T_2 alberi $\Rightarrow r(T_1, T_2)$ è un albero.



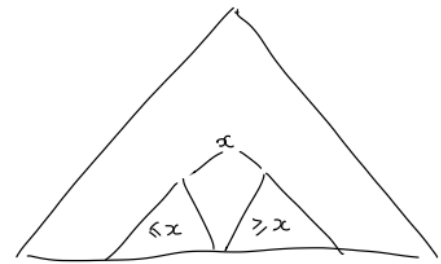
Ogni nodo x ha i seguenti campi:

- $x.p$
- $x.key$
- $x.left$
- $x.right$

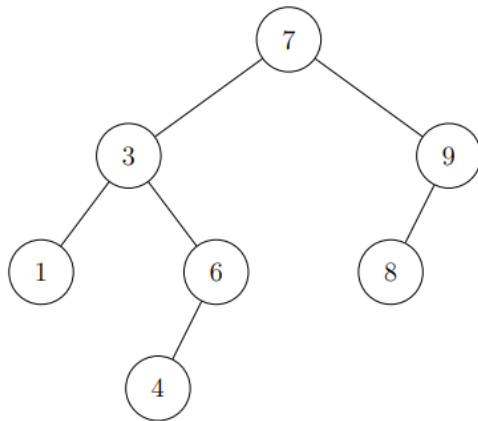
In merito invece agli alberi binari di ricerca

Proprietà $\forall r$

- Per ogni nodo y in T_1 $y.key \leq x.key$;
- Per ogni nodo y in T_2 $y.key \geq x.key$.



Esempio Ecco un **albero binario di ricerca** d'esempio:

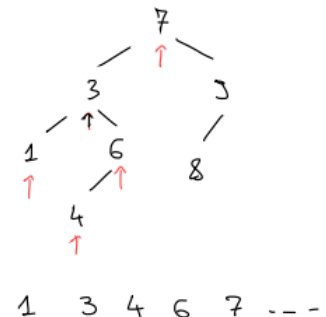
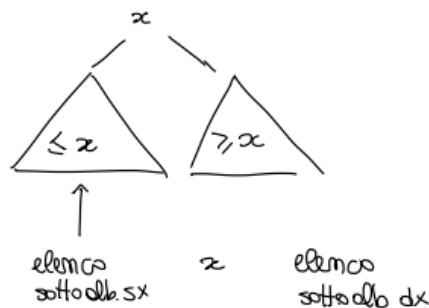


Listiamo le operazioni possibili:

- Visita simmetrica/*InOrder*: elencare gli elementi del sottoalbero radicato in un nodo x in ordine di chiave crescente

IN-ORDER(x)

- 1 if $x \neq \text{NIL}$
- 2 IN-ORDER($x.\text{left}$)
- 3 PRINT(x) // $\Theta(1)$
- 4 IN-ORDER($x.\text{right}$)



La complessità di tale operazione è lineare, dato da una visita di tutto l'albero.

Complessità

$$T(n) = \begin{cases} c & n = 0 \\ T(k) + T(n - k - 1) + d & n > 0, k < n \end{cases}$$

Stima di complessità: $T(n) = (c + d)n + c$.

Dimostrandolo per induzione:

$$(n = 0) \quad T(n) = c = (c + d) \cdot 0 + c$$

$(n \rightarrow n + 1) \quad T(n) = T(k) + T(n - k) + d$. Non basta l'induzione ordinaria, usiamo l'**induzione completa**.

$(n > 0)$ Proprietà vera per $n' < n$

$$\begin{aligned} T(n) &= T(k) + T(n - k - 1) + d \\ &\text{con } T(k) = (c + d)k + c \text{ e } T(n - k - 1) = (c + d)(c - k - 1) + c \\ &= (c + d)(k + n - k - 1) + 2c + d \\ &= n(c + d) - c - d + 2c + d \\ &= n(c + d) + c - d + d \\ &\cong \Theta(n) \end{aligned}$$

- Ricerca: data k chiave, cerca nel sottoalbero radicato nel nodo x un nodo con chiave k

In particolare:

- Se x è $nil \rightarrow$ ritorna nil
- Se $x.key = k \rightarrow$ ritorna x
- Se $k < x.key \rightarrow$ cerca in $x.left$
- Se $k > x.key \rightarrow$ cerca in $x.right$

SEARCH(x, k)

```

1  if ( $x = \text{NIL}$ ) or ( $x.key = k$ )
2      return  $x$ 
3  else if  $k < x.key$ 
4      return SEARCH( $x.left, k$ )
5  else
6      return SEARCH( $x.right, k$ )

```

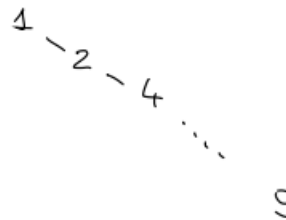
La complessità, nel caso peggiore, è la ricerca che continua fino ad una foglia ed il cammino radice-foglia è quello massimo $\rightarrow$ altezza h .

Complessità: $O(h)$

L'albero può avere, alla peggio, anche struttura simil-lineare.

Nota h può essere $\Theta(n)$

4, 2, 4, 6, 7, 8, 9



La procedura *Search* iterativa è così scritta:

Search(x, k)

```

while( $x \neq nil$ ) and ( $x.key \neq k$ )
    if  $k < x.key$ 
         $x = x.left$ 
    else
         $x = x.right$ 
return  $x$ 

```

Attenzione: se dobbiamo esaminare tutto l'albero, nel caso peggiore avremo una complessità $\Theta(n)$ sulla base di una relazione di ricorrenza $T(n) = c + T(k) + T(n - k - 1)$.

- Ricerca di *minimo* e *massimo* (rispettivamente, ricerca continuando ad andare verso sinistra oppure continuando ad andare verso destra). Per entrambi, la complessità è data dall'altezza dell'albero, in generale $O(h)$.

MIN(T)

```

1   $x = T.root$ 
2  if  $x = NIL$ 
3      return  $NIL$ 
4  else
5      while  $x.left \neq NIL$ 
6           $x = x.left$ 
7  return  $x$ 

```

MAX(T)

```

1   $x = T.root$ 
2  if  $x = NIL$ 
3      return  $NIL$ 
4  else
5      while  $x.right \neq NIL$ 
6           $x = x.right$ 
7  return  $x$ 

```

- Successore: Si intende il nodo elencato dopo un nodo x passato come parametro in una visita simmetrica. Dato x in *ABR*:
 - o minimo tra i nodi più grandi di x
 - o nodo che segue x in una visita *InOrder*
 - o se x ha sottoalbero dx non vuoto $\rightarrow$ il successore è $\min(x.right)$
 - o se x non ha sottoalbero dx $\rightarrow$ il successore è il più vicino antenato di x è nel sottoalbero sx

SUCCESSOR(x)

```

1  if  $x.right \neq NIL$ 
2      return MIN( $x.right$ )
3  else
4       $y = x.p$ 
5      while ( $y \neq NIL$ ) and ( $x = y.right$ )
6           $x = y$ 
7           $y = y.p$ 
8  return  $y$ 

```

Complessità $O(h)$

- Inserimento. La funzione *Insert* viene utilizzata per aggiungere un nuovo elemento in un albero di ricerca binario in una posizione appropriata. La funzione *Insert* deve essere progettata in modo tale da non violare la proprietà dell'albero di ricerca binario a ogni valore.
 - o Allocare la memoria per l'albero.
 - o Impostare la parte dei dati sul valore e impostare il puntatore sinistro e destro dell'albero su NULL.
 - o Se l'elemento da inserire sarà il primo elemento dell'albero, i puntatori sinistro e destro di questo nodo punteranno a NULL.
 - o In caso contrario, controlla se l'elemento è inferiore all'elemento radice dell'albero; se è vero, esegue ricorsivamente l'operazione con la sinistra della radice.
 - o Se è falso, allora si esegue questa operazione ricorsivamente con il sottoalbero destro della radice.

INSERT(T, z)

```

1   $x = T.root$ 
2   $y = NIL$ 
3  while  $x \neq NIL$ 
4       $y = x$ 
5      if  $z.key < x.key$ 
6           $x = x.left$ 
7      else
8           $x = x.right$ 
9   $z.p = y$ 
10 if  $y = NIL$ 
11      $T.root = z$ 
12 else
13     if  $z.key < y.key$ 
14          $y.left = z$ 
15     else
16          $y.right = z$ 

```

Complessità $O(h)$

- Cancellazione. Qui distinguiamo due casi:

1) z ha al più un figlio

2) z ha due figli $\rightarrow$ Pericoloso: ci sta il rischio che si copino i dati di y in z .

Creiamo una funzione *Transplant*, che cerca di sostituire il nodo u e il suo corrispondente sottoalbero con un altro nodo v e il suo sottoalbero. Suppongo che p stia per il genitore di un nodo. Esso ha complessità costante $O(1)$ (letteralmente, sposta i nodi, per questo).

TRANSPLANT(T, u, v)

```

1  if  $u.p = \text{NIL}$ 
2     $T.\text{root} = v$ 
3  else
4    if  $u = u.p.\text{left}$ 
5       $u.p.\text{left} = v$ 
6    else
7       $u.p.\text{right} = v$ 
8  if  $v \neq \text{NIL}$ 
9     $v.p = u.p$ 

```

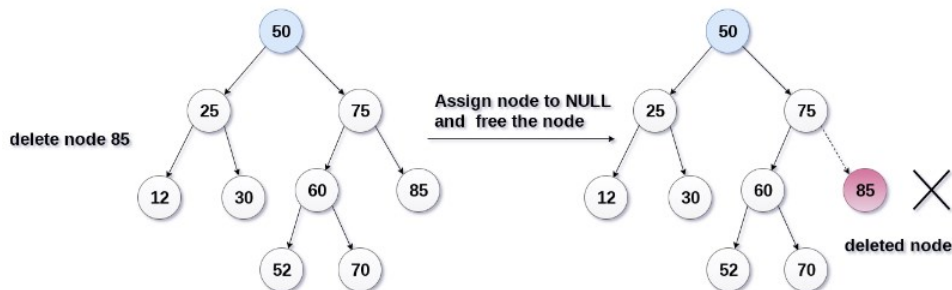
Inseriamo questa funzione in quanto utilizzata nella funzione di cancellazione dei BST *Delete*.

La funzione *Delete* viene utilizzata per eliminare il nodo specificato da un albero di ricerca binario. Tuttavia, è necessario eliminare un nodo da un albero di ricerca binario in modo tale da non violare le proprietà dell'albero di ricerca binario.

Esistono tre situazioni di eliminazione di un nodo dall'albero di ricerca binario.

- 1) Il nodo da eliminare è un nodo foglia

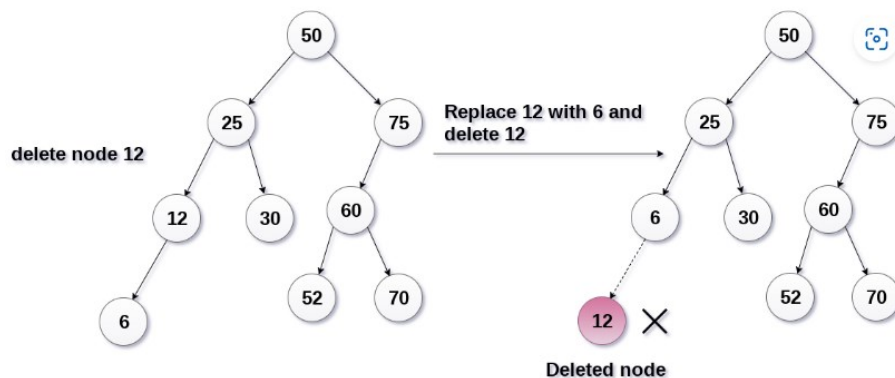
È il caso più semplice, in questo caso si sostituisce il nodo foglia con NULL e si libera semplicemente lo spazio allocato. Nell'immagine seguente, stiamo eliminando il nodo 85, poiché si tratta di un nodo foglia; quindi, il nodo verrà sostituito con NULL e lo spazio allocato verrà liberato.



- 2) Il nodo da eliminare ha un solo figlio.

In questo caso, si sostituisce il nodo con il suo nodo figlio e si elimina il nodo figlio, che ora contiene il valore da eliminare. È sufficiente sostituirlo con NULL e liberare lo spazio allocato.

Nell'immagine seguente, il nodo 12 deve essere eliminato. Ha solo un figlio. Il nodo verrà sostituito con il suo nodo figlio e il nodo 12 sostituito (che ora è un nodo foglia) verrà semplicemente eliminato.



3) Il nodo che deve essere cancellato ha due nodi figli

È un caso un po' più complesso rispetto agli altri due. Tuttavia, il nodo da eliminare viene sostituito con il suo successore o predecessore in ordine ricorsivo, finché il valore del nodo (da eliminare) non viene collocato sulla foglia dell'albero. Al termine della procedura, si sostituisce il nodo con NULL e si libera lo spazio allocato.

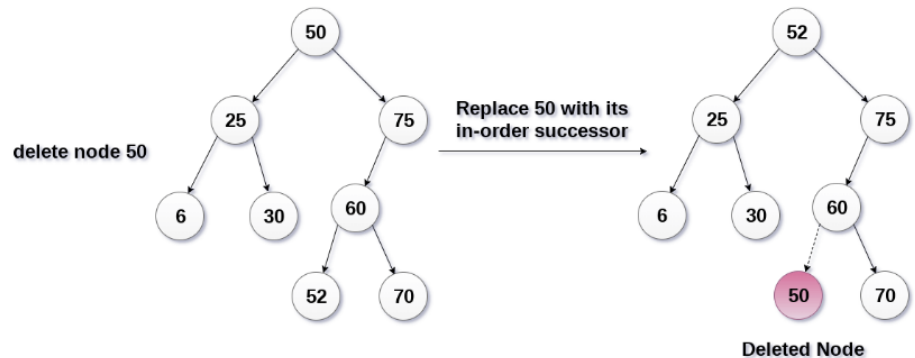
Nell'immagine seguente, il nodo da eliminare è il nodo 50, che è il nodo radice dell'albero.

L'attraversamento in ordine sparso dell'albero è riportato di seguito.

6, 25, 30, 50, 52, 60, 70, 75.

sostituire 50 con il suo
successore in ordine 52.

Ora 50 verrà spostato alla
foglia dell'albero, che verrà
semplicemente eliminata.



Fatte tutte queste premesse (meglio farle, per capire tutto), l'algoritmo è il seguente:

DELETE(T, z)

```

1  if  $z.left = \text{NIL}$ 
2      TRANSPLANT( $T, z, z.right$ )
3  else if  $z.right = \text{NIL}$ 
4      TRANSPLANT( $T, z, z.left$ )
5  else
6       $y = \text{MIN}(z.right)$ 
7      if  $y.p \neq z$ 
8          TRANSPLANT( $T, y, y.right$ )
9           $y.right = z.right$ 
10          $y.right.p = y$ 
11          $y.left = z.left$ 
12          $y.left.p = y$ 
13         TRANSPLANT( $T, z, y$ )

```

Attenzione: se dobbiamo esaminare tutto l'albero, nel caso peggiore avremo una complessità $\theta(n)$ sulla base di una relazione di ricorrenza $T(n) = c + T(k) + T(n - k - 1)$.

Complessità $O(h)$

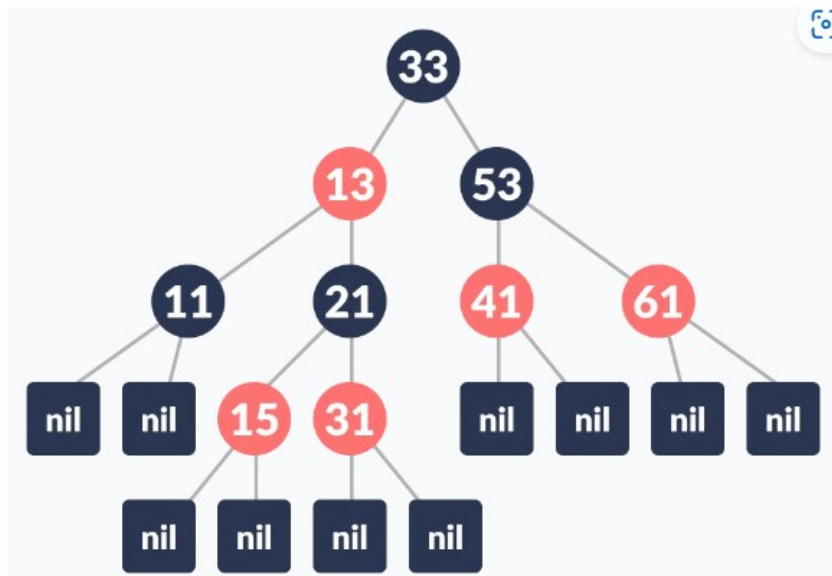
Lezione 11: Red-Black Trees

I Red-Black Trees sono ABR i cui nodi hanno un bit extra riservato ad un campo colore $x.color$, che può essere: *red* per il rosso, *black* per il nero.

Un albero rosso-nero soddisfa le seguenti proprietà:

- Proprietà Rosso/Nero: Ogni nodo è colorato, rosso o nero.
- Proprietà della radice: La radice è nera.
- Proprietà delle foglie: Ogni foglia (NIL) è nera.
- Proprietà del rosso: Se un nodo rosso ha dei figli, questi sono sempre neri.
- Proprietà Depth: Per ogni nodo, qualsiasi percorso semplice da questo nodo a qualsiasi foglia discendente ha la stessa profondità nera (il numero di nodi neri).

Qui un esempio di RB-Tree:



È possibile notare che:

- in caso non ci fossero nodi rossi, avremo un albero perfettamente bilanciato;
- in ogni cammino, il numero (#) di nodi neri è almeno la metà del numero (#) dei nodi rossi

Osservazione Se T è un **RB-Tree** con n nodi interni ($\neq \text{NIL}$) e h altezza, allora vale

$$h \leq 2 \log(n + 1)$$

La dimostrazione è per induzione su h_x (altezza del sotto-albero radicato in x).

($h_x = 1$) Allora ho solo $T.\text{nil} \Rightarrow n_x = 0 = 2^0 - 1$ (2^0 con $0 = bh(x)$)

($h_x > 1$) Consideriamo x radice. x ha due figli, x_1 e x_2 .

Sicuramente vale $h_1, h_2 < h$. Per ipotesi induttiva, valgono:

$$n_{x_1} \geq 2^{bh(x_1)} - 1$$

$$n_{x_2} \geq 2^{bh(x_2)} - 1$$

$$\begin{aligned} n_x &= n_{x_1} + n_{x_2} + 1 \\ &\geq 2^{bh(x_1)} + 2^{bh(x_2)} - 1 \\ &\geq 2 \cdot 2^{bh(x)-1} - 1 = 2^{bh(x)} - 1 \\ &\quad (\text{valgono } bh(x_1) \geq bh(x) - 1, \text{ } bh(x_2) \geq bh(x) - 1) \end{aligned}$$

Complessivamente

$$n = n_{\text{root}} \geq 2^{bh(\text{root})} - 1$$

Essendo $bh(\text{root}) \geq \frac{h}{2}$, posso ottenere

$$\begin{aligned} n_{\text{root}} &\geq 2^{bh(\text{root})} - 1 \\ &\geq 2^{\frac{h}{2}} - 1 \\ \Rightarrow 2^{\frac{h}{2}} &\leq n + 1 \\ \frac{h}{2} &\leq \log_2(n + 1) \Rightarrow h \leq 2 \log_2(n + 1) \end{aligned}$$

Le operazioni degli RB-Trees saranno chiaramente simili a prima, in particolare:

- *Search*
- *Max*
- *Min*
- *Succ* (trovare il successore)
- *Prec* (trovare il predecessore → Se X ha due figli, il suo predecessore è il valore massimo nel suo sottoalbero sinistro e il suo successore il valore minimo nel suo sottoalbero destro. Se non ha un figlio a sinistra, il predecessore di un nodo è il suo primo antenato a sinistra.)
- *Insert* (difficile mantenere la colorazione)
- *Delete* (difficile mantenere la colorazione)

La complessità delle operazioni è sempre data dall'altezza h dell'albero, come $h = O(\log(n))$ con $h \leq 2\log_2(n + 1)$.

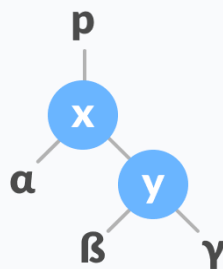
Per quanto descritto per l'inserimento e la cancellazione, a cui ci riferiamo come *RB – Insert* e *RB – Delete*, occorre parlare di un'operazione ulteriore che deve essere fatta (come prima per *Transplant*), in questo caso le rotazioni.

Nell'operazione di rotazione, le posizioni dei nodi di un sottoalbero vengono scambiate.

Esistono due tipi di rotazioni:

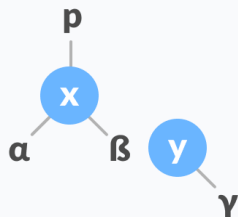
1) Rotazione a sinistra

Nella rotazione a sinistra, la disposizione dei nodi a destra viene trasformata in quella dei nodi a sinistra.



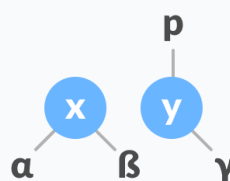
1. Sia l'albero iniziale:

2. Se y ha sottoalbero sinistro, assegnare x come padre del sottoalbero sinistro di y .

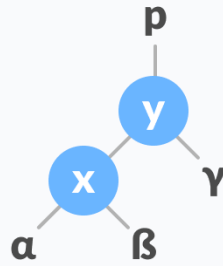


3. Se il padre di x è NULL, impostare y come radice dell'albero.

4. Altrimenti se x è il figlio sinistro di p , fai y come figlio sinistro di p .



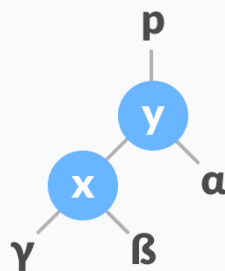
5. Altrimenti assegna y come figlio giusto di p .



6. Rendi y come genitore di x .

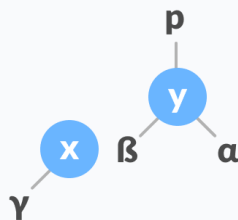
2) Rotazione a destra

Nella rotazione a destra, la disposizione dei nodi a sinistra viene trasformata in quella del nodo a destra.



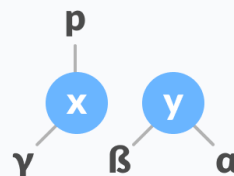
1. Sia l'albero iniziale:

2. Se x ha un sottoalbero destro, assegnare y come padre del sottoalbero destro di x .

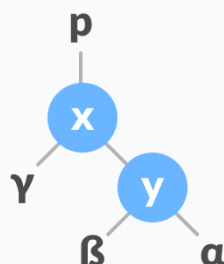


3. Se il padre di y è `NULL`, impostare x come radice dell'albero.

4. Altrimenti se y è il figlio giusto del suo genitore p , fai x come figlio giusto di p .



5. Altrimenti assegna x come figlio sinistro di p .



6. Crea x come genitore di y .

Si può osservare che il risultato di una visita *InOrder* è immutato per entrambe le procedure.

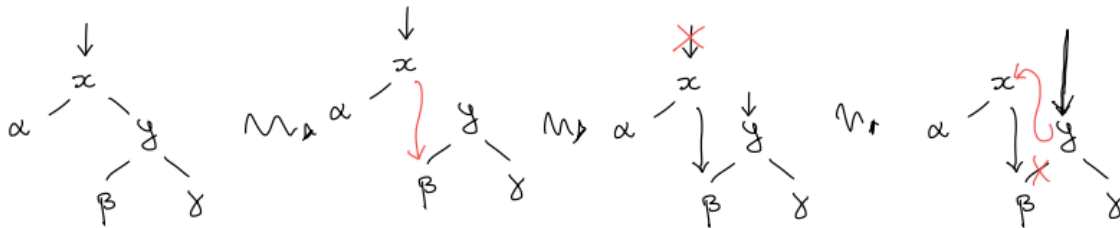
Ora, portiamo le pseudocodice della procedura di rotazione a sinistra *Left(T, x)*.

Left(T, x)

```

y = x.right
x.right = y.left
x.right.p = x
Transplant(T, x, y)
y.left = x
x.p = y

```



Per completezza, inseriamo anche la procedura *Right(T, y)*

Right(T, y)

```

x = y.left
y.left = x.right
if x.right != T.nil
    x.right.p = y
x.p = y.p
if y.p == T.nil
    T.root = x
else if y == y.p.right
    y.p.right = x
else y.p.left = x
    x.right = y
y.p = x

```

Ora, finalmente, l'inserimento *Insert*. Vogliamo inserire *z* nell'albero *T*. L'idea è quella di porre *z.color = RED* (migliore, in quanto andando a modificare il numero di nodi neri, cambia l'altezza nera, e la cosa è difficile da sistemare.).

- Se violo la proprietà (2, radice deve essere nera) $\rightarrow z.color = BLACK$
- Se violo la proprietà (4, se un nodo rosso ha dei figli, questi sono sempre neri)
 - o Risolvo localmente
 - o Sposto verso l'alto il problema

La procedura *RB – Insert* è così formata:

RB-INSERT(*T, z*)

1 **INSERT(*T, z*)**

2 ***z.color = RED***

3 **RB-INSERT-FIXUP(*T, z*)**

Da notare che:

- I nodi non cerchiati sono *red*

- I nodi cerchiati sono *black*

- I nodi tratteggiati e puntinati possono essere sia *red* che *black*

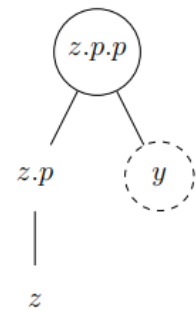
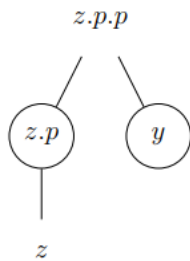
Abbiamo dei problemi che rompono le scatole rispetto alle proprietà degli RB:

- *z.color = RED*
- *z.p.color = RED*

Analizziamo il macrocaso $z.p$ è figlio sinistro (se destro, l'analisi è analoga e non affrontata nel corso).

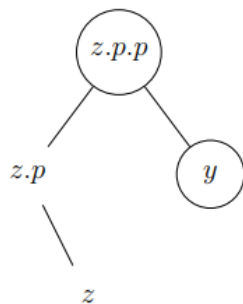
Abbiamo due possibilità per y :

- 1) $y.color = RED$. Inverto il colore di $z.p.p$ con quello dei figli, ottenendo:
In questo modo, risolviamo localmente e rimandiamo il problema verso l'alto.

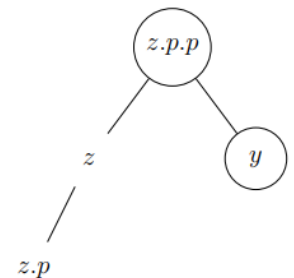


- 2) $y.color = BLACK$. Possiamo distinguere due sottocasi:

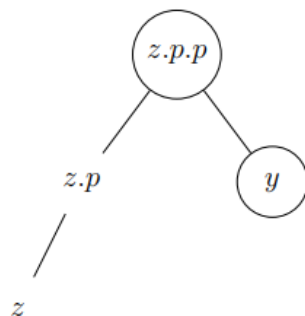
(2.1) z figlio destro.



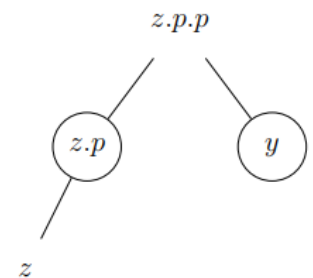
Applico Left($T, z.p$), ottenendo:



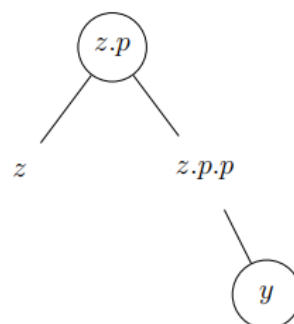
(2.2) z figlio sinistro.



Scambio i colori di $z.p.p$ con $z.p$, ottenendo:



Applico Right($T, z.p.p$)¹, ottenendo:



```

RB-INSERT-FIXUP( $T, z$ )
1  while  $z.p.color = RED$ 
2      if  $z.p = z.p.p.left$  // Macrocaso  $z.p$  figlio sinistro
3           $y = z.p.p.right$ 
4          if  $y.color = RED$  // Caso 1
5               $z.p.p.color = RED$ 
6               $z.p.color = BLACK$ 
7               $y.color = BLACK$ 
8               $z = z.p.p$ 
9          else // Caso 2
10             if  $z = z.p.right$  // Caso (2.1)
11                 LEFT( $T, z.p$ )
12                  $z = z.left$ 
13             // Caso (2.2)
14              $z.p.color = BLACK$ 
15              $z.p.p.color = RED$ 
16             RIGHT( $T, z.p.p$ )
17         else
18             ... // Macrocaso  $z.p$  figlio destro
19      $T.root.color = BLACK$ 

```

Complessità $O(\log n) + \max 2$ rotazioni

(Attenzione: gli RB-Tree si ricordino solo rapidamente per ragionamento. Sono complicati e in generale da noi ben poco trattati per fortuna; presenti per programma e per completezza totale, con approfondimenti come si vede).

Quale proprietà rosso-nera può essere violata quando si chiama *RB – INSERT – FIXUP*?

Proprietà 1: Certamente continua a valere.

Proprietà 2: Sarà violata quando z è la radice e z è rosso.

Proprietà 3: Certamente continua a valere.

Proprietà 4: Sarà violata quando sia z che $p[z]$ sono rossi.

Proprietà 5: Certamente continua a valere, perché coloriamo z di rosso.

Se c'è una violazione delle proprietà dell'albero rosso-nero, ce n'è al massimo una, ed è la proprietà 2 o 4.

I 3 casi seguenti violeranno le proprietà dell'albero rosso-nero e i passaggi successivi mostrano come *RB – INSERTFIXUP* funziona per ricorrere alle proprietà rosso-nera. In realtà, ci sono 6 casi in tutto, ma altri 3 e i successivi 3 sono simmetrici.

casi sono simmetrici.

- Caso 1: lo zio (uncle node, nodi allo stesso livello di profondità del genitore del nodo attuale) y di z è rosso.
 1. Colorare $p[z]$ e y di nero.
 2. Colorare $p[p[z]]$ di rosso.
 3. Spostare il puntatore z di due livelli su $p[p[z]]$.

Se il nuovo z è rosso, il ciclo while si ripete.
- Caso 2: lo zio di z , y , è nero e z è un figlio destro.
 1. Eseguire la rotazione a sinistra su z .
 2. Passare al caso 3.
- Caso 3: lo zio di z , y , è nero e z è un figlio di sinistra.
 1. Colorare $p[z]$ di nero.
 2. Colorare $p[p[z]]$ di rosso.
 3. Eseguire la rotazione a destra su $p[p[z]]$.

Per la cancellazione $RB - Delete(T, z)$ distinguiamo 2 casi:

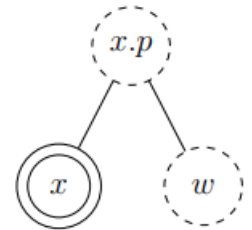
- 1) z ha un figlio
- 2) z ha due figli.

Ci comportiamo allo stesso modo della $Delete(T, z)$ per un ABR , facendo però un ulteriore accorgimento:

- se $z.color = RED$ non ho problemi
- se $z.color = BLACK$
 - o risolvo localmente
 - o sposto verso l'alto il problema.

Dunque, in seguito alla $Delete(T, z)$, il nodo x che ha preso il posto di z ne "assorbirà" il colore diventando doppiamente $BLACK$. Abbiamo due macrocasi:

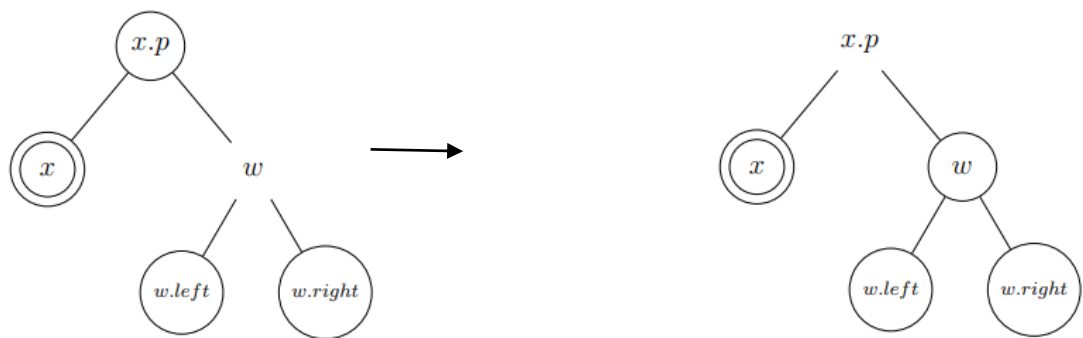
- 1) x è figlio sinistro (noi analizziamo solo questo, *immagine*)
- 2) x è figlio destro



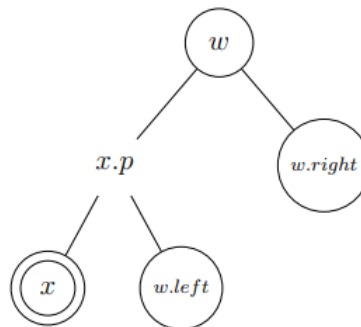
Abbiamo due possibilità per w :

1. $w.color = RED$

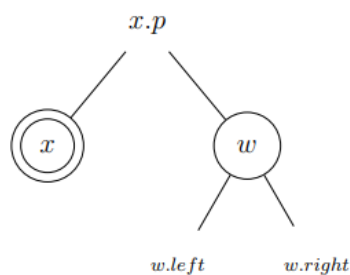
Scambio i colori di w con $x.p$, ottenendo:



Applico $Left(T, x.p)$, ottenendo:

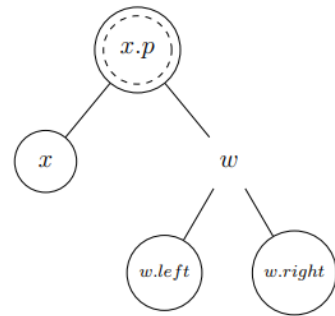
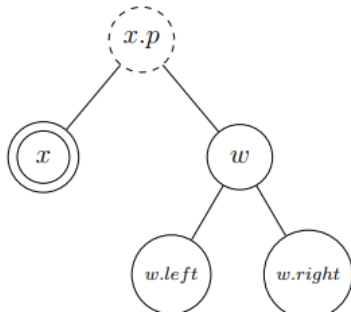


2. $w.color = BLACK$



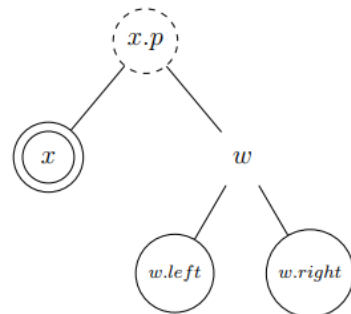
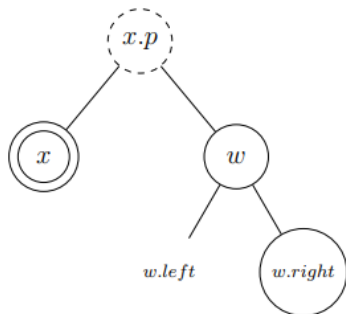
Possiamo distinguere tre sottocasi:

(2.1) $w.left.color = \text{BLACK}$ e $w.right.color = \text{BLACK}$



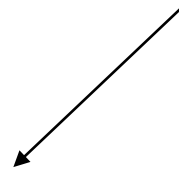
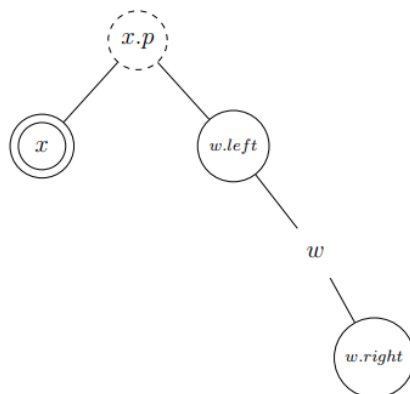
In questo modo, risolviamo localmente e rimandiamo il problema in alto.

(2.2) $w.right.color = \text{BLACK}$

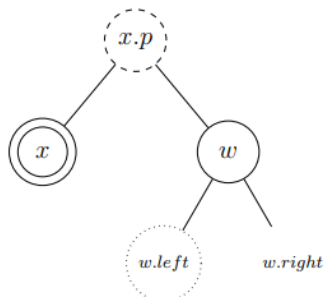


Scambio i colori di w con $w.left$, ottenendo:

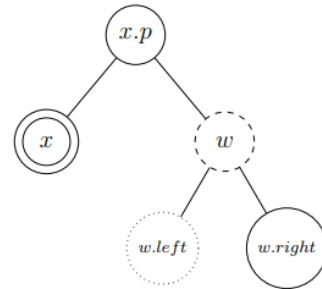
Applico $\text{Right}(T, w)$, ottenendo:



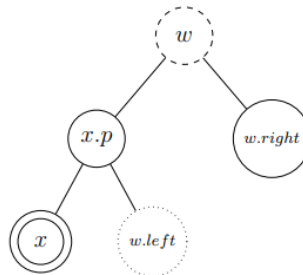
(2.3) $w.right.color = RED$



Scambio i colori di $x.p$ con w e $w.right$ diventa BLACK, ottenendo:



Applico $Left(T, x.p)$, ottenendo:



Ho risolto.

Complessità $O(\log n) + \max 3$ rotazioni

Fatte tutte queste premesse, allora, la funzione di cancellazione (chiamata e relativa chiamata) è:

```

RB-DELETE (T, z)
1 if left[z] = nil[T] or right[z] = nil[T]
2   then y ← z
3   else y ← TREE-SUCCESSOR(z)
4 if left[y] ≠ nil[T]
5   then x ← left[y]
6   else x ← right[y]
7 p[x] ← p[y]
8 if p[y] = nil[T]
9   then root[T] ← x
10  else if y = left[p[y]]
11    then left[p[y]] ← x
12    else right[p[y]] ← x
13 if y ≠ z
14   then key[z] ← key[y]
15   ▹ If y has other fields, copy them, too.
16 if color[y] = BLACK
17   then RB-DELETE-FIXUP (T, x)
18 return y
    
```

```

RB-DELETE(T, z)
1 DELETE(T, z) // O(log n)
2 RB-DELETE-FIXUP(T, z) // O(log n)
    
```

```

RB-DELETE-FIXUP( $T, x$ )
1  while  $x \neq \text{root}[T]$  and  $\text{color}[x] = \text{BLACK}$ 
2      do if  $x = \text{left}[p[x]]$ 
3          then  $w \leftarrow \text{right}[p[x]]$ 
4              if  $\text{color}[w] = \text{RED}$ 
5                  then  $\text{color}[w] \leftarrow \text{BLACK}$            ▷ Case 1
6                       $\text{color}[p[x]] \leftarrow \text{RED}$          ▷ Case 1
7                      LEFT-ROTATE( $T, p[x]$ )             ▷ Case 1
8                       $w \leftarrow \text{right}[p[x]]$          ▷ Case 1
9              if  $\text{color}[\text{left}[w]] = \text{BLACK}$  and  $\text{color}[\text{right}[w]] = \text{BLACK}$ 
10                 then  $\text{color}[w] \leftarrow \text{RED}$            ▷ Case 2
11                      $x \leftarrow p[x]$                  ▷ Case 2
12                 else if  $\text{color}[\text{right}[w]] = \text{BLACK}$ 
13                     then  $\text{color}[\text{left}[w]] \leftarrow \text{BLACK}$    ▷ Case 3
14                          $\text{color}[w] \leftarrow \text{RED}$          ▷ Case 3
15                         RIGHT-ROTATE( $T, w$ )             ▷ Case 3
16                          $w \leftarrow \text{right}[p[x]]$          ▷ Case 3
17                      $\text{color}[w] \leftarrow \text{color}[p[x]]$        ▷ Case 4
18                      $\text{color}[p[x]] \leftarrow \text{BLACK}$          ▷ Case 4
19                      $\text{color}[\text{right}[w]] \leftarrow \text{BLACK}$    ▷ Case 4
20                     LEFT-ROTATE( $T, p[x]$ )             ▷ Case 4
21                      $x \leftarrow \text{root}[T]$                ▷ Case 4
22                 else (same as then clause
                        with "right" and "left" exchanged)
23   $\text{color}[x] \leftarrow \text{BLACK}$ 

```

Lezione 12: Arricchimento di strutture dati e conclusioni su RB-Trees

Vedremo due esempi, uno per gli *RB – Trees*, e un altro per gli *ABR*:

- *Statistiche d'ordine*, struttura che parte da un *RB – Tree*
- *Interval Trees*, struttura che parte da un *ABR*

Statistiche d'ordine: Alberi binari di ricerca con due operazioni condizionali.

- $\text{Select}(T, i) = \text{nodo } x \text{ che occuperebbe la posizione } i \text{ nei nodi ordinati per chiave (in una visita InOrder);}$
- $\text{Rank}(T, x) = \text{posizione } i \text{ (in una visita simmetrica) che occupa il nodo } x.$

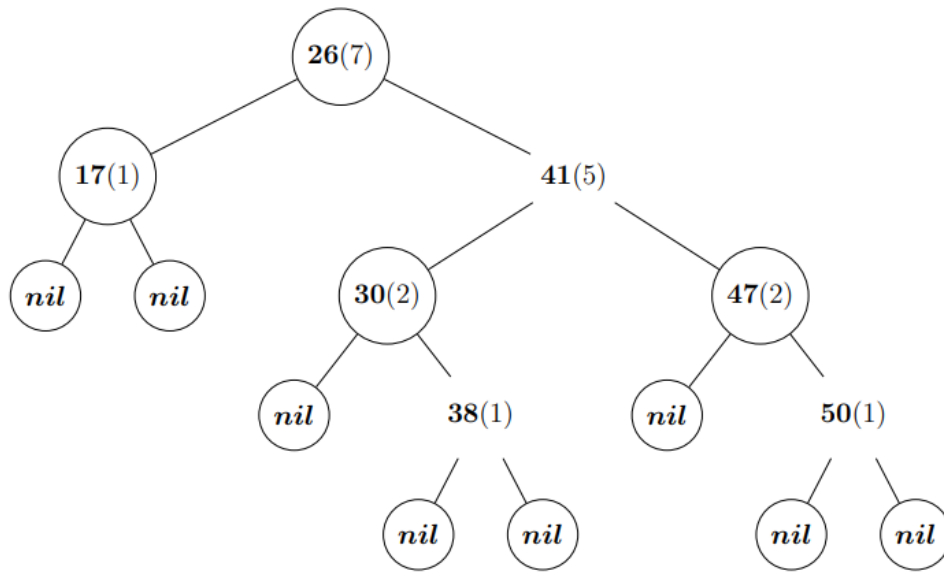
Per realizzarle entrambe, prendiamo tutti i nodi x di un albero RB ma aggiungendo un campo $x.size$ (che conta il numero di nodi interni in T_x).

Due premesse:

- $T.nil.size = 0$
- $x.size = x.left.size + x.right.size + 1$

Esempio

In ogni nodo, tra le parentesi è riportato la *size* di quel nodo.



Letteralmente, la procedura *Size* percorre l'albero, con complessità $O(n)$, quindi in generale non efficiente.

Questo perché le procedure *Insert/Delete* di un *RB-Tree* sono nel peggiore dei casi $O(h)$.

Questa procedura, *Select*, restituisce il nodo di posizione *i*-esimo nel sottoalbero T_x in una visita simmetrica.

SIZE(*x*)

```

1  if x = T.nil
2      x.size = 0
3  else
4      l = SIZE(x.left)
5      r = SIZE(x.right)
6      x.size = l + r + 1
7  return x.size
    
```

SELECT(*x*, *i*)

// Pre: $x \neq T.nil$, $i : 1 \leq i \leq x.size$

```

1  r = x.left.size + 1
2  if i = r
3      return x
4  else if i < r
5      return SELECT(x.left, i)
6  else
7      return SELECT(x.right, i - r)
    
```

Complessità $O(h) = O(\log n)$

Rank restituisce la posizione *i*-esima che occupa il nodo *x*.

RANK(*x*)

```

1  r = x.left.size + 1
2  y = x
3  while y ≠ T.root // idea: r contiene la posizione di x in  $T_y$ 
4      if y.p.right = y
5          r = r + y.p.left.size + 1
6      y = y.p
7  return r
    
```

Complessità $O(h) = O(\log n)$

Per mantenere aggiornato il campo *Size*, occorre aggiornare la procedura di inserimento *Insert* (per completezza, rimetto anche, a puro scopo visivo *RB – Insert – Fixup*)

RB-INSERT(*T*, *z*)

```
// (1) versione aggiornata di Insert
1  z.size = 1 // modifica 1
2  x = T.root
3  y = T.nil
4  while x ≠ T.nil
5      x.size = x.size + 1 // modifica 2
6      y = x
7      if z.key < x.key
8          x = x.left
9      else
10         x = x.right
11  z.p = y
12  if y = NIL
13      T.root = z
14  else if z.key < y.key
15      y.left = z
16  else
17      y.right = z
// (2) RB-Insert-Fixup
18  z.color = RED      z.size = 1;
19  RB-INSERT-FIXUP(T, z)
```

RB-INSERT-FIXUP(*T*, *z*)

```
1  while z.p.color = RED
2      if z.p = z.p.p.left // Macrocaso z.p figlio sinistro
3          y = z.p.p.right
4          if y.color = RED // Caso 1
5              z.p.p.color = RED
6              z.p.color = BLACK
7              y.color = BLACK
8              z = z.p.p
9          else // Caso 2
10             if z = z.p.right // Caso (2.1)
11                 LEFT(T, z.p)
12                 z = z.left
13             // Caso (2.2)
14             z.p.color = BLACK
15             z.p.p.color = RED
16             RIGHT(T, z.p.p)
17         else
18             ... // Macrocaso z.p figlio destro
19  T.root.color = BLACK
```

Complessità $O(\log n) + \max 2$ rotazioni

Similmente, si aggiungono modifiche alle procedure di rotazione (qui, per gli esempi trattati, usiamo *Left*), ma anche per la cancellazione.

LEFT(*T*, *x*)

```
1  x.right = y.left
2  x.right.p = x
3  y.left = x
4  x.p = y
5  TRANSPLANT(T, x, y)
6  y.size = x.size // modifica 1
7  x.size = x.left.size + x.right.size + 1 // modifica 2
```

Teorema dell'aumento degli alberi RB.

Se modifichiamo i nodi degli alberi RB aggiungendo un nuovo campo *x.field* tale che si possa calcolare in tempo $O(1)$ da *x*, *x.left*, *x.right* Allora *RB – Insert* ed *RB – Delete* possono essere modificate tali da mantenere aggiornato il campo *x.field* con complessità $O(\log(n))$.

DELETE(*T*, *z*)

```
// Identica a quella per gli RB-Trees
:
1  w = y.p
2  while w ≠ NIL
3      w.size = w.size - 1
4      w = w.p
```

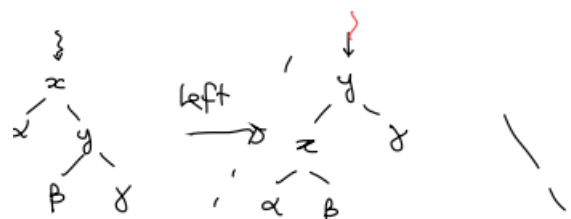
Versione aggiornata di RB-Delete:

RB-DELETE(*T*, *z*)

```
1  DELETE(T, z) //  $O(\log n)$ 
2  RB-DELETE-FIXUP(T, z) //  $O(\log n)$ 
```

Ex

$$x.size = x.left.size + x.right.size + 1$$



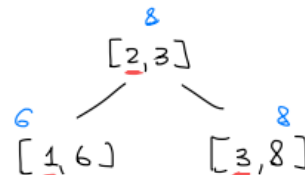
Per gli Interval Trees, L'idea è di aumentare un albero di ricerca binaria (BST) autobilanciato per mantenere un insieme di intervalli in modo che tutte le operazioni possano essere eseguite in tempo $O(\log(n))$

Ogni nodo dell'albero degli intervalli memorizza le seguenti informazioni, tali da rappresentare un intervallo.

- Un campo $x.int$
- Un campo $int.low$, che è la chiave
- Un campo $int.high$
- Un campo $x.max = \max$ estremo di intervallo per i nodi T_x , ossia

$$x.max = \max \begin{cases} x.left.max \\ x.right.max \\ x.int.high \end{cases}$$

$x.max$ = massimo $y.int.high$ in nodi di T_x

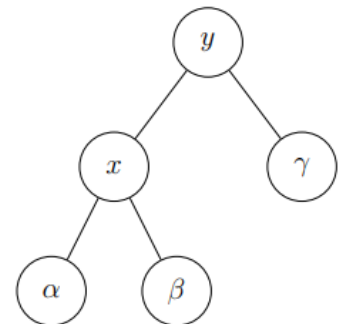
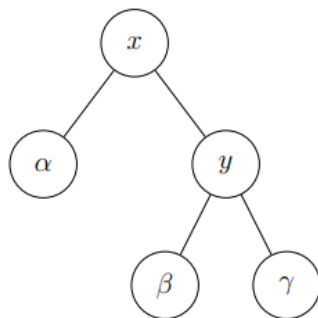


Consideriamo inoltre che, quando da un albero applicato la rotazione a sinistra, si sistemano i massimi (immagine).

- $T.nil.max = 0$
- $x.max = \max \{x.left.max, x.right.max, x.int.high\} \rightarrow \text{Insert/Delete } O(\log(n))$

Rotazioni Prendiamo il seguente albero di esempio.

Applico $\text{Left}(T, x)$, ottenendo



Sistemo i massimi. Left costa ancora $O(1)$

Vediamo quindi la funzione di ricerca definita come ISearch .

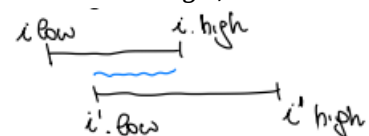
$\text{ISearch}(x, i)$

```

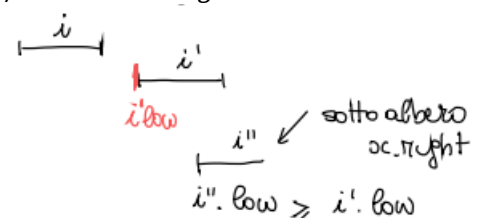
1  if  $(x = T.nil)$  or  $(x.int \cap i \neq \emptyset)$ 
2    return  $x$ 
3  else if  $(x.left \neq T.nil)$  and  $(x.left.max \geq i.low)$ 
4    return  $\text{ISearch}(x.left, i)$ 
5  else
6    return  $\text{ISearch}(x.right, i)$ 
    
```

2 Possibilità:

1) $i'.low \leq i.high, i \cap i' \neq \emptyset$



2) $i'.low > i.high \Rightarrow i'' \cap i = \emptyset$



Correttezza

◦ **Else if.** Consideriamo in $x.left$ un intervallo i' . Abbiamo 2 possibilità.

(1) $i \cap i' \neq \emptyset$

(2) $i \cap i' = \emptyset$, ovvero vale $i.high < i'.low$. Questo varrà per ogni nodo dei sotto-alberi, quindi è inutile ispezionare gli antenati di quel sotto-albero.

◦ **Else.** $\forall i' \text{ in } x.left \Rightarrow i' \cap i \neq \emptyset$.

Complessità $O(h) = O(\log n)$

Inizio Parte Scquizzato – Lezione 13: Programmazione dinamica

Il metodo divide et impera, anche definito come *Divide and Conquer*, non ha memoria; ogni volta che si calcola una soluzione, il processo richiede di ricalcolare la sottoistanza del problema che lo ha risolto, generando quindi un problema di sottoistanze ripetute.

Esempio: sequenza di Fibonacci: 1, 1, 2, 3, 5, 8, 13, ...

$$F(n) = \begin{cases} 1 & \text{se } n = 0, 1 \\ F(n-1) + F(n-2) & \text{se } n \geq 2 \end{cases}$$

L'algoritmo che la risolve è un classico algoritmo ricorsivo D&C/Divide and Conquer:

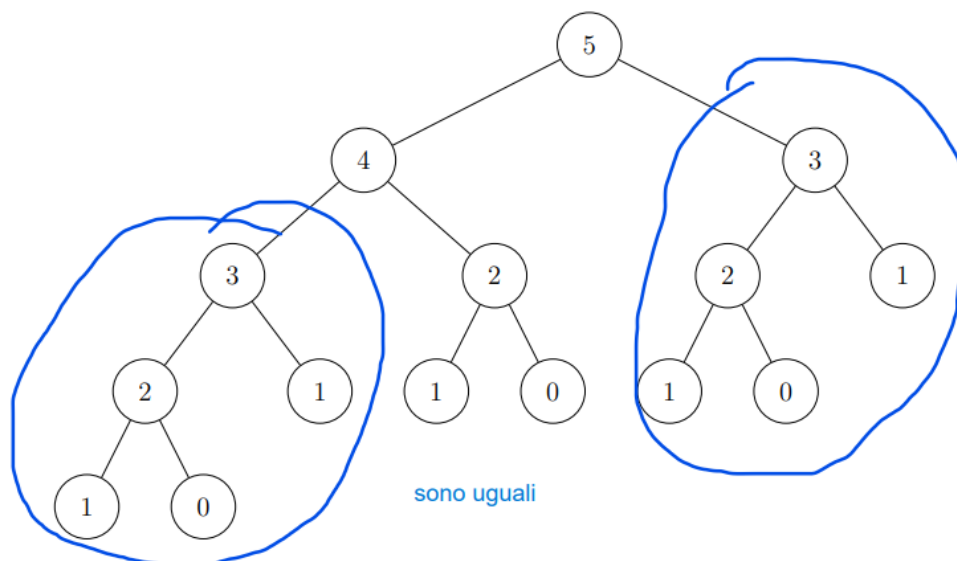
REC-FIB(n)

```

1  if ( $n = 0$ ) or ( $n = 1$ )
2      return 1
3  return REC-FIB( $n - 1$ ) + REC-FIB( $n - 2$ )

```

L'albero delle chiamate è il seguente (per $n = 5$):



Vengono ricalcolate $F(3)$ e $F(2)$.

Complessità

$$T(n) = \begin{cases} 0 & \text{se } n = 0, 1 \quad (\text{il "return" costa 0}) \\ T(n-1) + T(n-2) + 1 & \text{se } n \geq 2 \quad (\text{il "+" costa 1}) \end{cases}$$

Difficile da risolvere esattamente, ma a noi basta dire che $T_F(n) \geq f(n)$

L'*unfolding* (inteso come "spiegare" l'algoritmo) è come segue:

$$\begin{aligned} - \text{ Se } n \geq 2, \quad & T_F(n) = T_F(n-1) + T_F(n-2) + 1 \geq 2T_F(n-2) + 1 \\ & \geq 2(2T_F(n-2-2) + 1) + 1 = 2^2T_F(n-2-2) + 2 + 1 \geq \dots \\ & \geq \dots \\ & \geq 2^i T_F(n-2i) + \sum_{j=0}^{i-1} 2^j \end{aligned}$$

$$i_0 \rightarrow i = \left\lfloor \frac{n}{2} \right\rfloor$$

$$\text{se } n \text{ è pari: } 2^{\frac{n}{2}} T(n - 2 \frac{n}{2}) = 2^{\frac{n}{2}} T(0)$$

$$\text{se } n \text{ è dispari: } \left\lfloor \frac{n}{2} \right\rfloor = \frac{n-1}{2}$$

$$2^{\frac{n-1}{2}} T(n - 2 \frac{n-1}{2}) = 2^{\frac{n-1}{2}} T(1)$$

$\Rightarrow T_f = \Omega(2^{(n/2)}) \rightarrow$ Significa che il numero di sottoistanze ripetute è elevatissimo (addirittura esponenziale).

In verità,

$$T(n) = \Theta\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right)$$

Vediamo ora una versione iterativa:

IT-FIB(n)

```

1  if ( $n = 0$ ) or ( $n = 1$ )
2      return 1
3   $F[0] = 1$ 
4   $F[1] = 1$ 
5  for  $i = 2$  to  $n$ 
6       $F[i] = F[i-1] + F[i-2]$ 
7  return  $F[n]$ 
```

Consideriamo che F è la tabella (anzi, array) di n elementi. Questo algoritmo, presentato iterativamente, evita di ricalcolare il problema già risolto, ottenendo una complessità $O(n)$. Infatti, memorizza in una struttura dati ausiliaria tutte le soluzioni, evitando quindi di ricalcolarle.

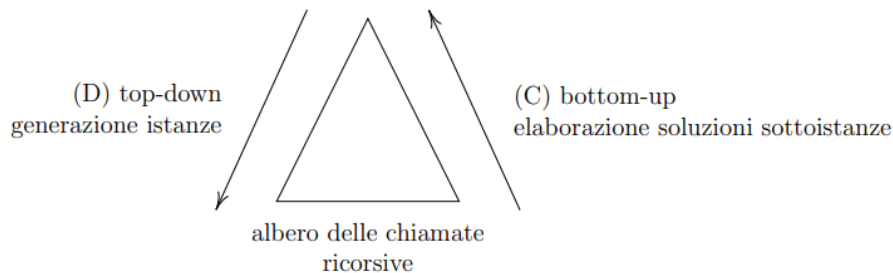
Il Divide and Conquer, infatti possiede due fasi:

- *Top-down*, con cui vengono generate le istanze risolte divide il problema originale in un numero di sottoproblema (*divide*), risolvendo i problemi ricorsivamente (*conquer*) e combinando le soluzioni dei sottoproblemi nella soluzione ai sottoproblemi originali (*combine*).
Come descritto, questo approccio viene adottato dalla programmazione Divide and Conquer.
- *Bottom-up*, normalmente *non ricorsiva* ma *iterativa* come appena visto per cui si caratterizza la struttura delle soluzioni ottimali, si definiscono ricorsivamente i valori delle soluzioni ottimali, si calcola il valore delle soluzioni ottimali e si costruisce una soluzione ottimale a partire dalle informazioni calcolate. Sinteticamente, si elaborano le soluzioni alle istanze presenti.

Ora:

- l'approccio della programmazione dinamica salta completamente l'approccio top-down, normalmente usando algoritmi iterativi per evitare di dover calcolare la soluzione
- la programmazione D&C usa entrambi gli approcci e normalmente in modo ricorsivo

81



Lezione 14 – Memoizzazione e problemi di ottimizzazione

È possibile mantenere i vantaggi del top-down e quindi anche del D&C senza incorrere nel problema del ricalcolo delle soluzioni? Normalmente si usa proprio il concetto di *memoizzazione*.

Definizione: Un algoritmo memoizzato è costituito da 2 subroutine/procedure:

- 1) *routine di inizializzazione*: risolve direttamente i casi base e inizializza una struttura dati “tabella” che contiene le soluzioni ai casi base ed elementi per tutte le sottoistanze da calcolare inizializzate ad un valore di default. Infine, invoca la struttura ricorsiva.
- 2) *routine ricorsiva*: segue il codice del D&C preceduto da un test sulla tabella per verificare se la soluzione è già stata calcolata e memorizzata. Se sì, si ritorna, altrimenti la si calcola ricorsivamente e la si memorizza nella struttura.

Riprendiamo l'esempio di prima su Fibonacci:

INIT-FIB(n)

```

1  if ( $n = 0$ ) or ( $n = 1$ )
2      return 1
3   $F[0] = 1$ 
4   $F[1] = 1$ 
5  for  $i = 2$  to  $n$ 
6       $F[i] = 0$ 
7  return REC-FIB( $n$ )
    
```

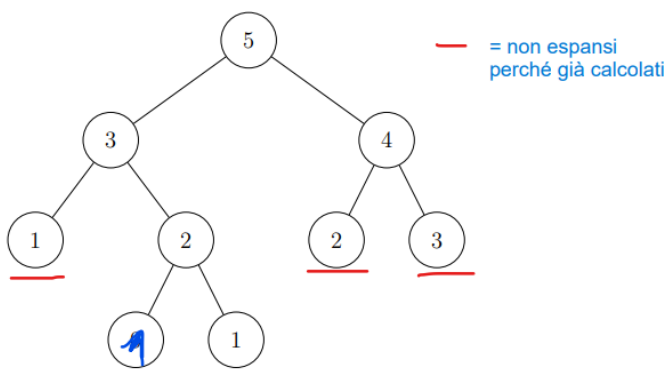
REC-FIB(i)

```

1  if  $F[i] = 0$ 
2       $F[i] = \text{REC-FIB}(i - 2) + \text{REC-FIB}(i - 1)$ 
3  return  $F[i]$ 
    
```

Riprendiamo l'esempio di prima, con $n = 5$

Complessità $\Theta(n)$



Numero nodi =

$n - 1 + n$
(interni) (foglie)

Complessità: $O(n)$

Ora, consideriamo i problemi di ottimizzazione, per cui si cerca di trovare una cosiddetta *soluzione ottima*, definita come s^* , definendo tutto come segue:

I = insieme delle istanze

S = insieme delle soluzioni

$\Pi \subseteq I \times S$

$\forall i \in I, S(i) = \{s \in S : (i, s) \in \Pi\}$ = insieme delle soluzioni ammissibili

funzione di costo $c: S \rightarrow \mathbb{R}$

Determinare, data $i \in I, s^* \in S(i) : c(s^*) = \min(/ \max)\{c(s) : s \in S(i)\}$

Paradigma generale sullo sviluppo di un algoritmo di programmazione dinamica.

Caratteristiche dei problemi:

- 1) Struttura ricorsiva: la soluzione ottima si ottiene come funzione di soluzioni ottime di sottoistanze ("proprietà di sottostruttura ottima", quindi il fatto che ogni soluzione contenga già soluzioni precedenti)
- 2) Esistenza di sottoistanze ripetute ("overlapping subproblems"), teoricamente risolvibili in maniera *ottima* (altrimenti, conviene applicare D&C)
- 3) Spazio dei sottoproblemi piccolo: poche sottoistanze che possono contribuire a creare la soluzione al livello superiore.

Ricetta

- 1) Caratterizza la struttura di una soluzione ottima s^* in funzione di soluzioni ottime $s_1^*, \dots, s_K^*$ di sottoistanze di taglia inferiore.
- 2) Determinare una relazione di ricorrenza del tipo $c(s^*) = f(c(s_1^*), \dots, c(s_K^*))$
- 3) Calcola $C(S^*)$ utilizzando la ricorrenza ma impostando il calcolo in maniera bottom-up (iterativo) oppure memoizzando il codice D&C basato sulla definizione di $C(S^*)$
- 4) (opzionale) Mantieni informazioni strutturali aggiuntive che permettono di ricostruire S^* .

Applicazioni della programmazione dinamica

Problemi su stringhe

Def Dato un alfabeto finito Σ , una **stringa**

$$X = \langle x_1, x_2, \dots, x_m \rangle, \quad x_i \in \Sigma \quad \forall 1 \leq i \leq m$$

è una concatenazione finita di simboli in Σ .

$m = |X|$ = lunghezza di X

Σ^* = insieme di tutte le stringhe di lunghezza finita costruibili su Σ

ε = stringa vuota

Data una stringa X , il **prefisso** di X è

$$X_i = \langle x_1, x_2, \dots, x_i \rangle, \quad 1 \leq i \leq m$$

Data una stringa X , il **suffisso** di X è

$$X^i = \langle x_i, x_{i+1}, \dots, x_m \rangle, \quad 1 \leq i \leq m$$

Per convenzione $X_0 = X^{m+1} = \varepsilon$

Def Data una stringa X , la **sottostringa** di X è

$$X_{i...j} = \langle x_i, x_{i+1}, \dots, x_j \rangle, \quad 1 \leq i \leq j \leq m$$

Per convenzione $X_{i...j} = \varepsilon$ se $i > j$

Quante sono le sottostringhe di una stringa?

$$1 + \sum_{i=1}^n \sum_{j=1}^m 1 = 1 + \sum_{i=1}^m (m - i + 1) = 1 + \sum_{k=1}^m K = 1 + m(m+1)/2 = \theta(m^2)$$

$\nearrow \varepsilon$ $\nearrow$ Stringhe con 1 carattere $\nearrow$ Sottostringhe con ≥ 2 caratteri
 $\left(1 + m + \frac{m(m-1)}{2}\right) = 1 + m + (m(m-1))/2 = \theta(m^2)$

➔ Spazio di sottostringhe non è troppo grande

Def Data una stringa

$$X = \langle x_1, x_2, \dots, x_m \rangle \in \Sigma^*$$

e

$$Z = \langle z_1, z_2, \dots, z_k \rangle \in \Sigma^*$$

si dice che Z è **sottosequenza** di X se $\exists$ una successione crescente di indici

$$1 \leq i_1 \leq i_2 \leq \dots \leq i_k \leq m : z_j = x_{i_j} \quad \forall 1 \leq j \leq k$$

Esempio

$$X = \langle a, b, c, b, b, d \rangle$$

$$Z_1 = \langle a, b, c \rangle = X_{1...3}$$

$$Z_2 = \langle a, c, b \rangle \quad i_1 = 1, \quad i_2 = 3, \quad i_3 = 4 \text{ o } 5$$

$$Z_3 = \langle b, b \rangle = X_{4...5} \quad i_1 = 2, \quad i_2 = 5$$

possibili sottosequenze di una stringa con m caratteri:

$$\sum_{k=0}^m \binom{m}{k} = 2^m$$

$\uparrow$
 stringhe lunghe k
 prese da un insieme
 di m elementi

➔ Spazio delle sottosequenze è enorme

Lezione 15: Longest Common Subsequence (LCS)

7.5.1 Problema di Ottimizzazione

Date due stringhe X, Y determina Z tale che:

- 1) Z è sottosequenza di X e Y ;
- 2) Z è la più lunga tra tutte le sottosequenze comuni.

Esempio

$$X = \langle a, b, c, b, b, d \rangle$$

$$Y = \langle a, d, c, c, b, d \rangle$$

$Z = \langle a, c, b, d \rangle$ è una LCS (in questo caso è l'unica)

$$i_1 = 1, \quad i_2 = 3, \quad i_3 = 4 \text{ o } 5, \quad i_4 = 6$$

$$j_1 = 1, \quad j_2 = 3 \text{ o } 4, \quad j_3 = 5, \quad j_4 = 6$$

Risolvero il problema:

$$|X| = m$$

$$|Y| = n$$

L'approccio "brute force" ha complessità $\Omega(2^m \cdot 2^n)$.

Avendo complessità esponenziale, si cerca di individuare una struttura ricorsiva, cioè una proprietà di sottostruttura, cioè la LCS deve "nascondere" al suo interno LCS di qualche stringa più piccola di X e Y .

$$X = \langle X', a \rangle \Rightarrow Z \text{ è la stringa più lunga tra } LCS(X', Y) \text{ e } LCS(X, Y')$$

$$Y = \langle Y', b \rangle \quad (\text{e in generale } Z \text{ o non termina con } a, \text{ o non termina con } b)$$

spazio sottoproblemi: $S = \{LCS(X_i, Y_j) : 0 \leq i \leq m, 0 \leq j \leq n\}$

$$|S| = (m+1)(n+1)$$

Proprietà di sottostruttura ottima per il sottoproblema $LCS(X_i, Y_j)$

Dati i prefissi

$$X_i = \langle x_1, x_2, \dots, x_i \rangle$$

$$Y_j = \langle y_1, y_2, \dots, y_j \rangle$$

$$\text{Sia } Z = \langle z_1, z_2, \dots, z_k \rangle = LCS(X_i, Y_j)$$

0. caso base: o $i = 0$ o $j = 0$

$$\Rightarrow Z = \varepsilon$$

1. $i, k > 0$

se $x_i = y_j$ allora

$$(a) \quad z_k = x_i (= y_j)$$

$$(b) \quad Z_{k-1} = LCS(X_{i-1}, Y_{j-1})$$

2. $i, j > 0$

se $x_i \neq y_j$ allora

Z è la stringa di lunghezza massima tra $LCS(X_i, Y_{j-1})$ e $LCS(X_{i-1}, Y_j)$

Dimostrazione

0. banale

1. $x_i = y_j$

$$Z = LCS(X_i, Y_j) = \langle z_1, z_2, \dots, z_k \rangle = \langle x_{i_1}, x_{i_2}, \dots, x_{i_k} \rangle = \langle y_{j_1}, y_{j_2}, \dots, y_{j_k} \rangle$$

$$1 \leq i_1 \leq i_2 \leq \dots \leq i_k \leq i, \quad 1 \leq j_1 \leq j_2 \leq \dots \leq j_k \leq j$$

(a) Ragioniamo per assurdo

$$z_k = x_{i_k} = y_{j_k}$$

$$z_k \neq (x_i = y_j)$$

$$\Rightarrow i_k < i, \quad j_k < j$$

$$Z' = \langle Z, x_i \rangle$$

$$1 \leq i_1 \leq i_2 \leq \dots \leq i_k \leq i_{k+1} = i, \quad 1 \leq j_1 \leq j_2 \leq \dots \leq j_k \leq j_{k+1} = j$$

(b) Devo dimostrare che

$$Z_{k-1} = LCS(X_{i-1}, Y_{j-1})$$

$$Z_{k-1} = \langle x_{i_1}, x_{i_2}, \dots, x_{i_{k-1}} \rangle = \langle y_{j_1}, y_{j_2}, \dots, y_{j_{k-1}} \rangle$$

$$i_{k-1} \leq i-1 < i$$

$$Z_{k-1} = CS(X_{i-1}, Y_{j-1})$$

Ora dimostro che

$$Z_{k-1} = LCS(X_{i-1}, Y_{j-1})$$

Suppongo per assurdo che

$$Z_{k-1} \neq LCS(X_{i-1}, Y_{j-1})$$

$$\Rightarrow \exists Z' \text{ con } |Z'| \geq k$$

$$\Rightarrow \text{creo } Z'' = \langle Z', x_i (= y_j) \rangle$$

$$\begin{array}{ccc} \uparrow & \uparrow & \\ \geq k & 1 & \Rightarrow \geq k+1 \end{array}$$

2. $x_i \neq y_j (i, j > 0)$ Basta dimostrare che $Z = LCS(X_i, Y_{j-1})$ oppure $Z = LCS(X_{i-1}, Y_j)$ a) $i_k = i \Rightarrow j_k < j \Rightarrow Z \text{ è } CS(X_i, Y_{j-1})$ dove CS indica "Common Sequence" $\Rightarrow x_i \neq y_j$ $Z \text{ è anche } LCS(X_i, Y_{j-1}) \rightarrow$ (per assurdo, come prima)b) $i_k < i \Rightarrow Z \text{ è } CS(X_{i-1}, Y_j)$ dove $Z \text{ è } CS(X_{i-1}, Y_j) \rightarrow$ (per assurdo, come prima)

Ricorrenza sui costi

La scrittura della ricorrenza sui costi è il secondo passo per costruire un algoritmo di programmazione dinamica.

Definisco

$$l(i, j) = |LCS(X_i, Y_j)|$$

$$l(i, j) = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 & (\text{caso 0}) \\ l(i-1, j-1) + 1 & \text{se } i, j > 0 \text{ e } x_i = x_j & (\text{caso 1}) \\ \max\{l(i, j-1), l(i-1, j)\} & \text{se } i, j > 0 \text{ e } x_i \neq x_j & (\text{caso 2}) \end{cases}$$

Alla fine ci interessa calcolare $l(m, n)$.

È chiara la presenza di sottoproblemi ripetuti (se volessimo vederlo in ottica D&C):

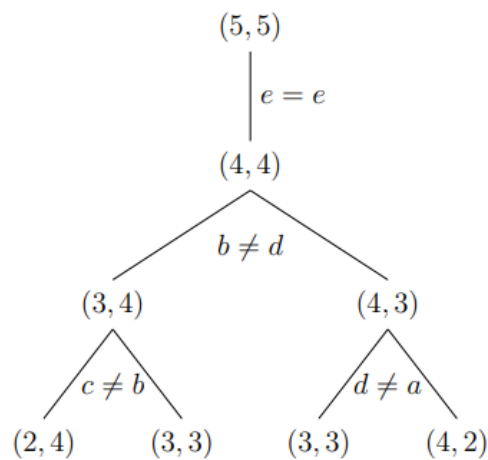
Esempio

$X = \langle a, b, c, d, e \rangle$

$Y = \langle b, c, a, b, e \rangle$

Trova $LCS(X, Y)$

Albero delle chiamate:



L'istanza (3, 3) è ripetuta.

Modello di costo: confronto tra caratteri.

$$T(n, m) = \begin{cases} 0 & \text{se } n = 0 \text{ o } m = 0 \\ T(n-1, m) + T(n, m-1) + 1 & \text{se } n, m > 0 \end{cases}$$

Si dimostra che

$$T(n, m) = \Theta \left(\binom{m}{n} \right)$$

$$\binom{m}{2} \geq \binom{m}{2}^n$$

Caso $m = 2n$

$$\binom{m}{2}^n = 2^n$$

esponenziale sulla taglia
($= 3n$)

$\Rightarrow$ D&C non funziona, pertanto scrivo codice di programmazione dinamica (bottom up/iterativo) o memoizzato $\Rightarrow L[i, j] \leftarrow l(i, j)$ dove con $L[i, j]$ indica una tabella bidimensionale.

Per calcolare $l(i, j)$ mi possono servire tre valori:

$$L = \left[\begin{array}{cc} (i-1, j-1) & (i-1, j) \\ & \swarrow \uparrow \\ (i, j-1) & \leftarrow (i, j) \end{array} \right]$$

Scansione "row-major": riempio la tabella per righe, da sinistra a destra.

Informazione aggiuntiva per costruire la sequenza (vera e propria):

$$b(i, j) = \begin{cases} \swarrow & \text{se } x_i = y_j \\ \leftarrow & \text{se } x_i \neq x_j \text{ e } \max = LCS(i, j-1) \\ \uparrow & \text{se } x_i \neq y_j \text{ e } \max = LCS(i-1, j) \end{cases}$$

Come funziona l'algoritmo:

Prendiamo due sequenze:

X

A	C	A	D	B
---	---	---	---	---

Y

C	B	D	A
---	---	---	---

I seguenti passaggi vengono seguiti per trovare la sottosequenza comune più lunga.

1. Create una tabella di dimensione $n+1*m+1$ dove n e m sono le lunghezze rispettivamente di X e Y . La prima riga e la prima colonna sono piene di zeri.

		C	B	D	A
	0	0	0	0	0
A	0				
C	0				
A	0				
D	0				
B	0				

2. Riempire ogni cella della tabella utilizzando la logica seguente.
3. Se il carattere corrispondente alla riga corrente e alla colonna corrente corrispondono, riempire la cella corrente aggiungendone uno all'elemento diagonale. Puntare una freccia verso la cella diagonale.
4. Altrimenti prendere il valore massimo dalla colonna precedente e dall'elemento riga precedente per riempire la cella corrente. Puntare una freccia sulla cella con il valore massimo. Se sono uguali, indica uno di essi.

		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0				
A	0				
D	0				
B	0				

5. Il passaggio 2 viene ripetuto fino a quando la tabella non viene riempita.

	C	B	D	A
A	0	0	0	0
C	0	1	1	1
A	0	1	1	2
D	0	1	2	2
B	0	1	2	2

6. Il valore nell'ultima riga e nell'ultima colonna è la lunghezza della sottosequenza

	C	B	D	A
A	0	0	0	0
C	0	1	1	1
A	0	1	1	2
D	0	1	2	2
B	0	1	2	2

comune più lunga.

7. Per trovare la sottosequenza comune più lunga, iniziare dall'ultimo elemento e seguire la direzione della freccia. Gli elementi corrispondenti al simbolo () formano la sottosequenza comune più lunga.

	C	B	D	A
A	0	0	0	0
C	0	1	1	1
A	0	1	1	2
D	0	1	2	2
B	0	1	2	2

Select the cells with diagonal arrows

	C	B	D	A
A	0	0	0	0
C	0	1	1	1
A	0	1	1	2
D	0	1	2	2
B	0	1	2	2

Pertanto, la sottosequenza comune più lunga è CA.

C

A

Pseudocodice

LCS(X, Y)

```

1   $m = X.length$ 
2   $n = Y.length$ 
3  for  $i = 0$  to  $m$ 
4       $L[i, 0] = 0$ 
5  for  $j = 0$  to  $n$ 
6       $L[0, j] = 0$ 
7  for  $i = 1$  to  $m$ 
8      for  $j = 1$  to  $n$ 
9          if  $x_i = y_j$ 
10              $L[i, j] = L[i - 1, j - 1] + 1$ 
11              $B[i, j] = '\nwarrow'$ 
12         else if  $L[i - 1, j] \geq L[i, j - 1]$ 
13              $L[i, j] = L[i - 1, j]$ 
14              $B[i, j] = '\uparrow'$ 
15         else
16              $L[i, j] = L[i, j - 1]$ 
17              $B[i, j] = '\leftarrow'$ 
18 return ( $L[m, n], B$ )

```

Complessità

$$T(m, n) = \Theta(m \cdot n)$$

$$\text{Caso } m = n \Rightarrow T(m, n) = \Theta(n^2)$$

Esercizio

$$X = \langle b, d, c, d \rangle$$

$$Y = \langle a, b, c, b, d \rangle$$

Restituisci $LCS(X, Y)$ e $|LCS(X, Y)|$

$$L = \begin{array}{c} \begin{array}{ccccc} & a & b & c & b & d \\ b & 0 & 0 & 0 & 0 & 0 \\ d & 0 & 0 & 1 & 1 & 1 \\ c & 0 & 0 & 1 & 2 & 2 \\ d & 0 & 0 & 1 & 2 & 3 \end{array} \end{array} \quad B = \begin{array}{c} \begin{array}{ccccc} & a & b & c & b & d \\ b & \uparrow & \nwarrow & \leftarrow & \nwarrow & \leftarrow \\ d & \uparrow & \uparrow & \uparrow & \uparrow & \nwarrow \\ c & \uparrow & \uparrow & \nwarrow & \leftarrow & \uparrow \\ d & \uparrow & \uparrow & \uparrow & \uparrow & \nwarrow \end{array} \end{array}$$

$$LCS(X, Y) = \langle b, c, d \rangle \quad |LCS(X, Y)| = 3$$

Procedura per stampare la LCS:

PRINT-LCS(B, X, i, j)

```

1  if  $i = 0$  or  $j = 0$ 
2      return  $\varepsilon$ 
3  if  $B[i, j] = '\nwarrow'$ 
4      PRINT-LCS( $B, X, i - 1, j - 1$ )
5      PRINT( $x_i$ )
6  else if  $B[i, j] = '\leftarrow'$ 
7      PRINT-LCS( $B, X, i, j - 1$ )
8  else //  $B[i, j] = '\uparrow'$ 
9      PRINT-LCS( $B, X, i - 1, j$ )

```

Complessità $\Theta(m) = \Theta(|LCS|)$

In particolare, per la LCS:

- La prima riga e la prima colonna sono sempre inizializzate a 0
- Appena si ha un nuovo match, si incrementa il conteggio di 1
- Le righe successive controllano sempre la riga precedente verificando se si ha avuto un match in quella posizione, oppure verificando in diagonale a sx se si ha avuto un match;
 - o se si ha avuto un match per la stessa lettera sulla stessa riga, ma in diagonale a sx si aveva 0, si ha 1 sulla riga attuale
 - o se si ha avuto un match per la stessa lettera sulla stessa riga, ma in alto o in diagonale a sinistra si ha lo stesso numero, allora si incrementa perché si trova un match
- Se si ha la freccia in alto, non si hanno avuti match
- Se si ha la freccia che va a sinistra, si ha avuto un match sulla stessa riga in posizione precedente

Ora scriviamo il codice *memoizzato* (e vedremo che in alcuni casi è più efficiente):

INIT-LCS(X, Y)

```

1   $m = X.length$ 
2   $n = Y.length$ 
3  if ( $m = 0$ ) or ( $n = 0$ )
4      return 0
5  for  $i = 0$  to  $m$ 
6       $L[i, 0] = 0$ 
7  for  $j = 0$  to  $n$ 
8       $L[0, j] = 0$ 
9  for  $i = 1$  to  $m$ 
10     for  $j = 1$  to  $n$ 
11          $L[i, j] = -1$ 
12 return R-LCS( $X, Y, m, n$ )

```

Si usa un array 2-D per memorizzare il valore calcolato di $lcs(m, n)$ in $arr[m - 1][n - 1]$, poiché l'indice della stringa inizia da 0.

Ogni volta che la funzione con gli stessi argomenti m ed n viene richiamata, non si eseguono ulteriori chiamate ricorsive e si restituisce $arr[m - 1][n - 1]$, poiché il calcolo precedente di $lcs(m, n)$ è già stato memorizzato in $arr[m - 1][n - 1]$, riducendo così le chiamate ricorsive che avvengono più di una volta.

R-LCS(X, Y, i, j)

```

1  if  $L[i, j] = -1$ 
2      if  $x_i = y_j$ 
3           $L[i, j] = \text{R-LCS}(X, Y, i - 1, j - 1)$ 
4      else if  $\text{R-LCS}(X, Y, i - 1, j) \geq \text{R-LCS}(X, Y, i, j - 1)$ 
5           $L[i, j] = L[i - 1, j]$ 
6      else
7           $L[i, j] = L[i, j - 1]$ 
8  return  $L[i, j]$ 

```

In particolare, si ha:

- una procedura di inizializzazione, per riempire la tabella bidimensionale di analisi (messo a 0 all'inizio per le celle i e j , per poi porre, al secondo giro di inizializzazione, -1.
- una procedura che controlla che tutte le celle siano a -1 e parte dalla fine della tabella fino all'inizio iterativamente (vuol dire che quei valori devono essere ancora esaminati) e:
 - o se i valori sono uguali procedo in diagonale
 - o se il valore della riga precedente/stessa colonna è $\geq$ rispetto al valore della stessa riga e colonna precedente, si sposta verticalmente
 - o altrimenti, si sposta lateralmente,

Infatti, ricordiamo che:

$$b(i, j) = \begin{cases} \swarrow & \text{se } x_i = y_j \\ \leftarrow & \text{se } x_i \neq x_j \text{ e } \max = \text{LCS}(i, j - 1) \\ \uparrow & \text{se } x_i \neq y_j \text{ e } \max = \text{LCS}(i - 1, j) \end{cases}$$

Al caso peggior, la complessità è $O(m * n)$, nel caso migliore ha complessità più bassa.

Osservazione:

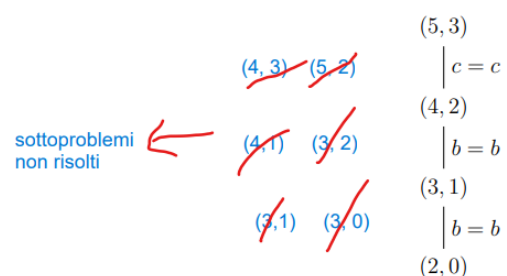
- algoritmo iterativo fa sempre $m * n$ confronti
- nell'algoritmo memoizzato, se $x_i = y_j$ non calcolo i sottoproblemi $i - 1, j$ e $i, j - 1 \Rightarrow$ nelle loro posizioni in L rimane il valore di default

Ad esempio

$X = \langle a, a, b, b, c \rangle$

$Y = \langle b, b, c \rangle$

Albero delle chiamate:



In generale, se Y è suffisso di $n \leq m$ caratteri di X , la complessità di R-LCS nel caso migliore è:

$$T_{R-LCS}(m, n) = n$$

(addirittura, sublineare)

Inoltre,

$$\begin{aligned}\Omega_{LCS}(m+n) &\cong \Omega(n) \\ O_{LCS, R-LCS}(m \cdot n) &\cong O(n^2)\end{aligned}$$

Spazio

$$S_{LCS}(m, n) = \Theta(m, n)$$

Tuttavia, posso migliorarlo a

$$\Theta(2n) = \Theta(n)$$

poichè mi bastano due righe della tabella in memoria ad ogni istante, quindi due vettori lunghi n .

Inoltre, se $m \ll n$, posso fare un'ulteriore ottimizzazione utilizzando la tecnica "column-major", cioè scansione per colonne, con due vettori lunghi m .

Lezione 16: Longest Increasing Subsequence

Esempio di problema dove applicheremo lo *Strenghtening (rafforzamento)*, cioè risolveremo sottoproblemi più vincolati.

Def Dato un alfabeto Σ totalmente ordinato ($\forall a, b \in \Sigma \quad a < b$ o $a = b$ o $a > b$) e dato $X = \langle x_1, x_2, \dots, x_n \rangle$, si dice che $Z = \langle z_1, z_2, \dots, z_k \rangle$ è **sottosequenza crescente** di X ($Z = LIS(X)$).

7.6.1 Problema di Ottimizzazione

Determinare la **più lunga sottosequenza crescente** di X ($Z = LIS(X)$)

Esempio

$$X = \langle 5, 2, 4, 3, 7, 8 \rangle$$

$$Z = LIS(X) = \langle 2, 3, 7, 8 \rangle = LIS(x)$$

$$Z' = LIS(X) = \langle 2, 4, 7, 8 \rangle \text{ è un'altra LIS}(x)$$

Algoritmo esaustivo $\rightarrow$ esponenziale

Provo a fare come per LCS: Dato X , cerco la $LIS(X_i) \forall i$ sapendo che $\exists$ una proprietà di sottostruttura ottima che mette in relazione $LIS(X_i)$ con altre LIS .

Tentativo Data X , calcolo $LIS(X_i) \forall 0 \leq i \leq n$ calcolando quindi:

$$Z = \langle z_1, z_2, \dots, z_k \rangle = LIS(X_i)$$

$$Z^{(i+1)} = LIS(X_{(i+1)})$$

$$\circ \text{ caso fortunato: } z_k < X_{i+1}$$

$$X_{(i+1)} = X_i + X_{(i+1)}$$

$$\circ \text{ caso sfortunato: } z_k \geq X_{i+1}$$

Def $Z = \overline{LIS}(X_i)$ è la più lunga tra le $IS(X_i)$ con $Z = \langle z_1, z_2, \dots, z_k \rangle = \langle x_{i_1}, x_{i_2}, \dots, x_{i_k} \rangle$ con $i_k = i$.

- Caso fortunato: $Z_k < X_{(i+1)} \Rightarrow LIS(X_{(i+1)}) = \langle Z^i, X_{(i+1)} \rangle$
- Caso sfortunato: $Z_k > X_{(i+1)}$ posso dire $\Rightarrow LIS(X_{(i+1)}) = Z^i$?
NO, perché Z^i è una $LIS(X_i)$: potrebbe essercene un'altra per cui mi troverei nel caso fortunato e quindi $|LIS(X_{(i+1)})| = 1 + |LIS(X_i)|$
Esempio: $X_i = \langle 4, 5, 6, 1, 2, 3 \rangle > X_{(i+1)} = 5$

Possibile soluzione

Sottoproblemi con proprietà aggiuntive:

Calcolare la più lunga IS di X_i che termina proprio con X_i

Definizione: $Z = \overline{LIS}(X_i)$ è la più lunga tra le $IS/Increasing Subsequence (X_i)$ con $Z = \langle z_1, \dots, z_k \rangle \geq \langle x_{i_1}, x_{i_2}, \dots, x_{i_k} \rangle$ con $i_k = i$

Esempio:

$X = \langle 8, 2, 5, 1, 3 \rangle$

$LIS(X_4) = \langle 2, 5 \rangle$

$\Rightarrow$ non c'è relazione immediata tra $LIS(X_4)$ e $\overline{LIS}(X_4)$

$\overline{LIS}(X_4) = \langle 1 \rangle$

deve finire con 1

$\overline{LIS}$ è un problema più vincolato $\Rightarrow |\overline{LIS}(X_i)| \leq |LIS(X_i)|$

Considero $LIS(X_i)$: da qualche parte deve pur finire! Rispetto a quella posizione, è anche $\overline{LIS}$. Ma non so dove finisce $\rightarrow$ calcolo tutte le $\overline{LIS}$ (una per ogni i) e ne considero il valore massimo di LIS , cioè:

$$|LIS(X)| = \max_{1 \leq i \leq n} |\overline{LIS}(X_i)|$$

Dimostrazione

$$\circ |LIS(X)| \leq \max_{1 \leq i \leq n} \{|\overline{LIS}(X_i)|\}$$

$$Z = LIS(X) = \langle x_{i_1}, x_{i_2}, \dots, x_{i_k} \rangle$$

$$Z = \overline{LIS}(X_{i_k})$$

$$\Rightarrow |Z| \leq \max_{1 \leq i \leq n} \{|\overline{LIS}(X_i)|\}$$

$$\circ |LIS(X)| \geq \max_{1 \leq i \leq n} \{|\overline{LIS}(X_i)|\}$$

Si dimostra analogamente al punto precedente.

Proprietà di Sottostruttura Ottima (tra le $\overline{LIS}$ dei prefissi piuttosto che tra le LIS)

1. caso base: $\overline{LIS}(X_1) = \langle x_1 \rangle (= LIS(X_1))$

2. $i > 1$

$$(a) \forall j : 1 \leq j \leq i \quad x_j \geq x_i$$

$$\overline{LIS}(X_i) = \langle x_i \rangle (= LIS(X_i))$$

$$(b) \exists \bar{j} : 1 \leq \bar{j} \leq i, \quad x_{\bar{j}} < x_i$$

$$|\overline{LIS}(X_i)| \geq 2$$

$$\overline{LIS}(X_i) = \langle z_1, z_2, \dots, z_k \rangle = \langle Z_{k-1}, x_i \rangle \text{ con } Z_{k-1} : |Z_{k-1}| = \max_{1 \leq j < i} \{|\overline{LIS}(X_j)| : x_j < x_i\}$$

Dimostrazione

1. banale

2. $i > 1$

$$(a) \quad \forall j < i \quad x_j < x_i$$

$\langle x_i \rangle = LIS(X_i)$ e chiaramente non può essere $|Z| > 1$

Suppongo per assurdo che

$$|Z| = |\overline{LIS}(X_i)| > 1$$

$$\Rightarrow Z = \langle z_1, z_2, \dots, z_k \rangle, \quad k > 1$$

$$Z = \langle x_{i_1}, x_{i_2}, \dots, x_{i_k} \rangle$$

$$i_k = i$$

$$\Rightarrow i_{k-1} < i_k = i$$

$Z_{(k+1)} = \overline{LIS}(X_{i_{(k-1)}})$: se non fosse la più lunga, ne esiste un'altra: gli aggiungo $x_{i_k} = x_i$ e ottengo una $\overline{LIS}(X_i)$ di lunghezza $|Z| = K$: assurdo. Ora dimostro che tra tutte le $\overline{LIS}$ dell'insieme è quella più lunga: se così fosse potrei aumentare $|Z| = K$: assurdo.

Ricorrenza sui costi

Definisco

$$l(i) = |\overline{LIS}(X_i)|$$

$$l(i) = \begin{cases} 1 & \text{se } i = 1 \\ 1 + \max_{1 \leq j < i} \{l(j) : x_j < x_i\} & \text{se } i > 1 \end{cases}$$

Per costruire la sottosequenza

$$\overline{LIS}(X_i) = \begin{cases} \langle x_i \rangle & (1) \\ \langle \overline{LIS}(X_{\bar{j}}), x_i \rangle & \text{con } 1 \leq \bar{j} < i \quad (2) \end{cases}$$

l'informazione addizionale è:

$$\circ \text{ prev}(i) = \begin{cases} 0 & \text{nel caso (1)} \\ \bar{j} & \text{nel caso (2)} \end{cases}$$

$$\circ \text{ len} = \max_{1 \leq i \leq n} \{l(i)\}$$

$$\circ \text{ end} = i \quad \text{se } \overline{LIS}(X_i) = LIS(X)$$

(mantiene l'indice dell'ultimo carattere della LIS)

La LIS è quindi composta da $\langle \dots, X_{(\text{prev}(\text{prev}(i)))}, X_{(\text{prev}(i))}, X_i \rangle$

Non possiamo usare un algoritmo ricorsivo, in quanto abbiamo diversi sottoproblemi ripetuti.

Scriviamo quindi l'algoritmo *bottom-up*, a partire da $\overline{LIS}(X_1)$.

Nell'approccio bottom-up, si risolvono prima i sottoproblemi più piccoli; quindi, si risolvono i sottoproblemi più grandi a partire da questi. Il seguente approccio bottom-up calcola $L[i]$, per ogni

$0 \leq i < n$, che memorizza la lunghezza della sottosequenza crescente più lunga del sottoarray $arr[0 \dots i]$ che termina con $arr[i]$. Per calcolare $L[i]$, si considera il LIS di tutti i valori minori di i (ad esempio j) già calcolati e si sceglie il massimo $L[j]$, dove $arr[j]$ è inferiore all'elemento corrente $arr[i]$.

Quindi:

LIS(X)

```

1  L[1] = 1
2  len = 1
3  end = 1
4  prev[1] = 0
5  for i = 2 to n
6      L[i] = 1
7      prev[i] = 0
8      for j = 1 to i - 1
9          if xj < xi
10             if L[i] < 1 + L[j]
11                 L[i] = 1 + L[j]
12                 prev[i] = j
13     if len < L[i]
14         len = L[i]
15     end = i
16 return (len, prev, end)
```

- inizializzo tutti i valori di ricerca della sottosequenza
- nel primo ciclo (che va in avanti), inizializzo la lunghezza dell'elemento attuale e inizializzo l'elemento precedente.
- nel secondo ciclo, che parte dalla fine per poter ottimizzare il proprio calcolo e avere già la soluzione ottimale partendo dal primo elemento, confronto l'elemento i-esimo con l'elemento j-esimo
- se fosse minore, si salva l'elemento attuale j-esimo nell'array i-esimo e si salva la lunghezza raggiunta e dove si ferma

Stampa della stringa con apposita procedura (considerando un indice *i* da cui inizia)

```

R-PRINT(X, prev, i)
1  if prev[i] ≠ 0
2      R-PRINT(X, prev, prev[i])
3  PRINT(xi)
```

$$\text{Complessità} \quad \sum_{i=2}^n \sum_{j=1}^{i-1} 1 = \sum_{i=2}^n (i-1) = \frac{n(n-1)}{2} = \Theta(n^2)$$

Esempio completo a:

<https://www.javatpoint.com/longest-increasing-subsequence>

<https://upload.wikimedia.org/wikipedia/commons/thumb/1/1d/LISDemo.gif/400px-LISDemo.gif>

https://astikanand.github.io/techblogs/dynamic-programming-patterns/assets/longest_increasing_subsequence_tabulation.gif

La pratica è questa:

- considero il mio array delle sottostringhe e cerco di trovare le sottosequenze che crescono
- nell'array delle lunghezze, consideriamo che noi teniamo traccia delle sottosequenze attuali che si incrementano, considerano che teniamo *solo le sottosequenze che, a parità di numero di elementi, hanno un valore nella stessa posizione minore. Non avremo mai due sottosequenze che si incrementano con lo stesso numero di elementi*
- noi confrontiamo gli elementi della sottostringa uno per uno; l'array delle lunghezze si aggiorna "per riga", quindi, salviamo solo le posizioni in cui la lunghezza della sottostringa sottostante aumenta, tale da poter individuare facilmente qual è la sottosequenza migliore.

Lezione 17: Shortest Palindrome Completion

Def Una stringa $Z = \langle z_1, z_2, \dots, z_m \rangle$ è **palindroma** se $z_{1+h} = z_{m-h} \quad \forall 0 \leq h \leq m-1$.

Problema Data $X = \langle x_1, x_2, \dots, x_n \rangle$, un **complemento palindromo** di X è una stringa $Z = CP(X) = \langle z_1, z_2, \dots, z_m \rangle$ con $m \geq n$ tale che:

- 1) Z è palindroma;
- 2) X è sottosequenza di Z

(cioè, Z è un palindromo che contiene X come sottosequenza).

Esempio: $Z = abccba$ è CP/Completing Palindrome di $X = acb$.

Lo spazio dei sottoproblemi è $X_{(i..j)}$ e algoritmo quadratico in $n \Rightarrow$ bisogna, $\forall i, j \ 1 \leq i < j \leq n$ determinare il minimo CP di $X_{(i..j)}$

Proprietà di sottostruttura ottima

Casi base:

1. $i = j$ (1 carattere)

$$X = \langle x_i \rangle$$

$$CP(X_{i..j}) = X_{i..j}$$

$$|CP(X_{i..j})| = |X_{i..j}|$$

(a) $x_i = X_{i+1}$

$$CP(X_{i..i+1}) = X_{i..i+1}$$

$$|CP(X_{i..i+1})| = 2$$

(b) $x_i \neq x_{i+1}$

Un possibile $CP(X_{i..i+1})$ è $\langle x_i, x_{i+1}, x_i \rangle$

$$|CP(X_{i..i+1})| = 3$$
2. $j = i + 1$ (2 caratteri)

$$X_{i..j} = \langle x_i, x_{i+1} \rangle$$

Caso generale:

$$X_{i..j} = \langle x_i, x_{i+1}, \dots, x_j \rangle$$

- (a) $x_i = x_j$

$$Z = CP(X_{i..j})$$

$$z_1 = z_k = x_i (= x_j)$$

$$Z_{2..k-1} = CP(X_{i+1..j-1})$$
- (b) $x_i \neq x_j$ Ci sono due possibili soluzioni:
 - i. $z_1 = z_k = x_i$ e $Z_{2..k-1} = CP(X_{i+1..j})$
 - ii. $z_1 = z_k = x_j$ e $Z_{2..k-1} = CP(X_{i..j-1})$

Dimostrazione

(b).i. Suppongo per assurdo che

$z_1 = z_k \neq x_i$
 $\Rightarrow X_{i..j}$ è sottosequenza di $Z_{2..k-1}$ che è palindroma e più corta di 2 rispetto a Z
 Assurdo perchè Z è la più corta!

(b).ii. Si dimostra analogamente al sottocaso precedente.

7.7.3 Ricorrenza sulle Lunghezze

Definisco

$$l(i, j) = |CP(X_{i..j})|$$

$$l(i, j) = \begin{cases} 1 & \text{se } i = j \\ 2 & \text{se } j = i + 1 \text{ e } x_i = x_j \\ 3 & \text{se } j = i + 1 \text{ e } x_i \neq x_j \\ 2 + l(i + 1, j - 1) & \text{se } j > i + 1 \text{ e } x_i = x_j \\ 2 + \min\{l(i + 1, j), l(i, j - 1)\} & \text{se } j > i + 1 \text{ e } x_i \neq x_j \end{cases}$$

Per calcolare $l(i, j)$ mi possono servire tre valori:

$$L = \begin{bmatrix} & & (i, j-1) \leftarrow (i, j) \\ & \swarrow & \downarrow \\ (i+1, j-1) & & (i+1, j) \end{bmatrix}$$

Riempio la tabella per diagonali, dall'alto verso il basso e da sinistra verso destra. Il motivo per cui comincio da in alto a destra della tabella è dato dal fatto di avere un palindromo. Partendo dalla fine, se il primo e l'ultimo sono diversi, capisco già che non sono palindromo.

L'algoritmo è così strutturato.

SPC(X)

```

1  n = X.length
2  for i = 1 to n - 1
3      L[i, j] = 1
4      if xi = xi+1
5          L[i, i + 1] = 2
6      else
7          L[i, i + 1] = 3
8  L[n, n] = 1
9  for l = 3 to n // scansione diagonale con l indice della diagonale
10     for i = 1 to n - l + 1
11         j = i + l - 1
12         if xi = xj
13             L[i, j] = 2 + L[i + 1, j - 1]
14         else
15             L[i, j] = 2 + min{L[i + 1, j], L[i, j - 1]}
16  return L[1, n]
```

Per l'algoritmo:

- salvo la lunghezza iniziale della parola
- inizializzo l'array delle lunghezze e copro i casi base (stringhe con 2 caratteri uguali a stringhe consecutive (metto 2) o stringhe con 2 caratteri uguali non consecutive (metto 3))
- procedo con una scansione diagonale (con l che parte da 3 avendo esaminato i casi base palindromo e assicurandomi di non avere stringa palindroma di 1-2 caratteri)
- l'altro ciclo esclude tutte le stringhe già controllate (a lunghezza 1/2)
- se le stringhe sono uguali, continuo a cercare in diagonale
- altrimenti, scelgo la stringa di lunghezza minima (perché cerco di ottimizzare il problema e quindi scelgo la stringa più corta da controllare) a sinistra e sotto.

Complessità

$$\sum_{i=1}^{n-1} 1 + \sum_{l=3}^n \sum_{i=1}^{n-l+1} 1 = \sum_{i=1}^{n-1} 1 + \sum_{l=3}^n (n-l+1) = \sum_{i=1}^{n-1} 1 + \sum_{i=2}^{n-1} (n-i) =$$

$$= \sum_{i=1}^{n-1} (n-i) = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2} = \Theta(n^2)$$

Scritto da Gabriel

$n=8 \rightarrow Z = \text{cccacabc}$ $S = \text{caca}$

1	2	1	1	1	1	1	1
1	1	3	3	1	1	1	1
1	1	1	3	1	3	3	1
1	1	1	1	1	3	1	1
1	1	1	1	1	1	1	3
1	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1

(La diagonale azzurra è riempita di tutti 1)

(La diagonale rossa è riempita di 2/3 a seconda dei casi)

(La parte sopra a destra viene riempita, spostandomi in diagonale dalla posizione verde)

(I match dell'algoritmo sono in grigio)

Non lavoro per righe, sarebbe errato, ma ragioniamo in diagonale partendo dal basso verso l'alto; notare che tolgo i caratteri uguali e considero solo una scansione diagonale per i caratteri che mi interessano (dal terzo in poi)

Esercizio: introduci sul codice nel codice il calcolo dell'informazione aggiuntiva e scrivi un algoritmo ricorsivo $PRINT_PC(i, j, B, X)$ che stampi $PC(X_{(i...j)})$.

L'informazione aggiuntiva sarà nell'array B. L'unica informazione aggiuntiva è capire come ci siamo spostati nella tabella e salvare quella nella tabella. Salviamo solo dove si è trovato il palindromo, tale poi da andare stampare quello.

```

SPC(X)
1 n = X.length
2 for i = 1 to n - 1
3     L[i, j] = 1
4     if  $x_i = x_{(i+1)}$ 
5         L[i, i + 1] = 2
6     else
7         L[i, i + 1] = 3
8 L[n, n] = 1
9 for l = 3 to n
10  for i = 1 to n - l + 1
11      j = i + l - 1
12      if  $x_i = x_j$ 
13          L[i, j] = 2 + L[i + 1, j - 1]
14          B[i, j] = ✓
15      else
16          L[i, j] = 2 + min{L[i + 1, j], L[i, j - 1]}
17 return L[1, n]
    
```

```

PRINT_PC(i, j, B, X)
1 n = X.length
2 if i = n or j = 0
3     return ε
3 if B[i, j] = ↖
4 PRINT_PC(B, X, i + 1, j - 1)
5 PRINT( $X_i$ )
    
```

Complessità: $\theta(m) = \theta(|LIS|)$

Lezione 18: Algoritmi greedy

Un algoritmo greedy è un approccio per risolvere un problema selezionando la migliore opzione disponibile al momento. Non si preoccupa di sapere se il risultato migliore attuale porterà al risultato ottimale complessivo. L'algoritmo non inverte mai la decisione precedente, anche se la scelta è sbagliata. Funziona con un approccio top-down. In particolare:

- sono semplici, costruiscono l'ottimo per scelte successive
- possiedono la proprietà di sottostruttura ottima per la decisione che prendono
- risolvono un solo sottoproblema
- hanno un campo di applicazione limitato

Vediamo un esempio classico di applicazione di un algoritmo greedy: la selezione di attività compatibili.

Abbiamo:

- risorsa condivisa (e.g. aula);
 - insieme di attività $S = \{a_i : 1 \leq i \leq n\}$
- $$a_i = [s_i, f_i), \quad 0 \leq s_i \leq f_i \quad (s_i = \text{tempo di inizio}, f_i = \text{tempo di fine})$$

Def Diciamo che a_i e a_j sono **compatibili** sse

$$[s_i, f_i) \cap [s_j, f_j) = \emptyset$$

Equivalentemente

$$f_i \leq s_j \text{ oppure } f_j \leq s_i$$

Esempio

Aula Lum250

$$a_1 = [10.30, 12.00)$$

$$a_2 = [11.30, 13.00)$$

$$a_3 = [12.30, 14.00)$$

a_1 e a_2 non sono compatibili

a_1 e a_3 sono compatibili

a_2 e a_3 non sono compatibili

Problema di ottimizzazione: determinare un sottoinsieme di massima cardinalità di attività mutuamente compatibili (cioè compatibili a coppie)

Esempio:

Ulteriore assunzione:

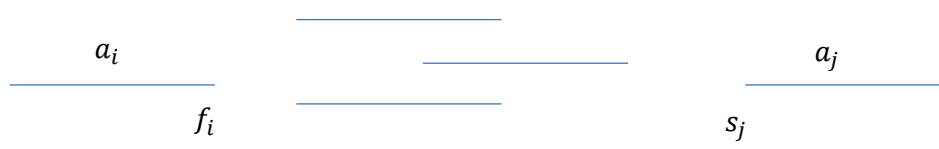
$$0 < f_1 \leq f_2 \leq \dots \leq f_n$$



Voglio applicare la programmazione dinamica:

determinare i sottoproblemi

$$S_{ij} = \{a_k : f_i \leq s_k < f_k \leq s_j\}$$



Osservazione

$$1. i > j \Rightarrow S_{ij} = \emptyset$$

perchè $f_i \leq s_k < f_k \leq s_j$

ma $f_i \geq f_j$

$$2. S_{ij} \text{ non contiene tutte le attività di indice } k \text{ con } i < k < j$$

$$3. |S_{ij}| \leq j - 1 - i, \quad 1 \leq i \leq n$$

Definisco due attività fittizie:

$$a_0 \rightarrow f_0 = 0$$

$$a_{n+1} \rightarrow s_{n+1} = +\infty$$

$$S = S_{0n+1}$$

Proprietà di sottostruttura ottima

Sia A_{ij}^* un sottoinsieme di attività compatibili di S_{ij} di cardinalità massima.

Caso base: $S_{ij} = \emptyset \Rightarrow A_{ij} = \emptyset$

Caso generale: $S_{ij} \neq \emptyset$

$$1. A_{ij}^* \neq \emptyset$$

$$2. \text{ se } a_k \in A_{ij}^* \text{ allora } A_{ij}^* = A_{ik}^* \cup A_{kj}^*$$

dove A_{ik}^* e A_{kj}^* sono soluzioni ottime per S_{ik} e S_{kj}

Dimostrazione**Dimostrazione**

Caso base: banale

Caso generale: $S_{ij} \neq \emptyset$

$$1. \text{ Un'attività è sempre compatibile con sè stessa } \Rightarrow |A_{ij}^*| \geq 1$$

$$2. \text{ Dimosteremo che una qualsiasi attività } \in A_{ij}^* \text{ e } \neq a_k \text{ si trova o in } S_{ik} \text{ o in } S_{kj}$$

Ho a_k , una qualsiasi attività $\in A_{ij}^*$ deve essere compatibile con a_k

Prendiamo $a_l \in A_{ij}^*$, con $l \neq k$

Deve valere

$$f_i \leq s_l < f_l \leq s_k \Rightarrow a_l \in S_{ik}$$

oppure

$$s_k < f_k \leq s_l < f_l \leq s_j \Rightarrow a_l \in S_{kj}$$

$$\text{Quindi } A_{ij}^* = \overline{A}_{ik} \cup \{a_k\} \cup \overline{A}_{kj}$$

dove $\overline{A}_{ij}$ = tutte le attività compatibili in S_{ij}

Ora dimostriamo che gli $\overline{A}$ sono anche A^*

Per assurdo suppongo che $\overline{A}_{ik}$ non sia l'unico insieme di attività compatibili massimale in S_{ik}

$$A_{ij}^* = \overline{A}_{ik} \cup \{a_k\} \cup \overline{A}_{kj} \rightarrow \text{soluzione con cardinalità } > |A_{ij}^*|$$

$\uparrow$
 A_{ik}^*

Ricorrenza dei costi

Definisco

$$c(i, j) = |A_{ij}^*|$$

$$c(i, j) = \begin{cases} 0 & \text{se } S_{ij} = \emptyset \\ 1 + \max_{a_k \in S_{ij}} \{c(i, k) + c(k, j)\} & \text{se } S_{ij} \neq \emptyset \end{cases}$$

L'algoritmo bottom-up ha complessità $\Theta(n^3)$ Abbiamo visto che $A_{ij}^* = A_{ik}^* \cup \{a_k\} \cup A_{kj}^*$ ma non so chi è k .Se conoscessi k potrei scrivere $1 + c(i, k) + c(k, j)$.Aspetto greedy del problema: $\exists$ attività di $S_{(ij)}$ che sta sicuramente in $A_{(ij)}^*$

Qual è questa attività?

Teorema Sia a_m l'attività tale che

$$f_m = \min\{f_k : a_k \in S_{ij}\} \quad \text{(cioè quella che finisce prima di tutte)}$$

Se $S_{ij} \neq \emptyset$ allora

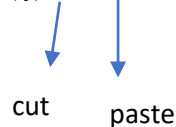
1. $\exists A_{ij}^*$ soluzione ottima tale che $a_m \in A_{ij}^*$
2. il sottoproblema $S_{im} = \emptyset$

$$\Rightarrow A_{(ij)}^* = \cancel{A_{(im)}^*} \cup a_m \cup A_{(mj)}^*$$

Dimostrazione

2) banale: per assurdo suppongo $\exists a_k \in S_{(im)}$; allora $f_i \leq s_k \leq f_k \leq s_m < f_m$
assurdo!

1) cut and paste ("exchange argument"):

sia $\overline{A_{(ij)}}$ una soluzione ottima per $S_{(ij)}$. Sia $a_k \in \overline{A_{(ij)}}$: $f_k = \min\{f_l : a_l \in \overline{A_{(ij)}}$ se $a_k = a_m$ ho finitosuppongo $a_k \neq a_m$; sia $A_{(ij)}^* = \overline{A_{(ij)}} \setminus a_k \cup a_m$ Ora basta dimostrare che $A_{(ij)}^*$ è soluzione ammissibile, visto che $|A_{(ij)}^*| = |\overline{A_{(ij)}}|$ Si osserva che $f_m \leq f_k$ e che $f_k \leq s_l \forall a_k \in \overline{A_{(ij)}}$ Quindi $f_m \leq f_k \leq s_l$, cioè a_m è compatibile con ogni a_l :

Abbiamo dimostrato la proprietà greedy.

Quindi, per risolvere $s_{(ij)}$ (nota: l'approccio è top-down)

Scritto da Gabriel

Strategia

- 1) Scegliere a_m
- 2) Risolvere S_{mj} (perchè $A_{ij}^* = \{a_m\} \cup A_{mj}^*$)

Per risolvere $S = S_{(0n+1)}$ viene invocata la soluzione solo di problemi del tipo $S_{(mn+1)} \Rightarrow$ spazio dei sottoproblemi è ridotto da quadratico a lineare: al più $n + 1$ sottoproblemi, cioè quelli che vanno da $m = 0$ ad $m = n + 1$.

```

OPT( $S_{(i\ n+1)}$ )
  if  $S_{(i\ n+1)} \neq \emptyset$  then
     $a_m : f_m = \min \{f_k : a_k \in S_{(i\ n+1)}\}$  //scelta greedy
    return  $a_m \cup \text{OPT}(S_{(m\ n+1)})$  //tail recursion  $\rightarrow$  ricorsione in coda al codice
  else
    return  $\emptyset$ 

```

L'implementazione dettagliata è la seguente:

```

REC-SEL( $S, f, i$ )
1   $n = S.length$ 
2   $m = i + 1$ 
3  while ( $m \leq n$ ) and ( $s_m < f_i$ )
4     $m = m + 1$ 
5  if  $m \leq n$ 
6    return  $\{a_m\} \cup \text{REC-SEL}(S, f, m)$ 
7  else
8    return  $\emptyset$ 

```

Complessità Modello di costo: confronti tra elementi s e f .

La complessità è $\Theta(n)$ perchè ogni attività viene coinvolta in un confronto una sola volta.

Spiegazione algoritmo greedy attività:

Versione iterativa:

```

GREEDY-SEL( $S, f$ )
1   $n = S.length$ 
2   $A = \{a_1\}$ 
3   $last = 1$  // indice dell'ultima attività selezionata
4  for  $m = 2$  to  $n$ 
5    if  $s_m \geq f_{last}$ 
6       $A = A \cup \{a_m\}$ 
7       $last = m$ 
8  return  $A$ 

```

Fase 1: ordinare le attività in ordine crescente in base al tempo di completamento.

Fase 2: selezionare la prima attività dall'array ordinato $A[]$ e scrivere l'indice per l'ultima attività selezionabile

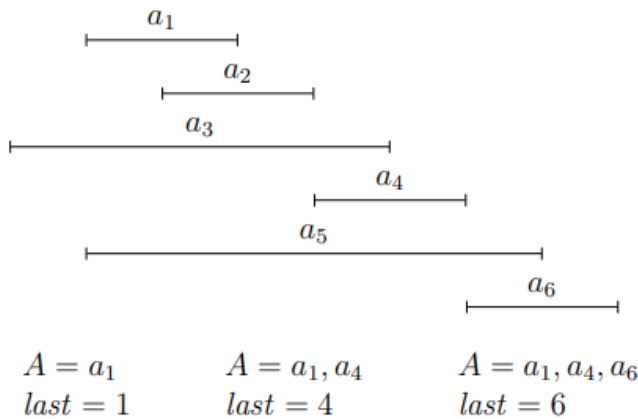
Fase 3: se l'ora di inizio dell'attività attualmente selezionata è maggiore o uguale all'ora di fine dell'attività precedentemente selezionata, aggiungerla all'array $A[]$ e salvare la posizione dell'attività selezionata

Fase 4: Procedere iterativamente e ritornare $A[]$

Esempio completo: considero le attività che finiscono prima in successione (quindi, trovo la prima attività che finisce prima e poi vado avanti trovando tutte le attività che finiscono prima, ovviamente la migliore caso per caso):

https://www.codesdope.com/staticroot/images/algorithm/greedy_activity.gif

Esempio



Paradigma generale

Si basa su 2 proprietà:

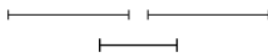
- Proprietà di scelta greedy: la soluzione ottima può essere costruita componendo scelte “ardite” (quella che può risolvere il sottoproblema, idealmente) e si dimostra col *CUT&PASTE*
- Proprietà di sottostruttura ottima con spazio dei sottoproblemi lineare: significa dimostrare che la soluzione ottima, che contiene la scelta greedy, contiene una soluzione all’unico sottoproblema da risolvere dopo aver fatto la scelta greedy

Scelte greedy alternative

Oltre alla scelta greedy vista precedentemente, ne esistono altre:

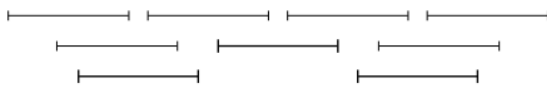
- Scegli l’attività di durata inferiore → non è ottima.

Controesempio:



- Scegli l’attività col minor numero di sovrapposizioni → non è ottima.

Controesempio:



- Scegli l’attività che inizia per prima → non è ottima.

Controesempio:



- Scegli l’attività che inizia per ultima → è ottima.

Lezione 19: Compressione dei dati e Huffman ed esempi di algoritmi greedy

La compressione può essere di due tipi:

- lossy, cioè con perdita di informazione (e.g. immagine, video);
- lossless, cioè senza perdita di informazione (e.g. testo).

Esempio File di caratteri con frequenze associate:

	a	b	c	d	e	f
frequenza (%)	45	13	12	16	9	5

La codifica ASCII usa 8 bit per carattere.

File con 100K caratteri $\rightarrow$ 800 Kbit

Definisco una codifica che usa 3 bit per carattere:

	a	b	c	d	e	f
codeword	000	001	010	011	100	101

File con 100K caratteri $\rightarrow$ 300 Kbit (risparmio più del 50%)
+ spazio per una tabella di conversione/decodifica

0	'a'
1	'b'
2	'c'
3	'd'
4	'e'
5	'f'

Nella posizione i si trova il carattere la cui codifica è i stesso (in binario).

Def

C = alfabeto dei simboli presenti nel file

funzione di encoding $e: C \rightarrow 0,1^*$

Proprietà che deve avere e :

- invertibile $\rightarrow$ iniettiva: se $a, b \in C, a \neq b \Rightarrow e(a) \neq e(b)$;
- ammettere algoritmi efficienti: e, e^{-1} .

Una funzione di encoding che associa ad ogni carattere di C una stringa di 0 e 1 della stessa lunghezza si chiama **fixed length encoding**.

L'idea di *fixed length encoding* è la più semplice possibile, ma ignora totalmente le frequenze. Posso usare questa informazione per migliorare le cose?

Idea: Variabile length encoding

Idea Associare a caratteri più frequenti codeword più corte e, di conseguenza, a caratteri meno frequenti codeword più lunghe.

	a	b	c	d	e	f
frequenza (%)	45	13	12	16	9	5
codeword	0	00	01	1	100	101

Quanto presentato è un buon encoding? No:

Problema $e(aa) = e(b)$

Non sono come decodificare in modo univoco perchè esistono delle codeword che sono prefissi di altre codeword.

Dopo aver visto il primo carattere (bit), non so se fermarmi perché ho visto una codeword completa oppure solo il prefisso di una codeword più lunga. Quindi il problema è che $\exists$ codeword che sono prefissi di altre codeword!

La codifica che sto cercando deve:

- avere lunghezza variabile della codeword;
- affinché si possa avere l'invertibilità, il codice deve essere libero da prefissi (**codice prefisso**¹), cioè $\nexists a, b \in C : e(a) \text{ è un prefisso di } e(b)$.

Soluzione

	a	b	c	d	e	f
frequenza (%)	45	13	12	16	9	5
codeword	0	101	100	111	1101	1100

Questo è un codice a lunghezza variabile libero da prefissi ed è ottimo tra tutti i codici che associano una codeword ad ogni singolo carattere.

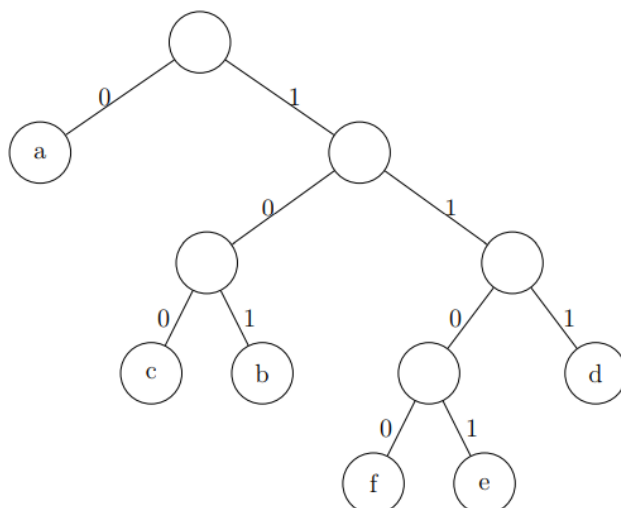
Spazio occupato da questo encoding:

$$100K \left(\frac{45}{100} \cdot 1 + \frac{13}{100} \cdot 3 + \frac{12}{100} \cdot 3 + \frac{16}{100} \cdot 3 + \frac{9}{100} \cdot 4 + \frac{5}{100} \cdot 4 \right) = 224 \text{ Kbit}$$

300 Kbit $\rightarrow$ risparmio $\approx 25\%$

Serve memorizzare dell'informazione aggiuntiva per la decodifica:

Decodifica: si usa un trie (In informatica, un *trie*, chiamato anche albero digitale o albero dei prefissi, è un tipo di albero di ricerca k-ario, una struttura di dati ad albero utilizzata per individuare chiavi specifiche all'interno di un insieme. Queste chiavi sono spesso stringhe, con collegamenti tra i nodi definiti non dall'intera chiave, ma da singoli caratteri. Per accedere a una chiave (per recuperarne il valore, modificarla o rimuoverla), il trie viene attraversato in profondità, seguendo i collegamenti tra i nodi, che rappresentano ogni carattere della chiave) che ha ogni lato etichetta con un valore binario (quindi 0 oppure 1).



Stringa codificata: 110001001101

Stringa decodificata: face

Un qualsiasi codice binario può essere rappresentato in modo compatto con un albero binario T . Se il codice è libero da prefissi, le codeword corrispondono solo alle foglie.

File = bits + "albero di codifica"

Parto dalla radice, scendo seguendo la stringa fino ad una foglia, restituisco il carattere associato alla foglia; ripeto algoritmo di decodifica (lineare sulla lunghezza della stringa).

Come stabilisco se un codice/albero è migliore di un altro?

$\forall c \in C$ ho $f(c)$ frequenza

Definisco

$$d_T(c) = \text{profondità di } c \text{ in } T$$

$$d_T('a') = 1$$

$$d_T('b') = d_T('c') = d_T('d') = 3$$

$$d_T('e') = d_T('f') = 4$$

La profondità corrisponde alla lunghezza della codeword associata.

Funzione di costo:

$$B(T) = \sum_{c \in C} f(c) \cdot d_T(c)$$

La dimensione del file compresso è

$$|F_c| = \frac{B(T) \cdot |F|}{100}$$

$\Rightarrow B(T)$, a meno di una costante (100) è l'inverso del fattore di compressione: minore è $B(T)$ e migliore è il fattore di compressione.

(Osservazione: stiamo assumendo di dover associare una codeword a ogni singolo carattere, il che non è sempre la cosa migliore da fare:

$a b b a b b a b b a b b \dots$

cioè vedremo il codice libero da prefissi ottimi tra tutti i codici che associano una codeword a ogni singolo carattere)

Proprietà Un albero ottimo è sempre pieno (cioè i nodi interni hanno due figli).

Spazio di problemi: spazio di alberi pieni con n foglie ($n = |C|$)

Obiettivo: trovare un albero ottimo T , cioè che minimizza $B(T)$

Idea: Partiamo dai nodi con frequenza minore (per esempio a, b, c ; eseguiamo un primo *merge* (unione) e il loro nodo radice rappresenta la somma dei nodi (per esempio, z rappresenta la somma dei nodi a, c).

Ulteriormente, eseguiamo un altro merge andando verso l'alto (più si va verso l'alto, più aumenta la frequenza dei caratteri (esempio: eseguiamo il merge di z e di b ottenendo z' che rappresenta la loro somma). Così via fino alla fine.

Prendo due nodi con frequenza minore, gli unisco in un nodo che avrà come frequenza la somma delle due, ripeto $n - 1$ volte. In questo modo, i nodi con la frequenza maggiore verranno aggiunti per ultimi alla cima dell'albero, quindi avranno una profondità bassa, ovvero la lunghezza dalla codeword corta, proprio come volevo.

Sottoproblemi = Alberi su meno caratteri

Scelta greedy = Merge tra nodi che hanno frequenza minore

Q : coda di priorità (Heap) di nodi con chiave $f(z)$

Attributi di un nodo:

- $z.left$
- $z.right$
- $z.f$

Pseudocodice

HUFFMAN(C, f)

```

1   $n = |C|$ 
2   $Q = \emptyset$ 
3  for each  $c \in C$  // inizializzazione
4       $z = \text{NEW-NODE}()$  // crea un nuovo nodo
5       $z.f = f(c)$ 
6       $z.left = \text{NIL}$ 
7       $z.right = \text{NIL}$ 
8       $\text{INSERT}(Q, z)$  //  $\Theta(\log n)$ 
9  for  $i = 1$  to  $n - 1$ 
10      $x = \text{EXTRACT-MIN}(Q)$ 
11      $y = \text{EXTRACT-MIN}(Q)$ 
12      $z = \text{NEW-NODE}()$ 
13      $z.left = x$ 
14      $z.right = y$ 
15      $z.f = x.f + y.f$ 
16      $\text{INSERT}(Q, z)$ 
17 return  $\text{EXTRACT-MIN}(Q)$ 
```

Passi descritti a lato:

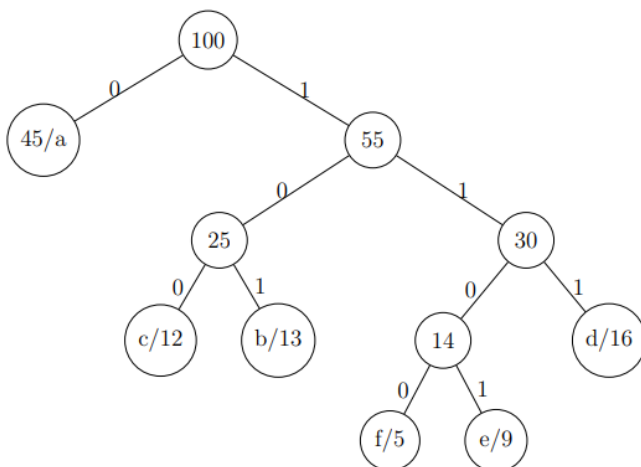
- creare una coda prioritaria Q composta da ogni carattere unico.
- ordinarli in ordine crescente di frequenza.

Per tutti i caratteri unici:

1. creare un nuovo nodo
2. estrarre il valore minimo da Q e assegnarlo al figlio sinistro del nuovo nodo
3. estrarre il valore minimo da Q e assegnarlo al figlio destro del nuovo nodo.
4. calcolare la somma di questi due valori minimi e assegnarla al valore del nuovo nodo
5. inserire questo nuovo nodo nell'albero
6. restituire il nodo radice

Complessità $\Theta\left(\sum_{i=1}^{n-1} \log(n-i)\right) = \Theta(n \log n)$

Esempio Riprendiamo l'esempio iniziale e applichiamo l'algoritmo.



	a	b	c	d	e	f
codeword	0	101	100	111	1101	1100

$$B(T) = \left(\frac{45}{100} \cdot 1 + \frac{13}{100} \cdot 3 + \frac{12}{100} \cdot 3 + \frac{16}{100} \cdot 3 + \frac{9}{100} \cdot 4 + \frac{5}{100} \cdot 4 \right) = 224$$

Scritto da Gabriel

Proprietà di scelta greedy

Sia C un alfabeto e siano x e y i caratteri in C di frequenza minore.
Allora esiste un codice prefisso ottimo T in cui x e y sono foglie attaccate allo stesso padre (sorelle).

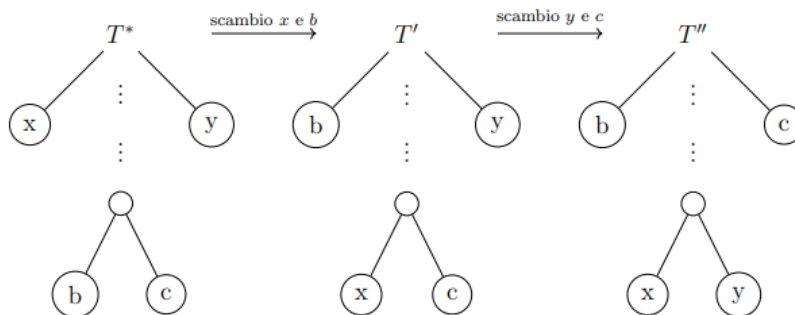
Dimostrazione

Sia T^* una soluzione ottima arbitraria $\Rightarrow B(T^*)$ è minima.

Non so dove siano x e y in T^*

Siano b e c le foglie sorelle di profondità massima in T^*

1. $d_{T^*}(b) = d_{T^*}(c) \geq d_{T^*}(x), d_{T^*}(y)$
2. $f(x) \leq f(b), \quad f(y) \leq f(c)$



$T^* \rightarrow T'$

$$B(T^*) \geq B(T')$$

$$B(T^*) - B(T') \geq 0$$

$$\begin{aligned} B(T^*) - B(T') &= \sum_{c \in C} f(c) d_{T^*}(c) - \sum_{c \in C} f(c) d_{T'}(c) \\ &= f(b) d_{T^*}(b) + f(x) d_{T^*}(x) - f(b) \underbrace{d_{T'}(b)}_{d_{T^*}(b)} - f(x) \underbrace{d_{T'}(x)}_{d_{T^*}(x)} \\ &= f(b) d_{T^*}(b) + f(x) d_{T^*}(x) - f(b) d_{T^*}(b) - f(x) d_{T^*}(x) \\ &= f(b)(d_{T^*}(b) - d_{T^*}(x)) - f(x)(d_{T^*}(b) - d_{T^*}(x)) \\ &= \underbrace{(f(b) - f(x))}_{\geq 0 \text{ perchè } f(x) \leq f(b)} \underbrace{(d_{T^*}(b) - d_{T^*}(x))}_{\geq 0 \text{ per il punto 1}} \geq 0 \end{aligned}$$

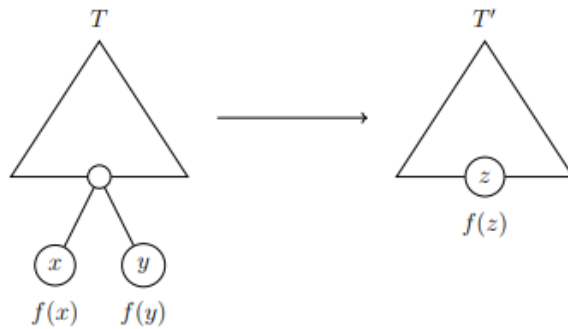
$T' \rightarrow T''$ si dimostra analogamente.

Proprietà di sottostruttura ottima

Sia T un codice prefisso ottimo che contiene la scelta greedy (sui caratteri x e y).

Sia z un nuovo carattere (cioè $z \notin C$) con $f(z) = f(x) + f(y)$.

Allora il codice prefisso $T' = T \setminus \{x, y\}$ è ottimo per $C \setminus \{x, y\} \cup \{z\}$



T ha n foglie.

T' ha $n - 1$ foglie.

Cerco una relazione tra $B(T)$ e $B(T')$.

Osservazione

1. $\forall c \in C \setminus \{x, y\}, c \neq z : d_{T'}(c) = d_T(c)$
2. $d_{T'}(z) = d_T(x) - 1$

Dimostrazione

$$\begin{aligned}
 B(T) &= \sum_{c \in C} f(c) d_T(c) & (d_T(x) = d_T(y)) \\
 &= \sum_{c \in C \setminus \{x, y\}} f(c) d_T(c) + (f(x) + f(y)) d_T(x) \\
 &= \sum_{c \in C \setminus \{x, y\}} f(c) d_T(c) + \underbrace{(f(x) + f(y))}_{f(z)} \underbrace{(d_T(x) - 1)}_{d_{T'}(z)} + (f(x) + f(y)) \\
 &= \sum_{c \in C \setminus \{x, y\} \cup \{z\}} f(c) d_{T'}(c) + f(x) + f(y) \\
 &= B(T') + f(x) + f(y)
 \end{aligned}$$

Suppongo per assurdo che T non sia il codice prefisso ottimo per il sottoproblema $C \setminus \{x, y\} \cup \{z\}$.

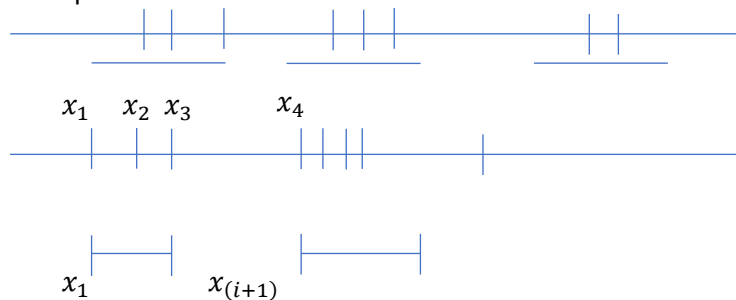
Allora $\exists T''$ di costo strettamente inferiore.

$$B(T) = \underbrace{B(T'')}_{< B(T')} + f(x) + f(y)$$

Assurdo!

Esercizio: Sia $X = x_1, x_2, \dots, x_n$ un insieme di punti ordinato non decrescente sulla retta reale. Si fornisca un algoritmo greedy che determini un insieme I di cardinalità minima di intervalli chiusi di ampiezza unitaria ($\{a, b\} \in I \Rightarrow b - a = 1$) tale che $\forall x_i \in X \exists i \in I, i = [a, a + 1]$, tale che $x_i \in i$.

Esempio:



L'algoritmo è così strutturato:

```

MIN_COVER (X)
  n ← length(X)
  last ← 1
  C ← [X1, X1 + 1]
  for i ← 2 to n
    if xi > x(last) + 1
      last ← i
      C ← {{xi, x(i + 1)} ∪ C
  return C
    
```

Proprietà di scelta greedy

Esiste un insieme ottimo di intervalli di copertura per X che contiene la scelta greedy $[X_1, X_1 + 1]$

Dimostrazione

Sia I^* soluzione ottima, $[a, a + 1] \in I^*$ un intervallo che copre x_1 , $a \leq x_1 \leq a + 1$ così:

- 1) $a = x_1$
- 2) $a \neq x_1$
 $\Rightarrow a < x_1$
 $\Rightarrow$ tutti i punti coperti da $[a, a + 1]$ sono anche coperti dalla scelta greedy $[x_1, x_1 + 1]$ (infatti, se x_j è coperto da $[a, a + 1]$ allora $a < x_1 \leq x_j \leq a + 1 < x_1 + 1$.

Allora considero $I' = I^* \setminus [a, a + 1] \cup [x_1, x_1 + 1]$

I' copre tutti i punti e $|I'| = |I^*| \Rightarrow I'$ è una soluzione ottima che contiene la scelta greedy

Proprietà di sottostruttura ottima

Sia I una soluzione ottima che contiene la scelta greedy. Allora $I \setminus [x_1, x_1 + 1]$ è soluzione ottima del sottoproblema $X \setminus \{x_j \in X: x_j \leq x_1 + 1\} \rightarrow A_1$

Dimostrazione

I punti $X \setminus A_1$ non possono essere coperti dalla scelta greedy.

Considero $I \setminus [x_1, x_1 + 1]$: per il punto sopra copre tutti i punti in $X \setminus A_1 \Rightarrow I \setminus [x_1, x_1 + 1]$ è soluzione ammissibile per $X \setminus A_1$. Suppongo per assurdo che non sia ottima per $X \setminus A_1$: prendo la soluzione con meno intervalli, ci aggiungo la scelta greedy, ottenendo una soluzione I^* per il problema originale che ha meno intervalli: assurdo perché I è ottimo.

Esercizio → metric matching on the line

Sia $S = s_1, s_2, \dots, s_n$ un insieme di punti ordinati sulla retta reale, rappresentanti dei server.

Sia $C = c_1, c_2, \dots, c_n$ un insieme di punti ordinati sulla retta reale, rappresentanti dei client.

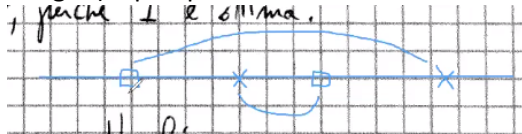
Il costo di assegnare un client c_i ad un server s_j è $|c_i - s_j|$. Si fornisca un algoritmo greedy che assegna ogni client ad un server distinto e che minimizzi il costo totale dell'assegnamento.

Soluzione

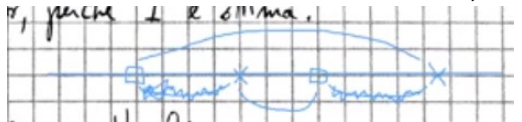
Si vuole minimizzare la somma degli assegnamenti:

- Cerco la coppia client-server più vicina e ripeto (non funziona)

Vediamo l'esempio di una coppia di client, dove si assegna la coppia più vicina (quella interna nel disegno) e poi quella esterna:



La soluzione ottima invece farebbe così, cioè assegna client/server a coppie:



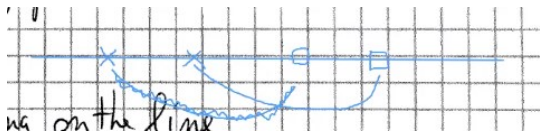
Il numero di client e di server è sempre n , quindi ogni client sarà assegnato ad un server.

Cosa si fa: il primo client a sx viene assegnato al primo server a sx.

È un algoritmo ottimo, nonostante non consideri le distanze.

Semplicemente, prende il primo client che trova e lo matcha con il primo server che trova.

Anche con la tecnica del *cut&paste*, si ha che la soluzione ottima (quella non tratteggiata) rimane quella buona, in quanto comunque il costo di assegnazione rimane $\leq$ al precedente.



Alla luce di questo, l'algoritmo greedy è il seguente:

METRIC - MATCHING(S, C)

$n = C.length$

$A = C[1] - S[1]$ # assegno la prima coppia client - server

$last = 1$

for $i = 2$ to n

for $j = 1$ to $n - 1$

if $C_i - S_j > C_{(last)} - S_{(last)} + 1$

$last = i$ # assegno il client i - esimo al server i - esimo

$A = [C_i - S_j] \cup A$

return A

Confronto programmazione dinamica e greedy

Nel caso degli algoritmi bottom up e relative ricorrenze, nelle immagini sono illustrati i passaggi generici di programmazione dinamica nel caso Iterativo (prima immagine) e caso Ricorsivo (seconda immagine).

```
dp[]
funzione(param)
    if dp[param]
        return dp[param]

    for <condizione caso base>
        // setta dp con i casi base

    for <condizione caso ricorsivo>
        // setta dp con la formula ricorsiva

    return dp[param]
```

```
dp[]
funzione(param)
    if dp[param]
        return dp[param]

    if caso base
        return <caso base>

    else
        dp[param] = <parte ricorsiva>

    return dp[param]
```

Si consideri in particolare che essendo un algoritmo bottom up, i costi sono sempre dati da una percorrenza inversa della matrice rispetto al senso di andata.

Cosa vuol dire? Se per esempio devo calcolare una LCS so per certo che, muovendomi nella matrice dall'alto in basso nella fase di andata per controllare dove si ha avuto un match, la fase di ritorno ragiona considerando gli indici precedenti ($i-1, j-1$), questo perché si è già calcolata la soluzione ottima, che detto in termini normali significa semplicemente che ho già calcolato con il costo ottimo quello che devo fare.

Sotto riassunto il caso LCS:

$$l(i, j) = |LCS(X_i, Y_j)|$$

$$l(i, j) = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 & (\text{caso 0}) \\ l(i-1, j-1) + 1 & \text{se } i, j > 0 \text{ e } x_i = x_j & (\text{caso 1}) \\ \max\{l(i, j-1), l(i-1, j)\} & \text{se } i, j > 0 \text{ e } x_i \neq x_j & (\text{caso 2}) \end{cases}$$

Alla fine ci interessa calcolare $l(m, n)$.

Per calcolare $l(i, j)$ mi possono servire tre valori:

$$L = \begin{bmatrix} (i-1, j-1) & (i-1, j) \\ (i, j-1) & \leftarrow (i, j) \end{bmatrix}$$

Scansione "row-major": riempio la tabella per righe, da sinistra a destra.

Si noti che la tabella è una struttura che viene sempre utilizzata; possono essere diversi sensi di scansione, per righe (*row major*), per colonne (*column major*) o al contrario (*reverse column major/reverse row major*). Dall'immagine si vede la scansione (in reverse i numeri vanno semplicemente invertiti).

Row-major	Column-major
0 1 2 3	0 4 8 12
4 5 6 7	1 5 9 13
8 9 10 11	2 6 10 14
12 13 14 15	3 7 11 15

Si ricordi che le ricorrenze considerano i casi

- $i=0, j=0$, nel qual caso il costo è sempre 0
- nel caso il nostro indice oltrepassi la grandezza dell'input, allora si pone ad infinito
- nel caso il nostro indice arrivi ad essere uguale alla taglia dell'input ($n-1$) o 0, sarà una costante

Scritto da Gabriel

- quando si hanno $i, j > 0$ ma $X_i = X_j$ si ha il min o max (dipende dal problema se min o max ovviamente) degli indici calcolati
- quando si hanno $i, j > 0$ ma $X_i \neq X_j$ si ha min o max + la soluzione ottima calcolata

In alcuni casi come costo si somma anche +1 o +2 alla soluzione ottima calcolata; ciò dipende dal tipo del problema e generalmente si fa proprio perché uno dei casi basi è la costante 1 oppure 2 oppure per non considerare nullo l'input (come il caso degli alberi dove si somma +1 perché si può avere output nullo dopo la ricorsione a sinistra oppure a destra).

Per riassumere queste considerazioni, prendiamo il caso del calcolo di costi in una scacchiera, come:

i. La caratterizzazione ricorsiva può essere:

$$G[i, j] = \begin{cases} 0 & \text{se } i = m + 1 \\ -\infty & \text{se } j = n + 1 \\ \max\{V[i, j] + G[i + 1, j], G[i, j + 1]\} & \text{altrimenti} \end{cases}$$

Si vedano i valori commentati che rispecchiano quanto è stato descritto. Si veda anche che il movimento che noi compiamo all'interno della scacchiera è sempre dall'alto al basso; quindi, posso avere il caso dove si muove la riga/la colonna insieme e ne calcolo il costo massimo.

Al di là del senso di scansione, generalmente negli algoritmi di programmazione dinamica si ha sempre:

- un ciclo di inizializzazione, in uno o due indici che sistema i casi base (pongo una matrice o array a 3 indici come 0, +infty, o similari. Il ciclo di inizializzazione serve a capire se ho già calcolato una particolare soluzione ottima.

In particolare, posso definire una situazione similare alla precedente, introducendo il controllo struttura attuale e parametro che devo calcolare:

```
funzione(par):
    if(DP[par] != "SEGNAPOSTO VALORE ASSENTE")
    {
        DP[par] = *chiamata ricorsiva di funzione(par2) su
        par2 != par*
    }
    return DP[par]
```

edited 18:37

- un ciclo di calcolo della soluzione ottima, sempre basato su confronti (minore o maggiore)
- ritorno della soluzione ottima

Si intende quindi come ottimizzazione significhi aver già calcolato in modo "efficiente" una certa soluzione, ciclando il numero strettamente necessario di volte in base ai limiti di indici individuati.

Algoritmi semplici (per davvero)

Nel caso greedy invece, l'algoritmo si basa normalmente sulla scelta di un indice, o primo o ultimo. L'algoritmo greedy ricordiamo che fa una scelta degli input ottimali, in maniera tale da minimizzare il numero di confronti e risolvere facilmente un determinato sottoproblema. Ad esempio nella selezione delle attività:

```
GREEDY-SEL( $S, f$ )
1   $n = S.length$ 
2   $A = \{a_1\}$ 
3   $last = 1$  // indice dell'ultima attività selezionata
4  for  $m = 2$  to  $n$ 
5      if  $s_m \geq f_{last}$ 
6           $A = A \cup \{a_m\}$ 
7           $last = m$ 
8  return  $A$ 
```

I passi si ripetono sempre fondamentalmente identici, in particolare si sceglie un punto particolare del nostro input e ciclo facendo un confronto.

Nel caso invece dovessi avere un intervallo di valori devo sempre considerare se è contiguo (caso intervallila cui differenza è 1, sotto riportato) o è un intervallo minimo, nel qual caso devo tenere un indice in più *first* controllando in maniera identica agli algoritmi presenti.

```
MIN_COVER( $X$ )
 $n \leftarrow \text{length}(X)$ 
 $last \leftarrow 1$ 
 $C \leftarrow \{[x_1, x_1+1]\}$ 
for  $i \leftarrow 2$  to  $n$  do
    if  $x_i > x_{last}+1$  then
         $last \leftarrow i$ 
     $C \leftarrow \{[x_i, x_i+1]\} \cup C$ 
return  $C$ 
```

Chiarimento sul calcolo esatto delle somme nelle serie

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i+2}^n 2 = 2 \sum_{i=1}^{n-2} n - i - 1 = 2 \sum_{k=1}^{n-2} k = (n-2)(n-1)$$

```
for i=1 to n-2 do
  for j=i+2 to n do
    M[i,j] = 0
```

Spiegazione:

La sommatoria interna è costituita da soli 2, in quanto richiede due moltiplicazioni tra tre numeri interi, nello specifico $(n-2)$, $(n-1)$, n .

Quanti due?

Beh, da $j = i + 2$ a n ci sono esattamente $n - (i + 2) + 1 = n - i - 1$ termini (sostituisco j in i , perché la serie è basata su i).

Calcolo:

$$\begin{aligned} T(n) &= \sum_{i=1}^{n-2} \sum_{j=i+2}^n 2 \\ &= 2 \sum_{i=1}^{n-2} \sum_{j=i+2}^n 1 \\ &= 2 \sum_{i=1}^{n-2} \left[\sum_{j=1}^n 1 - \sum_{j=1}^{i+1} 1 \right] \\ &= 2 \sum_{i=1}^{n-2} (n - (i+1)) \\ &= 2 \sum_{i=1}^{n-2} n - i - 1 \\ &= 2 \sum_{k=1}^{n-2} k = (n-2)(n-1) \end{aligned}$$

Quindi sarebbe sommatoria di $2(n - i - 1)$, poi si può portare fuori il 2 per linearità.

Poi sostituire con k accorgendoti che sono esattamente gli stessi termini della sommatoria, se provi a svilupparli, e l'ultima sommatoria la puoi riscrivere in quel modo, ricordandoti che la somma di $1 \dots n$ in generale è $n(n+1)/2$ quando si abbia come qui la serie con termine generale k (serie di Gauss).

Infatti, la sommatoria da 1 a $n-2$ è $(n-2)(n-2+1)/2$, semplificando diventa quello che vedi

Il 2 come argomento della serie deriva dal fatto che ogni termine $M[i, j]$ richiede due moltiplicazioni tra 3 numeri interi.

In altri casi, che dettagliamo ciascuno:

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i+2}^n 1 = \sum_{i=1}^{n-2} n - i - 1 = \sum_{k=1}^{n-2} k = (n-2)(n-1)/2$$

```
for i=1 to n-2 do
  for j=i+2 to n do
    M[i,j] = 0
```

Spiegazione:

La sommatoria interna è costituita da soli 1, in quanto richiede una moltiplicazione tra tre numeri interi, nello specifico $(n-2)$, $(n-1)$, n .

Quanti uno?

Beh, da $j = i + 2$ a n ci sono esattamente $n - (i + 2) + 1 = n - i - 1$ termini (sostituisco j in i , perché la serie è basata su i e il +1 si ha per il fatto che $i = 1$).

Algoritmi semplici (per davvero)

Poi sostituire con k accorgendoti che sono esattamente gli stessi termini della sommatoria, se provi a svilupparli, e l'ultima sommatoria la puoi riscrivere in quel modo, ricordandoti che la somma di $1 \dots n$ in generale è $n(n+1)/2$.

Non avendo il termine 2 per linearità della sommatoria, non viene moltiplicato con il "fratto 2" di $(n-2)(n-1)$ e quindi il risultato è proprio $(n-2)(n-1)/2$.

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i}^{n-2} 1 = \sum_{i=1}^{n-2} n-1-i = \sum_{k=1}^{n-2} k = (n-1)(n-2)/2.$$

Spiegazione:

La sommatoria interna è costituita da soli 1, in quanto richiede una moltiplicazione tra due numeri interi, nello specifico $(n-2)$, $(n-2)$ (apici della serie).

```
for i=1 to n-2 do
  for j=n-2 downto i do
    C[i,j] <- C[i-1,j] * C[i,j+1]
    M <- MAX(M,C[i,j])
  for i=1 to n-2 do
    for j=n-2 downto i do
      C[i,j] <- C[i-1,j-1] * C[i,j+1]
      m <- MIN(m,C[i,j])
```

Quanti uno?

Beh, da $j = n-2$ a i ci sono esattamente $n-i-1 = n-i-1$ termini (sostituisco j in i , perché la serie è basata su i e sottraggo 1 in quanto $i=1$ ed i sarebbe il valore di j ($j=i$ è il pedice della seconda serie)).

Poi sostituire con k accorgendoti che sono esattamente gli stessi termini della sommatoria, se provi a svilupparli, e l'ultima sommatoria la puoi riscrivere in quel modo, ricordandoti che la somma di $1 \dots n$ in generale è $n(n+1)/2$ quando si abbia come qui la serie con termine generale k (serie di Gauss).

Non avendo il termine 2 per linearità della sommatoria, non viene moltiplicato con il "fratto 2" di $(n-2)(n-1)$ e quindi il risultato è proprio $(n-2)(n-1)/2$.

$$\begin{aligned} T_z(n) &= \sum_{j=1}^{n-1} \sum_{i=2}^n 2 \\ &= \sum_{j=1}^{n-1} 2(n-1) \\ &= 2(n-1)^2. \end{aligned}$$

```
for j=n-1 downto 1 do
  for i=2 to n do
    z[i,j] = z[i-1,j] * z[i,j+1] * z[i-1,j+1]
  for j=n-1 downto 1 do
    for i=2 to n do
      c[i,j] = c[i-1,j] * c[i,j+1] * c[i-1,j+1]
```

Spiegazione:

La sommatoria interna è costituita da soli 2, in quanto richiede due moltiplicazioni tra due numeri interi, nello specifico $(n-1)$ ed n (apici della serie).

Quanti uno?

Beh, da $i=2$ ad n . Quindi, dovendo sostituire i in j (essendo la serie basata sull'indice più esterno), si ha che $n-1$ semplicemente (in quanto, l'indice j è pari ad 1 e il 2 verrebbe portato poi fuori per linearità).

Cioè, si esegue implicitamente la seguente cosa:

$$\sum_{j=1}^{n-1} 2(n-1) = 2 \sum_{j=1}^{n-1} (n-1)$$

Ora, siccome abbiamo una serie che ha come apice $(n-1)$, non possiamo esprimere la soluzione in questo modo, dato che $(n-1)$ compare sia a pedice che dentro la serie. Dobbiamo quindi esprimere la serie in termini di j . Ciò comporta che io vada a "portare dentro" $(n-1)$ nella serie, in quanto moltiplicheremmo implicitamente $2(n-1)$ per altre $(n-1)$ volte e quindi è come se andassi a fare

$$2(n-1)(n-1) = 2(n-1)^2$$